

RETI DI CALCOLATORI

A.A 2025 / 2026 – 12 CFU

Indice

[Introduzione](#)

[Reti di Reti – Internetworking](#)

[Livello Applicazione](#)

[Sicurezza in Rete](#)

[Network Layer](#)

[Wireless](#)

INTRODUZIONE

Una rete di calcolatori è un insieme di dispositivi autonomi ed interconnessi, in grado di eseguire e svolgere autonomamente i compiti di calcolo e comunicazione.

Classificazione delle Reti

Le reti di calcolatori si possono classificare in base alla loro dimensione geografica:

- PAN: Personal Area Network, verosimilmente la propria scrivania
- LAN: Local Area Network, come l'ufficio, un laboratorio, casa propria...
- MAN: Metropolitan Area Network che comprendono tutte le reti urbane, metropolitane e civiche
- WAN: Wide Area Network che identifica la connessione di aree ampie ed estese, come la reti nazionali ed internazionali

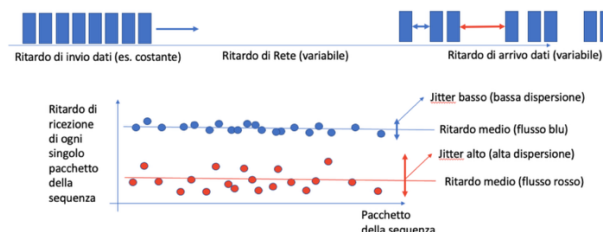
Internet, invece è definita come una rete di reti realizzata attraverso protocolli del tipo TCP/IP, protocolli che garantiscono la navigazione simultanea nascondendo eterogeneità.

Prestazioni

Uno degli aspetti cruciali della rete è proprio la sua prestazione che è misurabile in:

- **Capacità di trasmissione:** numero di bit trasmessi e ricevuti ogni secondo
- **Ritardo del collegamento:** tempo richiesto ai dati per transitare da mittente a destinatario. Il ritardo del collegamento può essere determinato sia dalla distanza fisica, ma anche dai tempi di gestione relativi ai protocolli di comunicazione. La latenza, ovvero il ritardo dipende dalla complessità di instradamento dei pacchetti in quanto la fibra (luce) non subisce ritardi per fattori esterni.

Il **Jitter** è la varianza sul ritardo di collegamento. Per chiarire, i pacchetti partono tutti in un dato istante, secondo dopo secondo (in base alla capacità di trasmissione), ma il loro ritardo non può essere stabile. Il Jitter è la varianza attorno al ritardo medio del collegamento. Più è alto il Jitter più memoria all'interno di un buffer verrà usata, comportando un ritardo notevole nella visualizzazione di una trasmissione in diretta.



Il RTT (Round Trip Time), invece è il tempo impiegato da un'informazione a partire dal mittente, arrivare al server e tornare indietro al mittente.

Componenti del Calcolatore per la Rete

La connessione di un calcolatore a una rete di calcolatori richiede un insieme essenziale di componenti, sia hardware che software, in aggiunta al calcolatore di base.

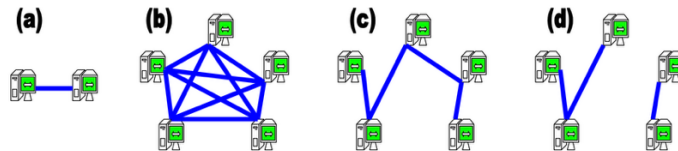
- **Scheda di rete:** dispositivo hardware di comunicazione che è in grado di codificare le trasmissioni e decodificare le ricezioni dal calcolatore alla rete e viceversa. Hardware fondamentale per l'interazione tra il computer e l'infrastruttura di rete.
- **Connettore di rete:** connettore per collegamento del calcolatore all'interno della rete (RJ45).
- **Mezzo di trasmissione:** supporto fisico alla propagazione e trasmissione dei segnali
- **Protocolli di rete:** software che implementa le regole di comunicazione all'interno della rete per garantire compatibilità e correttezza delle informazioni. Parleremo di TCP/IP



Collegamenti e Infrastrutture

L'infrastruttura di rete è la connessione dei collegamenti della rete tra nodi e host. Nella rete identifichiamo diverse infrastrutture, le quali sono così identificate:

- **Punto – Punto:** due nodi collegati solamente tra loro con una connessione diretta dove l'informazione può transitare in modo **half-duplex** (da A a B) o **full-duplex** (contemporaneamente da A a B (es upload) e B ad A (es download)). Le backbone sono full-duplex.
- **Completamente connesse:** più di due nodi collegati tutti tra loro, dando vita a $n-1$ collegamenti distinti
- **Parzialmente connesse:** Un unico collegamento che attraversa tutti i nodi
- **Partizioni di rete:** Collegamento spaccato che crea due isole di nodi connessi

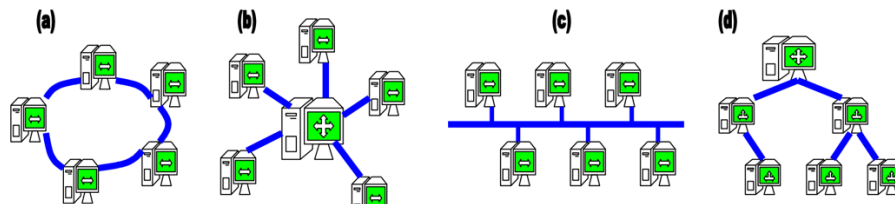


Le infrastrutture di rete possono assumere vari schemi di connessione (**topologia**). Le topologie sono di vario tipo, ma nessuna di queste è a sé stante nel modo reale, questo perché in internet compaiono delle topologie ibridate, ovvero miscugli delle diverse topologie qui sotto definite:

- **Anello:** ogni componente può comunicare con ogni altro componente inviando segnali attraverso le connessioni comunicando in senso orario o antiorario. Un anello capace di comunicare sia in senso orario che in senso antiorario permette di accorciare il cammino medio dei pacchetti. Se n è il numero di nodi $\#Path = n/2$. Questo metodo è ridondante (perché si hanno n collegamenti, e in una rete completamente connessa ne basterebbero $n-1$), ma ciò garantisce una situazione di ricaduta ottimale, infatti la rete con la mancanza di un nodo non si rompe, ma si chiude (automaticamente) su sé stessa e continua a gestire una rete ad anello connessa.
- **Stella:** ogni componente può solo comunicare con qualsiasi altro nodo periferico passando dal nodo centrale, il nodo centrale è il predominante e più importante. Il grado della rete è $n-1$ e qualora dovesse essere tagliato un collegamento si avrebbe già una partizione di rete. Ma se il nodo centrale (Hub) muore, allora il sistema è in panne (Single Point of Failure). In realtà questa topologia è la più usata perché sia ha un collegamento punto – punto per ogni nodo della rete, quindi si ha un grade vantaggio costo-efficienza. Tuttavia, l'Hub ogni volta che riceve un segnale lo reinoltra a tutti gli altri host, ma se due host nello stesso istante comunicano un'informazione il messaggio per il destinatario è compromesso e distrutto, ciò viene finito **dominio di collisione***. Ad oggi l'elemento centrale non è più un Hub, ma uno Switch, che invece di inondare a tutti l'informazione, guarda la strada (l'indirizzo all'interno del pacchetto, detto frame) del destinatario e inoltra l'informazione sulla strada corretta. Lo Switch a differenza dell'Hub è quindi selettivo garantendo comunicazione migliore e una privacy. Il bello dello Switch è che si auto compone la struttura della rete ed indirizza il pacchetto verso i collegamenti solo leggendo l'indirizzo Mac della scheda di rete del mittente e del destinatario. Quindi lo Switch ogni volta che deve comunicare oltre al pacchetto da trasmettere riceve sia l'indirizzo della scheda di rete (Mac) del mittente che del destinatario. Lo Switch riesce a compiere ciò perché dentro di sé ha una tabella che specifica come sono composti i collegamenti. Infine, lo Switch quando consegna il pacchetto trasmette un'informazione di acknowledgment al mittente per comunicare il successo della comunicazione.
- **Bus:** ogni componente è collegato in una dorsale che a sua volta è collegata con tutti i nodi. In questa topologia sono molto frequenti le collisioni e la sovrapposizione di trasmissioni. In questa

topologia è molto semplice compromettere le informazioni se due host parlano contemporaneamente. Topologia Broadcast, no switch o hub, solo tanti host interconnessi.

- **Albero:** ogni componente segue un ordine gerarchico. È la tipologia di rete che accorcia i cammini.



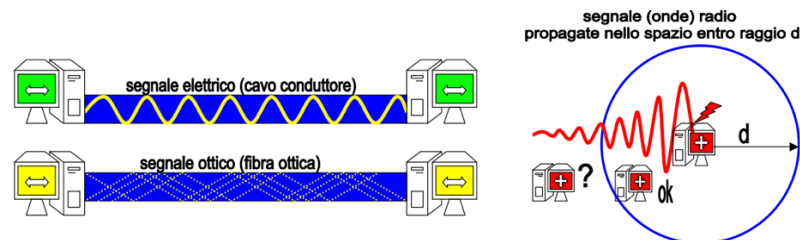
Internet ha una tipologia ibrida, detta a maglia.

***Dominio di collisione:** insieme di tutti i nodi che trasmettono contemporaneamente compromettono la comunicazione di ogni ricevitore potenzialmente interessato. Questo deve essere sistemato da un protocollo di comunicazione.

Mezzo Fisico di Trasmissione

Il mezzo fisico di trasmissione è quello che abbiamo sempre rappresentato in blu, che però può essere di diverse tipologie:

- **Cavi di Rame:** trasmettono segnali elettrici, sono affidabili, costano poco a discapito di un leggero disturbo. L'esempio più lampante di un cavo di rame è il cavo ethernet
- **Fibre Ottiche:** consentono il passaggio della luce all'interno di una fibra di vetro. Costano molto, funzionano bene su lunghe distanze e non lascia spazio a disturbi, il segnale che esce da dove entrato è immutato.
- **Wireless:** trasmissione di onde radio, non necessitano un mezzo fisico, si propagano anche nel vuoto, ma l'intensità dell'onda si riduce di molto dopo metro, rendendola il mezzo più versatile, ma più limitato.

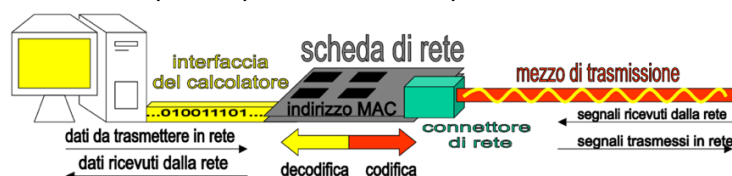


Senza Hub, Switch, backbone con Router, o amplificatori ~200Mt prima del fading cavo ethernet

Dispositivo o Schede di Rete

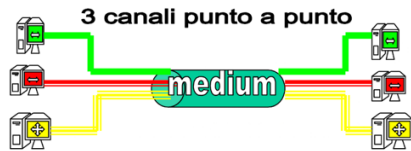
Il dispositivo, o scheda di rete è un componente dell'architettura di un calcolatore che permette al dispositivo di comunicare con altri nodi all'interno della rete. La scheda di rete dispone di un connettore di rete che permette di innestare il mezzo di trasmissione direttamente alla scheda. Da notare, che il mezzo di trasmissione può essere sia fisico che wireless, di conseguenza il connettore può essere di diverse tipologie. La scheda di rete, quindi ha il compito di trasmettere e ricevere i dati opportunamente codificati (dal calcolatore) e decodificati (dalla rete).

Inoltre, ogni scheda di rete ha un **indirizzo MAC** (48 bit di valore esadecimale) che permette di individuare univocamente un calcolatore all'interno della rete, identificando l'identità del destinatario e del mittente, rendendo molto più facile la comunicazione, specialmente attraverso l'uso di switch. L'indirizzo MAC è esadecimale in quanto permette di compattare i 48 bit.



Canali di Comunicazione di Rete

È possibile considerare il supporto fisico di un mezzo di trasmissione suddiviso in diversi canali di comunicazione separati. Ogni canale diverso può essere considerato un tubo virtuale sul quale possono essere trasmessi i bit di informazione. Il mezzo di trasmissione può quindi essere considerato come un fascio di canali di comunicazione.

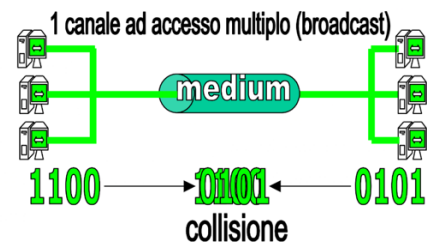


Nell'immagine riportata a lato vediamo un unico mezzo trasmissivo (immaginiamo sia fibra) entro il quale sono realizzati tre diversi canali che connettono tre coppie di stazioni punto a punto. Tutti i canali in figura passano attraverso l'unico mezzo trasmissivo (la fibra). Questo concetto si basa sull'accordo tra un mittente e un destinatario riguardante la definizione del canale

da usare (nella figura riportata il colore). In sintesi, da un unico mezzo trasmissivo, abbiamo ottenuto tre canali punto a punto che lavorano grazie ad accordi tra mittente e destinatario, questo significa che ogni calcolatore riceverà le informazioni di qualsiasi colore, ma scarterà le informazioni che non appartengono al colore (:= accordo) da lui sottoscritto.

Possiamo applicare ciò anche ad un circuito digitale, con il medesimo risultato: solo alcune frequenze verranno lette ed interpretate, le altre saranno immediatamente scartate.

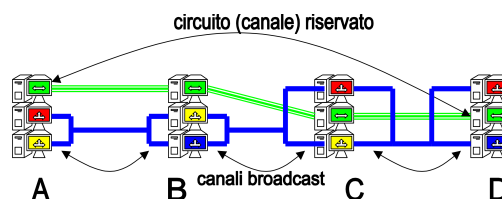
Nella figura di destra, lo stesso mezzo trasmissivo condiviso consente la creazione di un unico canale ad accesso multiplo (broadcast), che collega tre stazioni verdi sulla sinistra e tre stazioni verdi sulla destra, dove due sequenze di bit, contemporaneamente trasmesse sullo stesso canale danno luogo ad una collisione che distrugge l'informazione trasmessa. I broadcast sono canali sui quali tutti possono trasmettere e ricevere le trasmissioni. Il problema principale, come si evince dalla figura è legato alla possibile collisione di segnali appartenenti allo stesso canale di comunicazione; se due segnali si sovrappongono le informazioni vengono distrutte. In canali broadcast è quindi necessario un **protocollo di arbitraggio** del canale, in grado di regolare gli accessi da parte di molti utenti.



Il MAC Broadcast (FF: FF: FF: FF: FF: FF) è un indirizzo speciale che specifica tutte le schede di rete, quindi la comunicazione viaggia ovunque, arriva a tutti e viene assimilata da chiunque.

Reti di Commutazione a Circuito

Nelle reti a commutazione di circuito, i dati vengono trasmessi tra un mittente e un destinatario finale posti agli estremi di un cammino (circuito) di canali di comunicazione **punto a punto**. Il circuito viene negoziato e riservato a priori, attraverso opportune procedure. Una volta quindi identificati ed ottenuti i canali che collegano mittente e destinatario, la comunicazione dati può avvenire come un'unica sequenza di bit, senza interruzioni, in quanto sia mittente che destinatario sono già fissati e collegati al momento della creazione del circuito riservato. Il ritardo di trasmissione è ridotto, ma può sembrare abissale (questo perché il ritardo si accumula da nodo intermedio a nodo intermedio). I costi di questo canale virtuale sono elevati, in quanto si sta creando un canale virtuale appoggiandosi ad altre reti; infatti, il servizio non si paga ad utilizzo, ma a tempo di permanenza del circuito virtuale. Un esempio di questo tipo di collegamento è la linea telefonica di un tempo.



Reti di Commutazione a Pacchetto

Immaginiamo di avere sei nodi connessi punto a punto, dove in ogni rete punto a punto si trasmettono pacchetti, pacchetti che però vengono trasmessi non in modo continuo, ma con dei ritardi. È ovvio capire che si stanno usando tre canali tutti non sfruttati al massimo. I pacchetti che viaggiano sui tre canali differenti allora potrebbero essere trasmessi su un unico canale broadcast, in questo modo si avrebbe un singolo canale interamente occupato e sfruttato alle massime capacità. La maggior parte delle reti per trasmissione dati digitali è costruita in queste modalità. Quando si hanno pacchetti diversi da mittenti diversi a destinatari diversi è però necessario includere in ogni pacchetto l'informazione sull'identità del rispettivo mittente e destinatario. Componendo in serie canali broadcast a commutazione di pacchetto i nodi ricevitori devono di volta in volta farsi carico di verificare se il pacchetto sia giunto a destinazione (acknowledgment) e in caso contrario provvedere al reinoltro del pacchetto ricevuto. Il prezzo di questa tecnica è legato al maggiore ritardo per la comunicazione dati, dovuto all'esigenza di iterare più volte la ricezione e l'inoltro di pacchetti sui canali broadcast in sequenza. Il vantaggio, tuttavia è legato al maggiore utilizzo dei canali ad accesso multiplo, e quindi alla possibilità di tariffare la comunicazione in base ai dati trasmessi, e non in base al tempo necessario. Si hanno quindi **diversi flussi** di pacchetti indipendenti che seguono percorsi indipendenti.

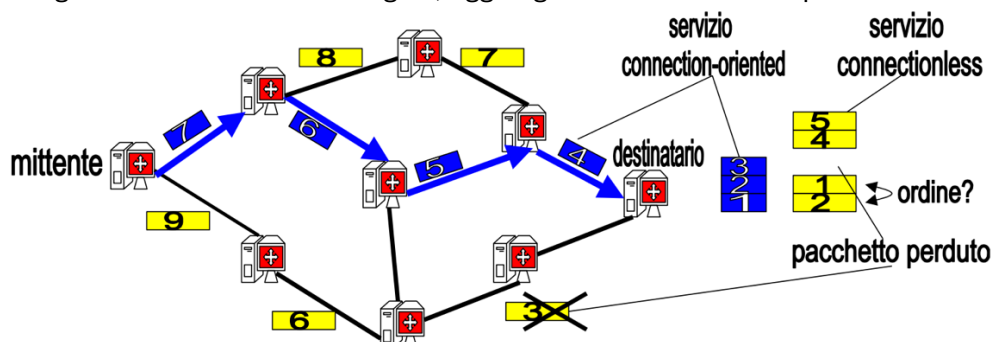


Servizi orientati alla connessione e non

Nelle reti di commutazione a pacchetto il servizio di trasmissione dati, tra mittente e destinatario ha proprietà determinate dal ruolo dei protocolli di rete utilizzati. I dati abbiamo detto essere spediti in pacchetti, dove in ogni pacchetto è indicato esplicitamente il mittente e il destinatario. Succede praticamente sempre che il mittente e il destinatario non siano collegati fisicamente, ma che siano all'interno di una rete di reti, di conseguenza, il pacchetto deve viaggiare attraverso intermediari che immagazzinano i pacchetti, leggono l'indirizzo MAC e l'indirizzo IP e in base a ciò decidono il percorso da far intraprendere al pacchetto (instradamento). Tuttavia, le strade non sono tutte uguali, alcune possono essere più trafficate, più vecchie e altre possono essere dei vicoli ciechi... L'arrivo dei pacchetti non è sempre omogeneo; infatti, il pacchetto X può essere spedito dopo il pacchetto Y, ma può arrivare prima a causa dell'instradamento migliore. Nasce quindi un'esigenza, ovvero avere un servizio orientato alla connessione. Analizziamo dunque i diversi servizi di rete proposti.

- **Connection-oriented:** garantisce la consegna **ordinata** dei pacchetti ricevuti, secondo l'ordine di invio, inoltre, garantisce la ritrasmissione di eventuali pacchetti perduti.
- **Connectionless:** i pacchetti seguono strade diverse e arrivano in ordine diverso o non arrivano mai. Nel primo caso il destinatario è sempre in grado di ricomporre il messaggio, nel secondo il pacchetto è perso e resta perduto, non verrà inviato e all'applicazione arriverà un messaggio troncato, ma comunque in parte comprensibile.

Spieghiamo meglio tutto ciò con un'immagine, aggiungendo informazioni e parole chiave.



Siamo all'interno di una rete a maglia, ovvero una topologia non tradizionale (verosimilmente Internet assomiglia a una cosa del genere), con un aspetto simile a un grafo generico. All'interno di questa rete si comunica con **commutazione a pacchetto** (perché è lo standard a oggi usato ed è ciò che ci interessa studiare). In questa rete vediamo un mittente e un destinatario e i due diversi servizi di connessione. Il pacchetto parte dal mittente, e arriva ad un **nodo** (rosso). Questo nodo **instraderà** il pacchetto ricevuto, ovvero porterà il pacchetto sulla strada più consona per garantire l'**inoltro** del pacchetto appena ricevuto. L'**instradamento** tuttavia non è casuale, ma segue una **tabella di instradamento**, decisa da un **protocollo** (detto **Routing**) che decide esattamente cosa scrivere all'interno della tabella di instradamento.

Notiamo adesso il pacchetto numero 3 che è stato perduto. Nel servizio connectionless il pacchetto è stato perso una volta per tutte, non verrà reinviato e non verrà mai ricevuto. Nel servizio connection-oriented, invece, il pacchetto viene riconsegnato e contribuisce a comporre il messaggio. In questo servizio, ogni volta che il mittente invia un pacchetto si aspetta una risposta da parte del destinatario, detta ack (acknowledgment) che comunica la corretta trasmissione del pacchetto di identificato come X. Se l'ack di un pacchetto manca e se si è in **timeout** (tempo limite di attesa per arrivo da mittente a destinatario di un pacchetto) allora il mittente può far ripartire il pacchetto mancante. Questo, tuttavia può dare problemi (si possono creare duplicati), specialmente quando il mittente reinvia il pacchetto nello stesso momento dell'invio dell'ack, questo è dovuto dal fatto che la rete verrà appesantita ulteriormente (pacchetto reinviato e ack reinviato a sua volta), inoltre il pacchetto verrà completamente scartato, poiché già presente nel **buffer** del destinatario. In una rete congestionata è quindi essenziale aumentare il timeout, altrimenti si vede la rete collassare per l'eccessivo congestionamento. Il **buffer**, accennato prima, è una memoria tampone che permette al destinatario di riordinare i pacchetti per comporre in modo corretto il messaggio, infatti in un servizio connection-oriented anche se manca solo un pacchetto il messaggio non viene mai consegnato all'applicativo che lo richiede, ma si aspetta **sempre** il pacchetto mancante. Questa attesa può dare vita ad attacchi hacker detti DOS (Denial of Service), dove si inviano messaggi molto grandi senza far mai arrivare il primo pacchetto, il buffer viene sovraccaricato e il servizio interrotto; in quanto tutti gli altri pacchetti che arriveranno non avranno spazio nel buffer e verranno automaticamente scartati.

Bonus! Connection-oriented: TCP/IP, connectionless: UDP

Protocolli di rete organizzati a livelli

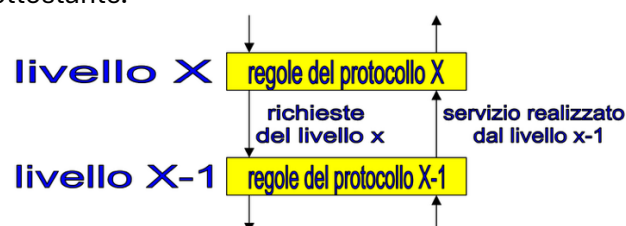
L'architettura dei collegamenti è una parte di design della rete che deve essere definita a un meta-livello da un'**architettura dei protocolli di rete**, che deve essere **rigida e ordinata**. I protocolli non devono essere spaghetti design, ma devono essere modulabili, facili da cambiare, modificare ed aggiornare senza intaccare altri protocolli di supporto.

Un **protocollo** sono regole e procedure **semantiche** di **gestione dei processi di comunicazione**, ma sono anche regole e formati **sintattici** per **definire lo scambio di messaggi** non ambigui, permettendo **compatibilità** dei dispositivi e dei sistemi conformi allo stesso **standard** (che usano lo stesso formato e lo stesso protocollo).

Architettura dei protocolli di rete: **separazione in livelli** dei protocolli di rete

- Ogni livello analizza, risolve e gestisce un solo problema
- I livelli superiori effettuano richieste ai livelli inferiori
- I livelli inferiori forniscono servizi ai livelli superiori
 - Andando verso il basso con le frecce (dette API) si pongono delle domande, mentre andando verso l'alto si danno delle risposte.

In sostanza, l'architettura dei protocolli di rete è basata sull'astrazione, ogni livello lavora su un'astrazione del livello sottostante.



Architettura standard protocolli di rete

Lo standard **ISO/OSI** è un insieme di livelli completo e rigoroso per supportare la comunicazione in rete. ISO/OSI definisce un'architettura dei protocolli di rete in 7 livelli, ogni livello fornisce un servizio al livello superiore nascondendo la complessità di quelli inferiori, e dialoga virtualmente in modo paritario con il suo omologo dall'altra parte.



Livello Fisico: trasmissione dei bit tramite segnali

Livello MAC/LLC: decide formato, sintassi e momento di invio del frame (pacchetto). Comprende sia l'indirizzo del mittente, del destinatario, il dato da trasmettere e il CRC, ovvero un metodo di controllo che individua eventuali errori di trasmissione. Inoltre, LLC esegue un controllo logico del canale, individua errori ed invia l'ack.

Livello Rete: si occupa della frammentazione del pacchetto, dell'indirizzamento e dell'instradamento. È il livello che si occupa di unificare ogni rete locale con tutte le altre singole reti, in pratica rende omogeneo ciò che è eterogeneo. Viene effettuato attraverso dei router, dispositivi che consentono la comunicazione tra reti diverse utilizzando indirizzi strutturati e gerarchici (IPv4/IPv6); tali indirizzi permettono di capire a quale rete si riferisce il MAC-address. Gli indirizzi IPv4 e IPv6 identificano nome e luogo del mittente, facilitando notevolmente l'instradamento dei router (devono lavorare in velocità, avendo poco tempo a disposizione). Inoltre, a questo livello si decide anche il ritmo di invio dei pacchetti (detti segmenti), attuando buffer di invio per non sovraccaricare una rete già congestionata, evitando quindi una saturazione del buffer. Questo livello implementa l'**internet working**.

Livello Trasporto: si occupa della connessione affidabile e del controllo della congestione. Gestisce il protocollo di trasmissione, se connection-oriented o connectionless; nel primo caso usa TCP (grazie agli ack), nel secondo caso usa UDP. Qui ci si occupa dell'affidabilità della trasmissione.

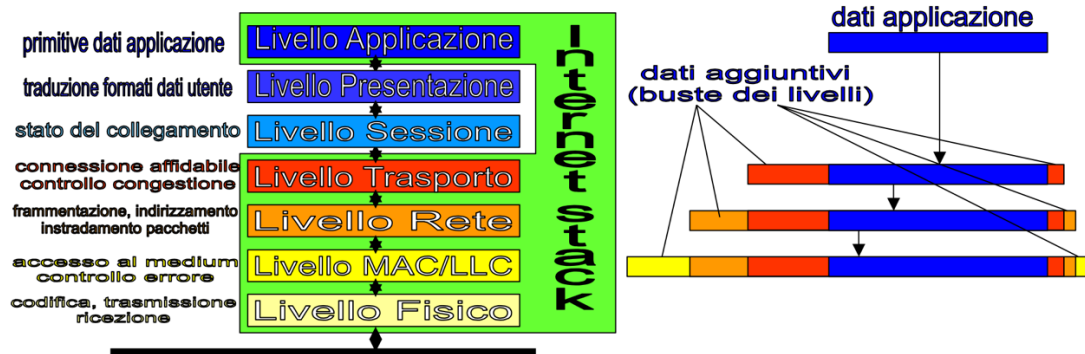
Livello di Sessione: ormai è obsoleto, ma si occupa di mantenere la sessione attiva tra mittente e destinatario. Al livello rosso si apre una connessione, e si incominciano a mandare dati, qualora si dovesse interrompere il collegamento e riaprirlo successivamente su una rete diversa il livello di sessione si ricorda da dove riprende il "download", sapendo quindi da che punto ricominciare a trasmettere i pacchetti e da che punto unire i pacchetti vecchi a quelli nuovi. Ormai questo livello è gestito lato server-side e dalle applicazioni.

Livello di Presentazione: anche questo è ormai obsoleto, serve per un'ulteriore conversione dei dati, (da little endian a big endian), in pratica si converte da un linguaggio da un altro.

Livello di Applicazione: vengono incarnati i protocolli che permettono alle applicazioni di comunicare, in pratica si sta guardando al lato browser-web service. Sono i protocolli necessari al browser per una comunicazione con il lato server-side, un esempio è il protocollo HTTP/HTTPS. Vengono definite quindi le primitive dati applicazione.

Architettura dei protocolli di Internet

L'architettura dei protocolli di internet utilizza solo 5 dei 7 livelli ISO/OSI, in trasmissione ogni livello riceve dati dai livelli superiori e li **incapsula** in buste, aggiungendo dei dati, utili ad istruire il medesimo livello del dispositivo ricevente. In ricezione ogni livello verifica i dati della busta e agisce di conseguenza, passando il contenuto ai livelli superiori (**decapsulamento**).



L'Internet stack come detto prima utilizza solo 5 dei 7 livelli ISO/OSI. I dati applicazione (striscia blu) vengono imbustati e passati al livello trasporto che imbusta i dati aggiungendo informazioni sull'ordinamento e controllo della velocità di invio delle buste (si occupa del servizio di comunicazione usato). Il livello rosso aggiunge dati in testa (**header**) e in coda (**trailer**). Nell'header viene messo l'identificativo univoco del pacchetto (che in questa parte si chiama segmento) e nel trailer viene inserito il protocollo usato (esempio TCP).

Successivamente la busta rossa viene passata al livello di rete che si occupa di frammentare i dati in pacchetti e decidere il cammino sul quale inviare il pacchetto in base all'indirizzo IP del destinatario. Nell'header viene inserita la versione del protocollo e l'attuale indirizzo del mittente e del destinatario, nel trailer si aggiungono sempre misure di controllo.

Ora la busta arancione passa al livello MAC/LLC che esegue la consegna finale dei dati a dispositivi diversi su una rete locale e detta un eventuale abbassamento del rate di codifica (velocità di trasmissione). Nell'header il MAC protocol inserisce i MAC address del mittente e del destinatario, in questo modo la funzione spedisci può lavorare correttamente, decidendo anche il momento di invio. Nel trailer viene inserito un altro livello di controllo, creando ridondanza diversificata; infatti, ogni controllo si riferisce a **situazioni e problemi** diversi. È quindi anche completata la busta gialla.

Nota! Ogni livello guarda solo i bit del suo colore.

Durante la trasmissione ad ogni passo effettuato il contenuto della busta gialla viene aperto e modificato, quando sia fa un passo, si guarda la busta gialla, si vede se il destinatario sono io, e poi si butta al di sopra la busta arancione (l'arancione non sa neanche cosa ha fatto il giallo). Il livello arancione guarda il contenuto della busta arancione, si guarda indirizzo IP e si verifica se si è destinatari o intermediari. Nel caso degli intermediari, siamo un router, destinatari parziali, serviamo solo per fare passamano; quindi, si guarda la rete del destinatario, si consulta la propria tabella di routing, si ricompone la busta arancione, si passa al livello sottostante che comporrà la busta gialla con mittente la scheda di rete del router che ha ricevuto il messaggio e destinatario (la scheda di rete del router successivo), identificata attraverso ARP (protocollo che fornisce il MAC address del router successivo). E poi si rimette il frame in viaggio (a questo livello il pacchetto si chiama frame).

Se siamo i destinatari finali, invece buttiamo su la busta rossa e quando si arriva al livello rosso si è sicuri di essere il destinatario finale. Di conseguenza, ack viene inviato. Nella busta rossa è quindi presente l'identificativo di quale applicazione necessita il dato ricevuto. Si necessita quindi il numero di porta applicazione e si butta su la busta dicendo verso chi bisogna andare.

Il numero porta è un socket di comunicazione: contiene indirizzo IP + un numero porta, permettendo quindi a qualsiasi processo di parlare con qualsiasi processo.

Livelli e integrazione delle reti – Analisi base

Nelle prossime pagine vendono esaminate in dettaglio quali sono le problematiche gestire dai protocolli di rete, a partire dai livelli più bassi (livello fisico), per andare ai livelli più alti (livello applicazione). Ad ogni livello i protocolli gestiscono problemi diversi, e la rete assume caratteristiche di integrazione e proprietà diverse. Diamo qualche definizione per ripassare:

- **Livello fisico:** la rete è un solo segmento, ovvero un mezzo di trasmissione condiviso tra dispositivi, si hanno regole per codificare e trasmettere dati con una tecnologia in comune. L'hub lavora qui.

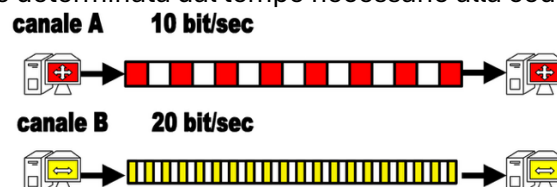
- Al livello **MAC/LLC** la rete è locale, possono essere integrati mezzi trasmissivi e tecnologie diverse, si hanno regole per indirizzi dei dispositivi, tempi di accesso al mezzo e gestione errori. Lo switch lavora a questo livello, sceglie selettivamente a chi mandare il messaggio, cercando di capire (tramite indirizzo MAC) chi è il destinatario del frame.
- Il livello di **rete** è una vera e propria collezione di reti, e assume una struttura gerarchica (reti di reti, sottoreti...), si hanno regole per indirizzi di rete che nascondono i dettagli della rete locale, e si utilizzano nuovi dispositivi (router) che smistano i pacchetti di dati tra rete e rete. Ogni rete locale ha un suo rappresentante, si occupa di parlare tra router (di reti diverse) per passare dalla busta gialla (locale) alla busta arancione (internet)
- Il livello di **trasporto** è una collezione di reti organizzare gerarchicamente, vengono attuate tutte quelle semantiche delle regole di trasporto, ovvero i protocolli per una spedizione affidabile di pacchetti e controllo della congestione della rete
- Livello **applicazione** è l'ultimo livello, qui tutto funziona, e tutto appare come una scatola magica.

Riassumendo, l'**Hub** riesce a rigenerare i segnali e come li riceve li spara fuori su tutte le sue porte, lo **switch** lavora su rete locale indirizzando in modo corretto il pacchetto, e il **router** appare come il ponte tra una rete locale e internet (rete di reti), ragiona a livello arancione e separa le due reti locali secondo nuovi indirizzi logici.

Nota! Il router più vicino ad una rete locale attua il firewall (:= regole di filtraggio).

Il livello giallo – Livello Fisico e Livello MAC/LLC

Iniziamo col parlare del livello fisico e la codifica dei dati digitali. La trasmissione dati su una canale di comunicazione richiede la codifica e la decodifica dei dati, uno di seguito all'altro sul mezzo trasmissivo, da parte di una scheda di rete. I dati digitali sono **bit**, e questi bit possono essere trasmessi a velocità diverse sul mezzo trasmissivo. **Erroneamente** per velocità si pensa in quanto tempo viene coperta una certa distanza da punto "A" a punto "B". In realtà, per velocità si intende la durata della rappresentazione dello stato di un bit, questo perché tutti i segnali viaggiano sempre a 300 mila km al secondo; dunque, più lo stato permane per un certo lasso di tempo, più la velocità di trasmissione sarà bassa. Per capacità massima di trasmissione si intende quindi il valore massimo di bit al secondo trasmissibili (è determinata dal tempo necessario alla codifica del valore di ogni bit).



Il canale B, nell'immagine sopra trasmette al doppio della velocità, ogni bit permane la metà di quanto lo stesso bit permane nel canale A.

Esaminiamo una rete a commutazione di pacchetto, nel piccolo di un segmento di rete locale.

In un segmento di rete locale si ha **un mezzo di trasmissione condiviso** con canale ad **accesso multiplo**, dove tutte le schede di rete ricevono le trasmissioni effettuate sul mezzo trasmissivo.

Ogni scheda di rete ha assegnato un indirizzo fisico univoco: **indirizzo MAC** (ogni scheda di rete assegna il massimo della dimensione di un frame, spiegato dopo). Inoltre, il livello due gestisce l'**accesso** al mezzo trasmissivo, ovvero gestisce la **trasmissione** (arbitraggio degli accessi del canale), l'**indirizzamento** di livello MAC (quale scheda di rete è destinataria del frame trasmesso) e la **ricezione** (controllo tra indirizzo destinatario del frame e proprio indirizzo della scheda di rete) con conseguente passaggio ai protocolli superiori.

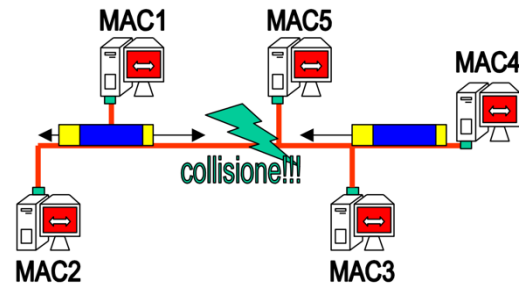


Questo è un **frame** di dati, ed è quello che viene trasmesso a livello due, ovvero all'interno di una semplicissima rete locale con commutazione a pacchetto. "D" è l'indirizzo MAC del destinatario e "M" è l'indirizzo MAC del mittente, mentre il trailer (parte gialla finale) è il bit di controllo errore, è dunque il bit che permette di eseguire la parità incrociata (algoritmo che permette l'auto-rilevamento di un

0	1	0	0	0	0	0	1
0	1	0	0	0	0	1	0
1	1	0	0	0	0	1	1
0	1	0	0	0	1	0	0
1	1	0	0	0	1	0	1
1	1	0	0	0	1	1	0
0	1	0	0	0	1	1	1
1	1	0	0	0	0	0	0

frame errato fino a 3 bit sbagliati per frame e l'auto-correzione fino a 1 bit errato per frame). Senza parità incrociata ogni frame trasmesso sbagliato deve essere gettato via, con questo algoritmo; invece, si ammette anche un frame con 1 bit errato; dunque, siamo in grado di autocorreggere il messaggio, riducendo la probabilità di insuccesso ad un numero bassissimo, quasi irrilevante. Nell'immagine a lato si nota che all'intersezione è presente il bit errato, che deve essere corretto con il suo complementare, in questo caso con lo zero.

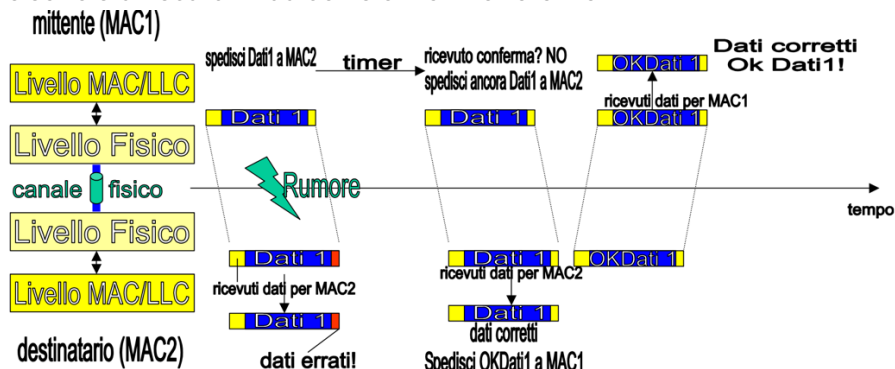
Nell'immagine qui a destra, invece si ha un segmento di rete LAN (canale condiviso) dove si ha una collisione tra frame. Per evitare questa collisione si elegge un rappresentante, un super calcolatore con il potere di gestire la politica di accesso al canale. Facciamo finta che il super rappresentante sia il MAC2, è lui quindi che dà il diritto ad altri calcolatori di parlare. La politica di scheduling, dunque, garantisce l'assenza di collisioni, grazie ad un protocollo implementato tra tutte le macchine. In pratica, ogni macchina ha il potere di trasmettere qualcosa ad ogni intervallo di tempo, in questo modo si evitano del tutto le collisioni, in più il tempo di comunicazione inutilizzato di una macchina può essere donato ad un'altra macchina.



Piccola Digressione

Questo protocollo però per Ethernet e Wi-Fi non viene utilizzato. Ad esempio, per Ethernet è possibile ascoltare il canale durante la trasmissione, dunque se ci si accorge che durante una trasmissione il canale è occupato la trasmissione viene subito interrotta. In pratica, si spreca del canale, con la speranza di non fare collisione, se la si fa si è sprecato solo un po' di canale e si dovrà scontare una finestra di contesa, ovvero un tempo da aspettare prima di trasmettere una volta che il canale si libera. In questo modo, ogni volta che il canale è libero si genera un'attesa ordinata, che evita le collisioni, dando subito accesso a chi deve aspettare meno tempo.

Abbiamo definito come non creare collisioni, ma vogliamo anche far sì che il canale sia un canale affidabile, dunque che non generi collisioni, o che non generi ulteriori collisioni in caso di errori di trasmissione che sono stati scaturiti da collisioni o interferenze.



Nell'immagine soprastante si vede che in un certo istante un frame viene inviato da un mittente al destinatario, destinatario che all'arrivo del messaggio riscontra un errore, e non invia l'ack. L'ack, non arrivando attiva il reinvio del messaggio da parte del mittente, che reinvia il frame. Il destinatario, questa volta ha il frame corretto e invia ack; ack che viene ricevuto e conclude la trasmissione. Da notare però che nel lasso di tempo che va dalla ricezione del frame, controllo e invio ack, il canale **deve** restare riservato al destinatario, in questo modo, il destinatario è sicuro che l'ack venga ricevuto correttamente e che non si attivi il timeout del mittente. Tutto questo però deve succedere nell'ordine dei micro/nanosecondi, altrimenti si rischia di entrare in timeout e di bloccare per troppo tempo la rete. **Nota!** La derivata (pendenza) delle linee trasversali rappresenta la velocità della trasmissione del frame.

Digressione su ack

Quando la busta gialla fa un passo in avanti, un ack fa un passo a ritroso per comunicare conferma della ricezione, e ciò viene fatto per ogni passo che un frame compie. Alla fine, però anche il destinatario finale manda un ack finale, e questo ack non riguarda più il livello MAC/LLC, ma riguarda il livello di trasporto (livello rosso), questo perché bisogna garantire il servizio di comunicazione end-to-end, e quello che lo può fare è solo il destinatario finale, il quale conferma di aver ricevuto il pacchetto completo e corretto (in caso di servizio connection-oriented).

Riparlano invece di reti locali, ma questa volta ad anello, non abbiamo più un “*super-responsabile*” della rete che decide chi può trasmettere e quando può farlo, ma abbiamo un **token ring** che gira all’interno della rete. Ovvero, chi detiene il token ha diritto alla trasmissione, una volta finita la tale comunicazione, il token viene accodato insieme all’ultimo pacchetto inviato. Adesso, chi detiene il token detiene esclusivamente il diritto di trasmettere. In questa configurazione ad anello (che con questa implementazione non può essere progettata per rompersi) gli ack non vengono inviati, infatti, se il messaggio ritorna per intero al mittente significa che anche il destinatario avrà ricevuto correttamente il messaggio, in caso contrario, i pacchetti verranno reinviati per intero. Con questo paragrafo si conclude il discorso sulle “isole gialle”, ovvero sulle reti locali. Continueremo a parlare delle reti locali, ma da un’altra prospettiva, da una prospettiva superiore, che parlerà in primo piano del livello tre dello stack di internet, ovvero il livello di rete.

Tempo di Trasmissione

La formula del tempo necessaria affinché un intero pacchetto arrivi a destinazione si calcola:

$$\text{Tempo Totale} = \text{Ritardo di Propagazione} + \text{Ritardo di Trasmissione}$$

Ritardo di Propagazione: è un valore molto basso, è quanto i bit impiegano a viaggiare lungo il canale trasmissivo. La formula è sostanzialmente la distanza percorsa / velocità della luce.

Il *Ritardo di Trasmissione* invece è il tempo fisico che la scheda di rete impiega a sparare i bit fuori dal calcolatore. La formula è la seguente:

$$\text{Ritardo Trasmissione} = \frac{\text{Dimensione del frame (o pacchetto)}}{\text{Capacità del canale di trasmissione}}$$

Throughput e Utilizzo Percentuale

Se definiamo la larghezza di banda *bandwidth* come il “diametro” del link fisico, un po’ come se fosse il diametro di un tubo che porta l’acqua e se definiamo il *throughput* come la quantità di dati utili trasferiti con successo da un nodo all’altro in un dato periodo di tempo allora possiamo ricavare le seguenti formule.

$$\text{Throughput (Mbps)} = \frac{D \text{ (bit)}}{T \text{ (secondi)}}$$

dove: **D** = *Quantità totale di dati trasferiti*; **T** = *Tempo impiegato per il trasferimento*
questa formula può essere riadattata per TCP

$$\text{Throughput (Mbps)} \leq \frac{W}{RTT \text{ (secondi)}}$$

dove: **W** è la *dimensione della finestra TCP* → *bit inviabili prima di attesa ACK*

Possiamo anche definire il rapporto tra traffico e capacità

$$\text{Utilizzo (\%)} = \frac{\text{Throughput (Mbps)}}{\text{Bandwidth (Mbps)}} * 100$$

Se la rete locale utilizza anche un Proxy, con una determinato *cache hit* la formula dell'*utilizzo percentuale della rete* è la seguente:

$$Utilizzo (\%) = \frac{Throughput (Mbps) * (1 - Cache Hit)}{Bandwidth (Mbps)} * 100$$

Sia dato invece il seguente esempio di esercizio:

Al router di bordo arrivano simultaneamente flussi di dati da $N = 4$ switch interni alla LAN.

Ciascuno dei 4 switch trasmette traffico verso il router a un tasso costante di 35 Mb/s.

Il router deve inoltrare tutto questo traffico verso l'esterno usando sempre il singolo link di accesso da 100 Mb/s. Il router ha una memoria di buffer dedicata all'accodamento dei pacchetti pari a 15 MB.

Possiamo determinare la velocità netta in Mb/s in cui si riempie il buffer

$$Velocità Riempimento Buffer = Capacità Input (Mb/s) - Capacità Output (Mb/s)$$

Conseguentemente, possiamo anche calcolare il *tempo di saturazione* del buffer

$$Tempo di Saturazione = \frac{Dimensione Buffer (Mbps)}{CI (Mbps) - CO (Mbps)}$$

Dove: CI = Capacità in Input; CO = Capacità in Output

Approfondimenti

Capitolo 1

Topologie

La topologia a stella prevede che ogni nodo è fisicamente collegato ad esattamente altri due nodi, il precedente e il successivo, andando di fatti a creare un circuito chiuso. La tecnologia di comunicazione più famosa all'interno di questa topologia è la **token ring**, dove questo testimone passa da nodo a nodo, e chi detiene tale token in quel dato istante ha diritto alla comunicazione. La comunicazione parte da un nodo e viaggia in senso orario, o antiorario di nodo in nodo fino al raggiungimento del destinatario.

Il vantaggio principale è l'assenza di collisioni, infatti solo chi detiene il token può comunicare; quindi, la qualità del servizio è considerata buona. Un altro vantaggio è che se si utilizza un doppio anello o un anello ridondante, qualora si dovesse tranciare un cavo la rete rimarrebbe usufruibile ed utilizzabile. Un partizionamento di rete occorre quindi quando due sugli "n" subiscono un malfunzionamento. Lo svantaggio principale è il costo e la scalabilità, infatti in una rete ad anello già costruita e funzionante è difficile inserire un ulteriore nodo, richiederebbe l'inserimento (in modo fisico) del nuovo nodo in modo tale da poter tirare un cavo alla sinistra e alla destra del nodo, in modo tale da allargare l'anello e ricomporre l'anello.

Nella topologia a stella invece la struttura prevede una serie di nodi, connessi tutti ad un nodo centrale. Ogni nodo, quindi, comunica con il nodo centrale, e il nodo centrale a sua volta comunica con il nodo destinatario. Un tempo il nodo centrale era un hub, tale dispositivo aumentava a dismisura le collisioni; dunque, da un hub si è passati ad uno switch, che è selettivo; infatti, controlla l'indirizzo MAC del destinatario ed invia solo al destinatario il messaggio inviato dal mittente.

Il grande vantaggio è la scalabilità, infatti per aggiungere un nodo basta tirare un semplice filo dal nuovo nodo allo switch (plug and play, grazie agli switch di nuova generazione).

Il grande svantaggio è il single point of failure del nodo centrale, che però nelle moderne reti viene ovviato con la costruzione di una ridondanza che permette di ripristinare in pochi secondi la rete. Inoltre, visto che gli switch costano poco, la sostituzione dello switch è economica, e quasi sempre immediata.

La topologia a stella è quella più usata, per **rapporto costo-efficienza**, scalabilità, facilità di installazione, manutenzione ed utilizzo. Inoltre, la diffusione degli switch al posto degli hub ha permesso di dominare la scala delle topologie.

Particolarità:

Nelle reti ad anello nessun ack viene inviato, se il messaggio partito dal mittente, ritorna al mittente, allora la comunicazione è andata a buon fine.

Ritardo di Ricezione

Nei servizi connection-oriented il ritardo di ricezione dei pacchetti e il conseguente RTT influisce sull'efficacia di utilizzo della rete principalmente in due modi

- 1- Tempi di inattività: affinché la comunicazione sia affidabile, il mittente deve attendere il messaggio di conferma ACK dal destinatario. Se il ritardo di rete è elevato, il mittente trascorrerà molto tempo in attesa del messaggio lasciando il canale di trasmissione inattivo, non sfruttando la banda
- 2- Ritrasmissioni e spreco di banda: caso in cui la rete è molto congestionata e il timeout scatta pochi istanti prima dell'effettiva consegna del messaggio, portando il mittente a reinviare il messaggio e a sprecare banda.

Ack

Ci possono essere due tipi di ACK, quelli del livello MAC/LLC e quelli a livello di Trasporto.

Livello MAC/LLC: inviato quando un frame passa da un nodo “n” a un nodo “n+1” in modo corretto, quindi senza errori.

Livello Trasporto: inviato quando un pacchetto giunge dal mittente al destinatario in modo corretto, dunque senza errori.

Internet funzionerebbe anche senza gli ACK del livello MAC/LLC, ma sarebbe molto più lento in quanto sono proprio gli ACK del livello MAC/LLC che garantiscono la rapidità e l'efficienza.

Immaginiamo di dover trasmettere un messaggio da Roma a Tokyo con un servizio connection-oriented in modo cablato (no reti wireless coinvolte).

Se già il primo tragitto computer – router incontra errore, l'ACK a livello MAC/LLC non arriverebbe, dunque il computer ritrasmetterebbe subito il primo frame dopo pochi millisecondi. Se si usasse solo l'ACK a livello di trasporto, il primo nodo, ovvero il mittente si accorgerebbe dell'errore (compiuto nel nostro esempio al primo passo) solo una volta che il pacchetto (danneggiato) è giunto al destinatario finale, dunque il messaggio verrebbe ritrasmesso solo dopo un numero considerevole di millisecondi.

Ack MAC/LLC filtra istantaneamente gli **errori fisici** e le **collisioni della rete**.

Ack Trasporto gestisce le **perdite reali** causate dai colli di bottiglia e dalla congestione globale dei router (come buffer overflow).

MAC Address e IP

Il MAC Address è indirizzo a 48 bit univoco per ogni calcolatore, viene utilizzato **per identificare un dispositivo all'interno di una rete locale**. È legato all'hardware e piatto, in quanto non fornisce nessuna informazione sulla posizione del dispositivo nella rete globale.

Indirizzo IPv4 è un indirizzo a 32 bit, viene utilizzato per **identificare le reti locali nel mondo**. È un indirizzo strutturato e gerarchico.

L'indirizzo IP fornisce informazioni da un punto dal punto di vista geografico della posizione di quel calcolatore nel mondo, è in generale un'informazione più semantica dell'indirizzo MAC.

I router seguono le tabelle di instradamento leggendo l'indirizzo IP del destinatario, traendo conclusioni e decisioni in pochissimi millisecondi, devono essere veloci. Inoltre, i router solo leggendo i primi bit dell'indirizzo IP riescono a stabilire l'instradamento da fornire al pacchetto.

Gli indirizzi MAC invece sono unici, ed effettuare l'instradamento di un pacchetto solo su quelli sarebbe impossibile, ogni router dovrebbe avere una tabella contenente tutti gli indirizzi MAC presenti al mondo e scansare per ogni pacchetto tutta la tabella per recapitare il pacchetto. Inoltre, se il destinatario cambiasse la sua rete, anche le tabelle di instradamento di ogni router dovrebbero adattarsi di conseguenza, cercando di capire la posizione geografica del mittente. Questo approccio è impossibile.

Half Duplex e Full Duplex

In una connessione **Half-Duplex**, i dati possono viaggiare in entrambe le direzioni, ma **solo in una direzione alla volta**. Se il dispositivo A sta trasmettendo, il dispositivo B deve stare in ascolto. Solo quando A ha finito, B può iniziare a trasmettere a sua volta. Se A e B trasmettono insieme si ha una collisione.

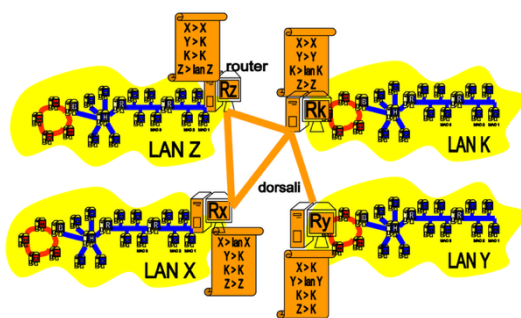
In una connessione **Full-Duplex** i dati possono viaggiare in entrambe le direzioni **contemporaneamente**. Sul cavo di rete (es. il classico cavo Ethernet in rame) ci sono fili fisicamente separati: alcuni dedicati esclusivamente a trasmettere (Tx) e altri dedicati esclusivamente a ricevere (Rx). **Il vantaggio**: A può inviare dati a B, e nello stesso istante B può inviare dati ad A. Le **collisioni sono matematicamente impossibili**, raddoppiando di fatto la capacità teorica del cavo (un cavo da 100 Mbps in full-duplex gestisce 100 Mbps in upload e 100 Mbps in download contemporaneamente).

Reti di Reti – Internetworking

l'isola arancione dello stack di Internet

*“Le reti locali sono connesse attraverso collegamenti organizzati in modo gerarchico, basati su calcolatori rappresentanti della rete locale (**router**), a loro volta collegati da linee dati veloci, o dorsali”*

La comunicazione da calcolatore a calcolatore non sempre avviene all'interno di una rete locale, ma avviene in “Internet”; la rete di reti per eccellenza. Per garantire la comunicazione è quindi essenziale capire se il mittente e il destinatario appartengono (risiedono fisicamente) alla stessa rete, o sono in due posti (reti) diverse. Fino a poco fa abbiamo parlato delle reti locali, la comunicazione tra dispositivi connessi in una rete locale **avviene** e **rimane** all'interno delle isole gialle. Adesso iniziamo a parlare di quando dispositivi diversi non sono nella stessa rete locale, ma sono connessi a due reti locali distinte. In quest'ultimo caso dobbiamo quindi scalare la piramide dello stack per salire al livello arancione, il **livello di rete** che permette il collegamento e la comunicazione fra isole gialle distinte.



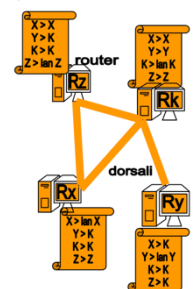
Nell'immagine qui a lato vediamo come per ogni rete locale gialla si ha un rappresentante, detto **router**, che è fisicamente collegato con gli altri **router** delle altre reti locali. Il problema adesso è quindi capire il **percorso** che il pacchetto deve intraprendere per partire dal mittente e arrivare al destinatario, che non è più nella stessa isola gialla. Questo problema, detto **instradamento** viene risolto attraverso le **tabelle di routing**, ovvero strutture dati proprietarie contenute all'interno del router che agiscono al livello tre e che contengono esplicitamente il

percorso che deve intraprendere un pacchetto in partenza da un punto “A” e destinato a punto “B”.

Nell'immagine a destra si vedono le tabelle di routing dell'immagine soprastante.

Prendiamo per esempio il router “Z”. Se il pacchetto è destinato verso la rete locale “X”, allora si consegna direttamente il pacchetto, per via di un collegamento diretto.

Se invece il pacchetto da consegnare è destinato alla rete “Y”, si passa il pacchetto al router della rete “K”, e sarà poi lui ad indirizzarlo in maniera diretta alla rete locale “Y”, ma sempre seguendo la propria tabella di routing. In quest'immagine non c'è, però se nella mia tabella non è presente il percorso per la destinazione, ho un percorso che porta al **default router**. Questo router, che è a un livello superiore, saprà sicuramente dove indirizzare il pacchetto per farlo giungere al destinatario. In sostanza, ogni volta che non si ha una strada diretta si va verso l'alto cercando di risalire la gerarchia dei router, sperando (prima o poi) di trovare una strada diretta che permette al pacchetto di arrivare alla “isola gialla” di arrivo.



Internet Protocol (IP)

È un protocollo di internet che utilizza un indirizzamento globale e **gerarchico (indirizzamento IP)**, fornendo indirizzi specifici alla rete locale e ai nodi della rete locale. Si avvale **dell'instradamento** dei pacchetti (forwarding) grazie a dispositivi amministratori di livello tre, i **router**, che si avvalgono a loro volta delle tabelle di instradamento, celando in qualche modo i dettagli interni della rete LAN. Questo protocollo si occupa anche di imbustare la busta arancione e di frammentare i dati da spedire in pacchetti.

Indirizzamento IPv4

Il protocollo IP definisce una nuova specie di indirizzi, gli indirizzi **IP**. Un indirizzo IP è un identificatore che viene associato a una scheda di rete in modo univoco, in questo modo **tutti** i calcolatori attualmente accesi e con al loro interno una **scheda di rete** hanno indirizzi IP (identificativi) diversi. Connettersi alla rete significa quindi attribuire alla macchina un indirizzo IP. Tale indirizzo può essere **statico** (non cambia mai, fondamentali per risorse famose che devono essere chiamate frequentemente e che devono rimanere note) o **dinamico** (indirizzo IP si ricicla).

Da notare però che l'indirizzo IP cambia a seconda di quale rete sono attualmente connesso; dunque, una macchina non avrà mai lo stesso indirizzo IP, a meno che ad essa non sia stato attribuito un indirizzo IP statico.

Gli indirizzi IP sono **gerarchici** e **strutturati**, hanno 32 bit (IPv4), e sono suddivisi in blocchi da 4 Byte ognuno, dove ogni blocco viene separato da un punto. Un indirizzo è del tipo 130.136.25.1

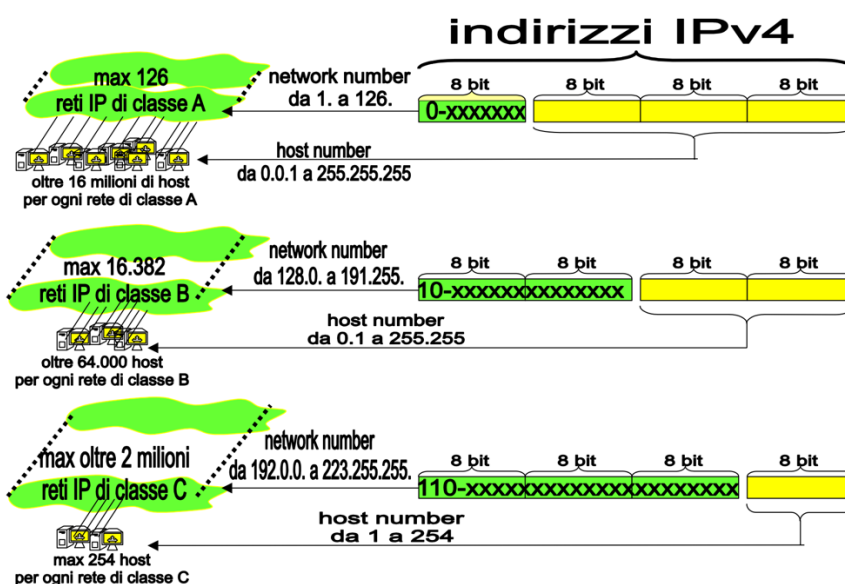
Un indirizzo IP si suddivide in:

- Network Number: è l'identificativo (**dominio**) dell'isola gialla al quale la macchina è connessa
- Host Number: è l'identificativo della macchina connessa ad una specifica rete locale

La lunghezza del Network Number e Host Number può variare in base alla classe di rete utilizzata.

Si nota inoltre che si può avere un indirizzo IP con stesso Host Number e diverso Network Number.

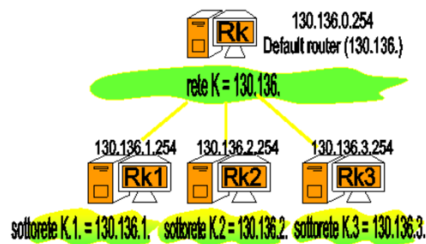
Qui di sotto è riportato l'immagine che spiega le diverse classificazioni degli indirizzi IP



All'interno di una rete, avvalendosi degli indirizzi IP si può creare un broadcast logico, dove tutti i byte riservati agli host sono posti a uno, prendendo ad esempio la rete 130.136 il broadcast logico di questa rete di classe B è 130.136.255.255. Da notare anche due piccole cose, l'indirizzo IP 130.136.0.0 non esiste, o meglio, esiste ma è riservato all'identificazione della rete stessa, mentre l'indirizzo IP 130.136.255.254, ovvero il penultimo host ha solitamente lo scopo di identificare il router di quella rete. Abbiamo capito il funzionamento generale degli indirizzi IPv4 che permettono di identificare delle reti, ora vogliamo fare di più, ovvero vogliamo dividere in sottoreti una rete, in questo modo, una rete grande, come può essere una di classe B, o una di classe A può essere divisa in sottoreti indipendenti l'una dall'altra.

Sottoreti

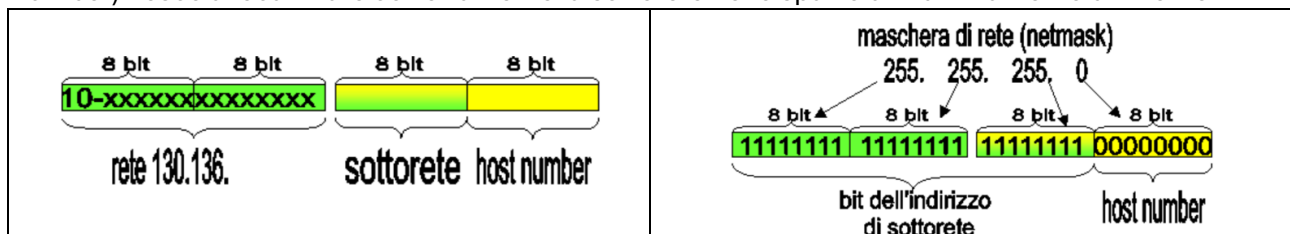
Reti IP e router sono organizzati secondo una gerarchia, basata sul concetto di rete e sottorete. L'indirizzo IP tramite una maschera di rete (netmask) viene spezzato in due componenti logiche, l'**indirizzo di sottorete o rete** (nel caso non ci siano sottoreti) e l'**host number** (numero dell'host appartenente a quella sottorete). Questa breve introduzione l'esamineremo tra poco in modo dettagliato con tanto di esempi. Prese come "blackbox" le sottoreti (supponendo di sapere già il loro funzionamento) è essenziale ricordarsi la gerarchia dei router; infatti, la rete primaria ha un default router, e ogni sottorete della rete primaria ha un proprio sotto-router di riferimento, dunque, anche come evidenziato dall'immagine sottostante si nota che si ha una struttura **gerarchica** ad **albero**. La prima rete K è la genitore, mentre le altre sottoreti sono tutte isolate l'una rispetto all'altra.



Precedentemente abbiamo introdotto gli indirizzi IP e la loro composizione, riprendiamola e osservando come sono composti i 4 byte degli indirizzi IPv4.

Network Number		Host Number	
130	136	xxxxxxx	xxxxxxx

Per ottenere una suddivisione simile a quella della figura, dobbiamo rubare dei bit dall'Host Number e convertirli in Network Number, così da riuscire a creare più sottoreti. Se quindi rubo il terzo byte dell'indirizzo IP e lo promuovo a network number ottengo 256 sottoreti da 256 host, che grazie alla maschera di rete (composta da tutti i bit impostati a 1 per quanti sono i byte impostati come Network Number) riesco a localizzare correttamente la sottorete nello spazio di indirizzamento di internet.



È vero che il concetto non è stato spiegato benissimo, però il concetto è che si prendono dei bit per creare delle sottoreti, ognuna indipendente dalle altre e si utilizza una maschera di rete per identificare queste sottoreti. Il problema principale è che questo metodo crea delle sottoreti pigre, ovvero un po' a caso; infatti, nell'immagine sopra per creare tre sottoreti ne sono state create 256, sprecando uno spazio per altre 253 reti, ognuna limitata a soli 256 host; dunque, collegare 512 host nella stessa sottorete è impossibile. Si vuole fare di più, ovvero si vuole scegliere esattamente il numero di sottoreti creabili e un numero di host preciso per ogni sottorete.

Per spiegare questo facciamo un esempio

Prediamo la nostra rete 130.136. x. y, e suddividiamola in 8 sottoreti, usando un metodo che garantisce qualità, scalabilità e risparmio di spazio indirizzamento IP.

Abbiamo il network number 130.136 di un indirizzo IP di classe B immutabile, è quello e rimane quello.

Però abbiamo 2 byte = 16 bit di host number. Vogliamo creare 8 sottoreti, quindi per identificare queste 8 sottoreti sono necessari $\log_2 8 = 3 \text{ bit}$. Dunque, creiamo una maschera di rete che garantisca questa suddivisione, sapendo che dai 16 bit di NetNumber si è passati a 19 bit.

La maschera di rete sarà quindi la seguente: 11111111.11111111.11100000.00000000 = 255.255.224.0, questa maschera di rete ha lo scopo di identificare a che rete e a che rete appartiene un host. Facciamo un esempio.

Host: 130.136.169.4 in binario: 10000010.10001000.10101001.00000100

Maschera: 255.255.224.0 in binario: 11111111.11111111.111.00000.00000000 := /19

----- And bit a bit

Risultato: 10000010.10001000.10100000.00000000

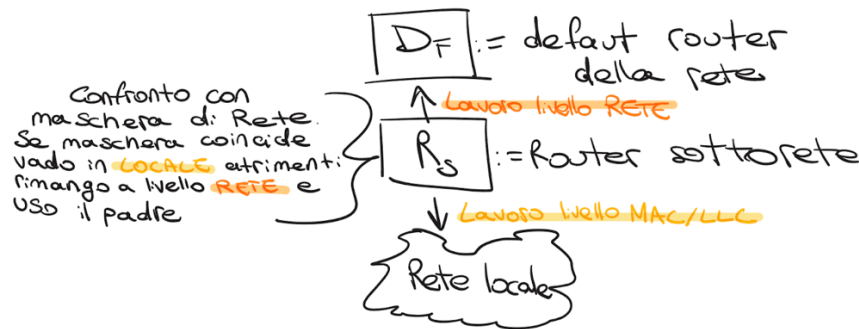
Conversione: 130.136 → Rete di appartenenza; 5° sottorete

Numero di Host: 01001.00000100 → 2308

Andando nel dettaglio, abbiamo creato 8 sottoreti (identificati dai bit azzurri), utilizzando

$\log_2 8 = 3 \text{ bit}$, e ogni sottorete ha la capacità di contenere $2^{(32-19)} = 2^{13} = 8192 \text{ host per sottorete}$

Con questo esempio abbiamo visto come la maschera di rete permette di identificare a quale rete e a quale sottorete della rete l'host appartiene. Quando i router inoltrano i pacchetti (hanno la maschera di rete della rete a cui appartengono) in due cicli di clock capiscono se il destinatario appartiene alla sottorete o se il destinatario non appartiene a quella sottorete. Se il destinatario appartiene alla stessa rete del mittente, allora l'and bit a bit tra maschera di rete e mittente + maschera di rete e destinatario avrà lo stesso risultato, quindi il pacchetto verrà imbustato in una busta gialla e fatto viaggiare nella rete locale (lavora il livello MAC/LLC), se l'and bit a bit invece evidenzia due risultati diversi, allora il router passa il pacchetto al router soprastante a lui nell'albero gerarchico (definito come default router) e questo passaggio fa rimanere la comunicazione a livello di rete (il pacchetto rimane nella busta arancione).



Facciamo un altro esempio, per capire se il pacchetto viene mandato attraverso la rete locale o al default router.

Mittente: 130.136.169.4 → in binario: 10000010.10001000.10101001.00000100

Destinat: 130.136.160.11 → in binario: 10000010.10001000.10100000.00001011

Maschera: 255.255.224.0 → in binario: 11111111.11111111.11100000.00000000

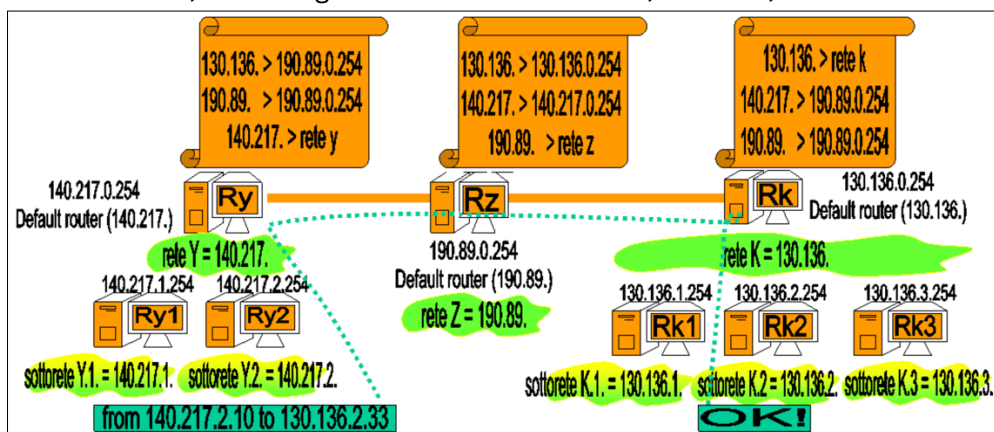
-----And bit a bit

Mittente: 10000010.10001000.10100000.00000000

Destinatario: 10000010.10001000.10100000.00000000

La and bit a bit produce lo stesso risultato sia per il mittente che per il destinatario, dunque, il router in due cicli di clock capisce che deve imbustare il pacchetto nella busta gialla e spedirlo in rete locale.

L'instradamento dei pacchetti, che mette in pratica tutta la teoria vista fino ad ora è riassumibile nell'immagine sottostante, dove vengono messe in mostra: reti, sottoreti, router e default router.



Instradamento (forwarding) dei pacchetti

Esempio di esercizio

Indicare rete, sottorete, host del seguente indirizzo IPv4 130.146.9.1/21

IPv4: 130.136.0000 1001.0000 0001

/21: 130.136.1111 1000.0000 0000 → 255.255.248.0

&Bb: 130.136.0000 1000.0000 0000 → 130.136.8.0/21 sottorete

Abbiamo quindi che: 130.136.8.0/21 è la sottorete 1* delle $2^5 = 32$ sottoreti disponibili (da 0 a 31), in quanto abbiamo rubato 5 bit di host dalla rete 130.136.0.0/16 e li abbiamo utilizzati codificandoli come sottorete. Quindi la macchina con indirizzo IPv4 130.146.9.1/21 è il 257-esimo host della sottorete 130.136.8.0/21.

*Guarda la parte bianca di IPv4: 130.136.0000 1001.0000 0001. La parte azzurra serve per determinare il numero di host, mentre la parte bianca serve per determinare il numero della sottorete e capire il taglio, dunque quanti bit sono stati rubati.

Routing

Le tabelle di forwarding sono il cuore pulsante del protocollo di rete, sono quelle che decidono su quale porta del router far uscire il pacchetto. Queste tabelle possono essere statiche o dinamiche, ma il “succo” del discorso è che un pacchetto deve impiegare il meno tempo possibile per giungere da punto “A” a punto “B”; dunque, è necessario scegliere il cammino di costo minimo.

Le tabelle di forwarding dinamiche si adattano in base a diversi fattori:

- Mobilità degli host (reti wireless)
- Accordi commerciali
- Relazioni tra router (tramite protocollo del tipo RIP, OSPF, BGP)

L'adattamento più interessante è sicuramente l'ultimo, ovvero la relazione da parte dei router per scoprire nuovi cammini per mezzo di **algoritmi di routing**. Non tutti i router sono in comunicazione tra loro, ma solo i router adiacenti sono in diretto contatto, quindi la scelta è immediata, ovvero si sceglie sempre il cammino di costo minore, pur non sapendo la correttezza della strada intrapresa.

Ad esempio, un pacchetto diretto a Milano, che parte da Bologna può seguire una strada del tipo Bologna-Rimini-Verona-Milano, perché la strada diretta Bologna-Milano aveva troppo traffico dati, dunque era lenta. Però in una comunicazione partono più pacchetti, ogni pacchetto può intraprendere una strada diversa, in base al traffico dati di quell'istante. Dunque, i pacchetti di una comunicazione possono andare perduti, seguire strade diverse e arrivare disordinati, tutto questo deve essere gestito da un protocollo (TCP), il protocollo che si occupa di ordine e instradamento.

Per gestire le tabelle di instradamento, aggiornarle in modo tale che il traffico venga rediretto dove c'è più strada libera è necessario un protocollo che metta in comunicazione tutti i router tra loro. In sostanza, i router non possono comunicare in modo diretto tra loro, ma possono farlo se oltre al pacchetto da inviare al destinatario viene allegata un'informazione semantica a loro riservato che evidenzia lo stato della rete.

Il **protocollo ICMP** è un protocollo dei messaggi di controllo per denotare eventuali problemi di comunicazione, creando uno standard per definire la comunicazione di informazioni utili alla gestione di Internet. Il protocollo è quindi utile per notificare errori di configurazione e gestione dei cammini e collegamenti del tipo:

- destinazione non raggiungibile (interruzione di rete?)
- destinazione sconosciuta (indirizzo mal specificato?)
- host destinatario non raggiungibile
- host sconosciuto
- ...

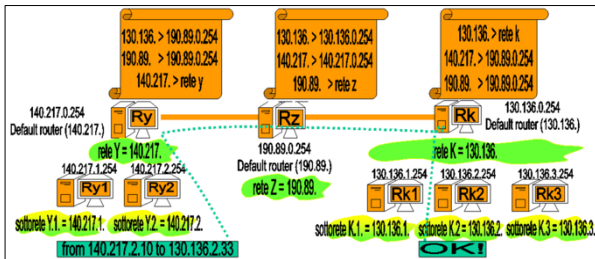
In pratica **protocollo ICMP** definisce informazioni utili alla gestione di internet, gli algoritmi di **routing** permettono di modificare le tabelle di inoltramento in base alle condizioni del traffico dati, il protocollo **TCP** permette di ricompattare il messaggio che ha subito strade diverse, o di far reinviare un pacchetto che è andato perso durante la trasmissione.

Il ping e il traceroute sono due applicazioni basate su ICMP. La prima permette di calcolare il RTT tra host1 e host2, mentre la seconda permette di identificare la sequenza di router attraversati da un pacchetto per giungere all'host2.

Nota! Ogni pacchetto spedito ha un numero, il TTL, un numero che identifica il numero massimo di router che il pacchetto può attraversare. Ad ogni passaggio attraverso un router il TTL diminuisce di uno, quando arriva a zero, il pacchetto viene ucciso, in questo modo si evitano pacchetti zombie all'interno della rete, che girano all'infinito privi di destinazione.

Protocollo ARP e RARP

Riprendiamo in mano il discorso di forwarding dei pacchetti riguardando la seguente immagine.

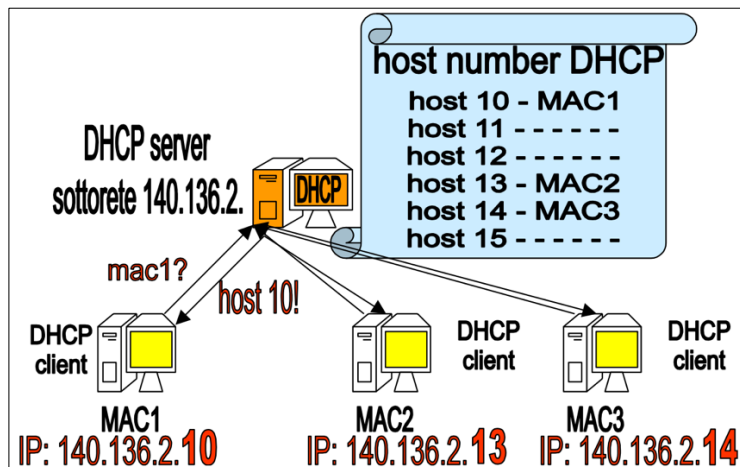


Quando il pacchetto partito dalla rete 140.217 arriva alla rete 130.136.2, il pacchetto deve essere imbustato nella busta gialla per arrivare al destinatario passando per rete locale (frame). Il problema è che il router della sottorete 130.136.2 potrebbe non sapere l'indirizzo MAC del destinatario. Questo problema viene risolto dal protocollo ARP (Address Resolution Protocol), dove il

router della sottorete produce un frame broadcast a tutti gli host chiedendo esplicitamente "qual è l'indirizzo MAC dell'host con indirizzo IP X?" Se tale dispositivo esiste, il router riceverà una risposta e il pacchetto verrà consegnato all'host destinatario. Il protocollo **RARP** è la versione opposta di ARP.

DHCP (Dynamic Hosting Configuration Protocol)

Quando si compra una rete di una certa classe, poniamo una classe C con 256 host, è sicuro che nell'arco della sua vita veda connessi molti di più di 256 dispositivi. Questo significa che dotare ogni singolo dispositivo di IP statico non ha senso. L'indirizzo IP statico deve essere assegnato a macchine sempre accessibili, mentre per tutti gli altri dispositivi si può adottare l'indirizzo IP dinamico. Questo metodo permette di riciclare indirizzi IP inutilizzati. DHCP viene usati in reti wireless, locali e connessioni domestiche, ed è un protocollo che implementa il servizio di assegnazione dell'indirizzo IP agli host che ne fanno richiesta. Quando un dispositivo si connette ad una rete è spaesato, non sa dove si trova, e necessita almeno di una maschera di rete, di un indirizzo IP, di un default router e di un server DNS, per questo appena connesso, il nuovo dispositivo bombarda le rete di richieste di aiuto. Il server DHCP quando riceve una richiesta, compila una offer, comprendente tutte le richieste prima ricevute e la manda in broadcast. Il client riceverà l'offer, e manderà al server DHCP un'accettazione dell'offerta. In questo modo il nuovo dispositivo viene riconosciuto all'interno della rete e ad esso viene associato un indirizzo IP dinamico, che molto probabilmente cambierà in caso di disconnessione e riconnessione.



Livello Trasporto

Adesso concentriamoci a parlare del quarto livello dello stack ISO/OSI, il trasporto. I protocolli di questo livello sono essenzialmente due:

- Transmission Control Protocol (TCP), servizio **affidabile** connection-oriented, utilizzato quando è essenziale non perdere dati durante la trasmissione e riceverli in ordine
- UDP, servizio **best effort** connectionless

Partiamo a fare una breve introduzione del protocollo TCP, quello più interessante.

TCP è un protocollo di trasporto **affidabile** del tipo connection-oriented, si avvale di un **numero di porta** (port number – individua servizio all'interno di un host) del **socket TCP** (4-upla per comunicazione), garantisce una numerazione sequenziale dei pacchetti, gestendo ordinamento ed eliminazione dei duplicati. TCP per garantire un servizio affidabile necessita di **acknowledgment**, ovvero pacchetti di conferma di ricezione/non ricezione. Infine, TCP attua anche una gestione della **congestione** e il controllo del **flusso** dei pacchetti, tramite un flusso di byte controllato, stabile e progressivo (sliding-window, pacchetti inviati in gruppo). **UDP**, invece non ha nulla di tutto ciò.

piccola tabella comparativa di introduzione all'argomento

Caratteristica	TCP	UDP
Tipo di servizio	Connection-oriented	Connectionless
Affidabilità	Sì (ACK, ritrasmissioni)	No
Ordinamento	Garantito	Non garantito
Controllo di flusso	Sì	No (a carico dell'app)
Controllo di congestione	Sì	No
Overhead	Maggiore	Minore
Latenza	Più alta	Più bassa
Uso tipico	Web, file, email, SSH, DB	VoIP, streaming, giochi, DNS

Iniziamo ad analizzare il protocollo TCP, che è lo standard di fatto usato su internet prevedendo il connubio **TCP/IP**. Precedentemente abbiamo detto che TCP consente lo smistamento dei pacchetti verso le rispettive applicazioni in ascolto su **“porte”**. Una porta è un identificativo a **16 bit** (0-65535) che identifica l'applicazione all'interno di un host. Se l'IP individua la macchina, la porta individua il **processo**/servizio/applicazione su quella macchina.

Nell'immagine sottostante c'è un piccolo esempio di porte conosciute.

- **0–1023** → *well-known ports* (standard):
 - 80: HTTP
 - 443: HTTPS
 - 22: SSH
 - 25: SMTP
- Prima di scambiare dati, TCP deve creare una connessione logica tra mittente e destinatario, non un cavo dedicato, ma un canale logico stabilito tramite protocolli di **“handshaking”**. TCP richiede quindi l'**attivazione della connessione** punto a punto tra due socket, dove un socket è uno strumento che indica chi è l'applicazione sulla

macchina identificata da quell'indirizzo IP. Il socket è quindi così composto:

(*indirizzo IP + numero porta applicazione livello superiore*)

In questo modo TCP mette in comunicazione le applicazioni in attesa sui socket.

Facciamo un breve esempio

```
socket del client      socket del server
(192.168.1.10:52133) → (93.44.12.5:80)
```

PC1 (client) invia richiesta TCP di connessione sul socket del PC2 (server) - **SYN**. Se il socket esiste e non è occupato, TCP di PC2 invia un **SYN-ACK**, un vero e proprio “ok”. PC1, invia **ACK** a PC2 e da quel momento in poi si è attivata la connessione nel canale logico, dove solo questi due socket possono parlare nella connessione. Una volta terminato lo scambio dati viene **rilasciata la connessione** e le

porte vengono liberate. Abbiamo quindi espanso il concetto di porte e socket, adesso dobbiamo parlare del **controllo di flusso** e **congestione di rete**. TCP crea un canale logico che mette in collegamento due dispositivi (anche molto distanti tra loro). Tuttavia, TCP ha dei problemi:

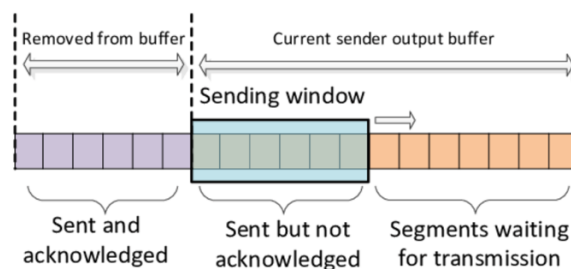
- Latenza di rete (tempo di andata e ritorno dei pacchetti) è alta, ogni pacchetto deve arrivare e poi deve confermare di essere arrivato, e fare tutto ciò con un pacchetto di pochi Mb per grossi file è lento e non sostenibile. Inviare invece tanti pacchetti, da pochi Mb tutti in una volta crea congestione dei router e dispersione dei pacchetti.

Bisogna controllare il **flusso**, inviando pacchetti al massimo ritmo sostenibile del destinatario finale, e bisogna controllare la **congestione** inviando pacchetti al massimo ritmo sostenibile dal router più lento della rete (nel percorso mittente-destinatario). Esistono diversi metodi, nel prossimo paragrafo vedremo il meccanismo **sliding window** (SW).

Una sliding window non è altro che un valore interno **sw** che rappresenta il numero massimo di pacchetti che un mittente può spedire di seguito in attesa di ricevere la loro conferma. Abbiamo detto precedentemente che inviare un pacchetto, e attendere la sua conferma è troppo lento, con questo metodo **ogni** mittente **TCP** spedisce al massimo ritmo possibile un gruppo “sw” di pacchetti. Il **flusso** viene controllato, in quanto **non vengono spediti più pacchetti oltre l'ultimo non confermato**. In pratica, se i primi sw pacchetti vengono confermati, allora vengono spediti i successivi sw pacchetti. Se un pacchetto invece non viene confermato, tale pacchetto viene rispedito prima di spedire i successivi sw pacchetti. Il controllo della **congestione** invece permette di spedire un numero variabile di pacchetti

- **Caso 1:** Sw di pacchetti sono tutti confermati → aumento sw
 - Esempio: parto con 2, poi aumento a 4, poi a 8 e così via finché...
- Un pacchetto degli Sw inviati non viene confermato (**timeout** scaduto). Per assunzione ciò è dovuto al router, dunque si riparte da Sw minimo

Il meccanismo sliding window è molto versatile in quanto si inizia ad aumentare il ritmo, finché quel ritmo non è saturo; quindi, si cerca di sfruttare sempre tutta la rete disponibile, evitando congestioni e prevenendole. In aggiunta, la “sliding window” può adottare due politiche di (Re)Transmit qualora un pacchetto di una “sending window” non sia arrivato (poniamo il secondo pacchetto evidenziato in blu dell'immagine).



- **Selective** (Re)Transmit: Ritrasmetto solamente il segmento andato in timeout (2); quindi, mantengo nel buffer di trasmissione tutti i segmenti (della sliding window blu) precedentemente inviati e “ackappati”. Debole per attacchi Ddos.
- **Cumulative** (Re)Transmit: Quando un solo pacchetto va in timeout, dropo completamente tutti i pacchetti nel buffer di trasmissione, quindi per colpa di un pacchetto, tutti i pacchetti della sliding window blu verranno eliminati, ritrasmessi e necessiteranno un nuovo ack. Questo metodo carica la rete, ma non il buffer del server, che ipoteticamente non sta lavorando solo per me, ma per diverse migliaia di utenti.

Nota! La finestra scorrevole (blu) si blocca finché tutti i pacchetti della corrente sliding window non vengono consegnati e “ackappati”.

Riassumendo, il controllo del flusso serve per evitare il sovraccarico del ricevente, mentre il controllo della congestione serve al fine di evitare la saturazione della rete, uno dei metodi implementativi è la sliding window.

DNS (Domain Name Server)

Sinteticamente permette la risoluzione dei nomi di dominio in indirizzi IP pubblici.

Usare indirizzi IP per identificare risorse in rete è scomodo e controintuitivo. Il Server DNS permette di associare a risorse di rete nomi mnemonici usando gerarchici del tipo (*nome.dominio*). Il problema è per l'appunto che i router ragionano per indirizzi IP, per loro è quindi impossibile decifrare la risorsa chiesta. Il DNS è quel meccanismo che assomiglia alle nonne di paese, sanno tutto di tutti e sono organizzati attraverso una struttura gerarchica. Il DNS sa quindi esattamente l'indirizzo IP della risorsa chiesta, e se quel server DNS non conosce quell'informazione, in modo ricorsivo si procede a consultare i server DNS che sono gerarchicamente più in alto. Questo meccanismo però ha due problemi da risolvere, ovvero l'host deve conoscere l'indirizzo IP del server DNS di diretto riferimento, e ogni server DNS deve conoscere l'indirizzo IP del suo default server, in questo modo la struttura ad albero può crearsi. Se un utente richiede una risorsa, e quel server non la conosce ci sono due modus operandi:

- Server DNS consulta i suoi predecessori, quando trova l'indirizzo IP della risorsa, lo salva in cache e lo invia al richiedente, altrimenti invia errore. Metodo lento e faticoso per i DNS
- Server DNS fornisce al client indirizzo IP del server DNS superiore, è direttamente il client che ogni volta chiede al DNS l'IP della risorsa cercata. I DNS sono più scarichi, e lavorano più velocemente.

Nota! Le richieste del server DNS viaggiano spesso in locale e quindi usano un protocollo UDP per velocità e semplicità.

Quindi, quando si cerca www.google.com in realtà si interpella un server DNS.

Livello Applicazione

Siamo quindi giunti alla busta “blu”, il livello applicazione (livello 7 stack), dove primitive e controlli vengono effettuati per spedire e ricevere dati dalle applicazioni seguendo dei protocolli specifici per ogni applicazione interpellata. Il livello applicazione si appoggia sul livello di trasporto (in particolare sul protocollo TCP), e tale livello usa un “numero di porta”, ovvero un valore numerico (16-bit) per identificare il protocollo da utilizzare (regole da seguire) per il processo che si vuole intraprendere. Il socket, accennato precedentemente è quindi: (*indirizzo IP dest + numero porta*), in questo modo client e server si scambiano dati utilizzando lo stesso standard. Facendo un esempio diventa tutto più chiaro. Quando ci si scambia un messaggio di posta elettronica (E-Mail) entra in gioco TCP, e il messaggio si basa su protocolli di livello applicazione (SMTP – invio e POP3/IMAP – visualizzazione) che vengono usati come standard di invio messaggi. SMTP utilizza la porta 25 per comunicare e il suo protocollo ed utilizza:

{ **HELO**: identifica la macchina mittente (pluto.cs.pippo.it)
{ **FROM**: mittente (pluto.paperino@truffa.it)
{ **RCPT TO**: identifica il destinatario (tizio.caio@gmail.com)
{ **DATA**: il vero messaggio da trasmettere (sei stato truffato!)



SMTP è un protocollo quindi che utilizza qualsiasi Mailer. Un altro protocollo è http, il protocollo utilizzato per la trasmissione delle pagine web. Ogni protocollo usa una porta, da vedere come un buco (comunicatore) tra il livello di trasporto e applicazione, ovvero uso TCP con questo protocollo di comunicazione Client-Server adatto per il processo che l'utente vuole attuare sulla propria macchina. In sostanza, la porta è quel **tramite** che permette di interpretare ed utilizzare i dati trasmessi, è il più alto livello di astrazione dello stack ISO/OSI.

Nota! Ogni porta ha un numero e la stessa porta può essere utilizzata per processi diversi, ma simili. Immaginiamo di aver aperto 3 browser diversi, tutti su una pagina diversa, tutti e tre utilizzano la stessa porta. Questo è un problema, bisogna capire a quale browser appartiene l'informazione. I

socket entrano in azione, avendo caratteristiche programmabili riescono ad indirizzare correttamente l'informazione verso il browser corretto, la scheda corretta, la macchina corretta, questo solo ed esclusivamente grazie all'indirizzo IP del server.

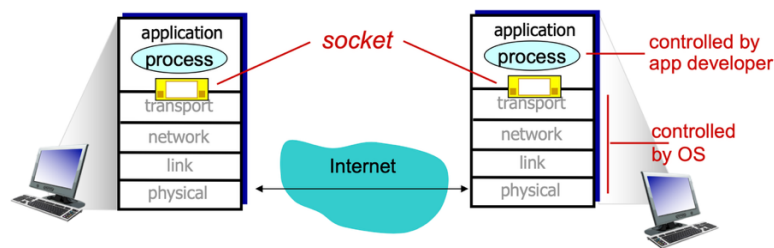


Tabella differenze TCP – UDP

TCP	UDP
Handshaking, si apre una connessione tramite un canale logico. Avviene un vero e proprio collegamento, il client prima di trasmettere apre la connessione, e la chiude una volta che termina la trasmissione. All'inizio si usa una welcoming port, e poi si usa la porta specifica. Ogni client usa un socket diverso.	No Handshaking, il mittente usa esplicitamente l'indirizzo IP del destinatario e il numero di porta. Connection-Less e Best Effort
Connection-Oriented, se un pacchetto non arriva, si ritenta. Se il servizio non è attivo e il Client prova a chiamarlo, si ha un errore.	Receive da parte del client è bloccante, se il processo non riceve nulla rimane per un tempo indefinito in attesa, è compito dell'utente riprovare trasmissione. Se il servizio non è attivo e lo si vuole usare non si incorre in errori.
Viene usato un buffer di ricezione ma anche un coda FIFO per gestire le richieste, chi arriva primo (guardando il momento preciso in cui avviene handshake) viene soddisfatto per primo. Se il Server adotta multithreading allora più richieste sono fatte in contemporanea, altrimenti un client che si è collegato per primo, ma deve ancora inviare la richiesta è bloccante per tutti gli altri. Possono essere adottate politiche difensive di kickoff per inutilizzo	Server usa un buffer di ricezione, che viene deallocato quando il client decide di chiudere la trasmissione. Non c'è priorità, chi arriva prima meglio alloggia. Non esiste una coda per i processi, nessun processo blocca altri processi

Approfondimenti

Capitolo 2

Internet Protocol (IP)

L'Internet Protocol è un protocollo di rete che mira a costruire un nuovo tipo di indirizzamento globale e gerarchico (**indirizzo IP**) che fornisce un nuovo tipo di indirizzi alla rete locale e ai suoi nodi. Questo protocollo fornisce anche **forwarding** e servizi alla connessione connectionless ed introduce nuovi dispositivi amministratori del livello di rete, ovvero i **router**.

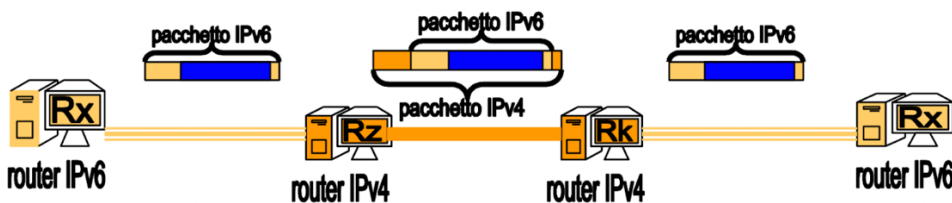
IPv6 e IPv4

Gli indirizzi IPv6 sono degli indirizzi logici (strutturati e gerarchici) che a differenza degli indirizzi IPv4 hanno uno spazio di indirizzamento gigantesco, utilizzando 128 bit, ovvero 16 byte.

Gli indirizzi IPv6 però non sono retrocompatibili con l'attuale struttura di rete; infatti, i router che "parlano" IPv4 non riescono a decifrare un indirizzo IP con 128 bit; infatti, i "vecchi" router utilizzano tabelle di instradamento diverse e sono costruiti per natura in modo differente.

Il problema, dunque è il binding, ovvero come far coesistere nello stesso ambiente indirizzi IP diversi che sono interpretabili da un'infrastruttura diversa.

La soluzione è molto semplice quanto ingegnosa. In sostanza quando un pacchetto proveniente da una rete IPv6 deve attraversare una "vecchia" porzione di Internet (composta da router IPv4) il router di confine incapsula l'intero pacchetto IPv6 all'interno di un normale pacchetto IPv4. In questo modo il pacchetto IPv6 può viaggiare sotto-copertura come un pacchetto IPv4 per tutto il tempo necessario fino al raggiungimento del router di confine. Questo tragitto di incapsulamento viene chiamato **tunnel**. Questo meccanismo permette alla rete di effettuare una transizione lenta e graduale in Internet.



Algoritmi di Routing

Gli algoritmi di routing sono quei sistemi che decidono qual è la strada migliore da far intraprendere a un pacchetto dati che parte da un mittente e giunge ad un destinatario. La comunicazione tra router avviene tramite il protocollo ICMP.

OSPF è un algoritmo matematico in cui ogni router studia l'intera topologia della rete, conoscendo tutti i percorsi possibili, conoscendo anche con precisione la larghezza di banda di ogni cavo ed eventuali guasti. Con questi dati il router può calcolare la rotta matematicamente più breve (o quella meno costosa in termini di tempo). Questo è l'algoritmo utilizzato all'interno di un Autonomous System (AS), ossia è il protocollo standard per le reti interne complesse, grazie alla sua rapidità di ricalcolo.

RIP è un protocollo vecchio, e non conosce nulla sulla rete, la sua unica metrica di routing è quanti salti (tra ogni router) un pacchetto deve compiere. Se una strada "A" è la più veloce, ma richiede 4 salti, allora l'algoritmo sceglierà la strada "B", più lenta, ma che richiede 2 salti.

BGP è un sistema di routing basato interamente su accordi commerciali tra diversi **AS**. In sostanza il traffico non viene inviato per efficienza, ma per il provider che offre il servizio migliore.

Per capire bene questo concetto è utile parlare degli **AS**. Gli AS sono una rete privata gigantesca messa a disposizione da un provider a scopo di lucro o a scopo di studio o ricerca. Il passaggio attraverso queste reti è proprio regolata dall'algoritmo di routing *BGP*.

Esempio: se si naviga su Amazon dall'università di Bologna, un pacchetto viaggia attraverso due AS, la rete GARR (italiana) e la rete di AWS (Amazon Web Services).

In sostanza, internet è l'unione fisica e logica di decine di migliaia di sistemi autonomi sparsi per il mondo, ad esempio, Amazon ha un proprio AS, come Google, e come la rete GARR

ICMP

ICMP è il protocollo dei messaggi di controllo di internet. ICMP è sostanzialmente un protocollo usato da *host*, *router* e *gateway* per scambiare informazioni di livello di rete usando pacchetti definiti con il protocollo IP. Il protocollo ICMP è sostanzialmente utilizzato per la comunicazione e la segnalazioni di problemi in rete, creando uno standard di gestione.

ICMP interviene per: percorso interrotto, indirizzo IP sbagliato, dispersione del pacchetto (TTL azzerato)...

L'applicazione **PING** è un'applicazione basata su ICMP e serve per comprendere il **RTT** tra il mittente e il destinatario. Il mittente invia un pacchetto ICMP speciale al destinatario facendo partire un cronometro. Se la destinazione è attiva, è obbligata a rispondere con un "eco" al mittente. La differenza tra istante di partenza e istante di ritorno dell'applicazione è il RTT.

L'applicazione **TRACEROUTE** è un'applicazione che invece di mandare un pacchetto normale manda un pacchetto con un TTL maggiore ad ogni iterazione, in questo modo grazie agli errori di "uccisione" del pacchetto si riesce a capire quale router ha ucciso il pacchetto, e dunque quale percorso il pacchetto ha intrapreso.

Quando il pacchetto muore su un router, tale router potrebbe essere configurato in modo da non generare e non inviare il messaggio di errore del pacchetto. Se l'host che ha lanciato il mittente non riceve la risposta entro un timeout, l'host mittente invia la richiesta per 3 volte. Se entro tre volte, il TTL verrà automaticamente incrementato e parte del percorso sarà nascosto.

La configurazione del router di non invio dell'informazione ICMP viene impostata per motivi di sicurezza. Spesso si vuole nascondere la topologia della rete, quindi per evitare attacchi di ricognizione o attacchi DDoS (ping flooding), oppure per un semplice ottimizzazione delle prestazioni.

Bisogna notare che **TRACEROUTE** non è un mezzo affidabile per valutare il congestionamento di un router, questo perché il comando traceroute costruisce la catena di router attraversata nel modo precedentemente descritto; quindi, l'applicazione è suscettibile alla fluttuazione delle code, in un dato istante è possibile che il pacchetto ICMP sia incappato in una coda piena, che all'istante dopo è libera in quanto il router ha smaltito tutti i pacchetti accumulati, senza riceverne di nuovi.

Sostanzialmente, la misurazione con traceroute non fotografa l'interno percorso in un secondo, ma lo costruisce facendo misurazioni in istanti diversi.

Inoltre, alcuni router possono adottare una politica di scheduling che assegna ai pacchetti dell'applicativo ICMP una priorità più bassa, aumentando il loro RTT.

Per testare il vero congestionamento di uno specifico router bisogna usare il comando ping, che manda 64 byte di pacchetto ICMP in un tempo continuo di tempo e ne fa la media. Se la media è alta, allora il router è congestionato

DNS Server

I server DNS permettono la traduzione di indirizzi nomi di host fittizi, logici e facilmente memorizzabili in indirizzi IP effettivi. Ci sono due tipologie di server DNS. Insieme i DNS sono strutturati gerarchicamente in una struttura ad albero.

Di base un host deve conoscere almeno un DNS server e ogni DNS server deve conoscere almeno un DNS server superiore.

I primi sono i **DNS locali** che non appartengono ad una gerarchia, ma sono una sorta di cache/proxy locale delle recenti traduzioni name-to-address. Questo è chiamato anche il default name server. Sostanzialmente è quello essenziale fornito all'host dal DHCP quando l'host si collega ad una rete sconosciuta.

Poi troviamo i **root name server** che sono contattati dai DNS locali che non riescono a risolvere un nome. Questi root contattano non conoscono l'indirizzo finale del sito, ma sanno a quale TLD DNS reindirizzarmi in base all'estensione del dominio.

TLD Server o Top Level Domain sono i responsabili di **primo livello** del dominio cercato (desinenza finale). Il compito di un top-level domain DNS è quello di dire chi gestisce quello specifico dominio di secondo livello. In sostanza il TLD fornisce un ridirezione chi fa la domanda verso qualcuno che ne sa di più. In sostanza il TLD server conosce l'autoritative DNS dei nomi che finiscono con uno dei seguenti suffissi: *.com .org .net .edu* e per i country names come *.uk .fr .ca .it*

Autoritative DNS (autoritario) è il capolinea, ovvero il server autorevole che possiede effettivamente i registri ufficiali e definitivi (come i veri e propri indirizzi IP) per uno specifico dominio. Quando riceve una richiesta il server consulta i propri file di configurazione e restituisce l'indirizzo IP esatto del server web.

Questi server sono propri dell'organizzazione, che forniscono mappature autorevoli tra nome host e indirizzo IP per gli host denominati dall'organizzazione. Possono essere gestiti dall'organizzazione o da un fornitore di servizi.

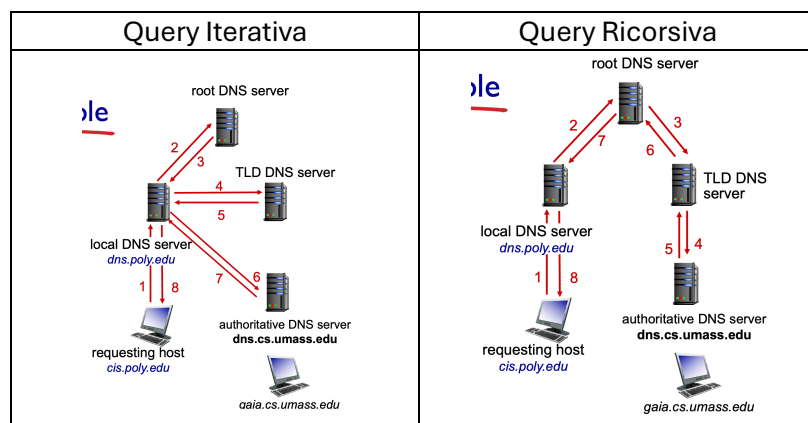
Sostanzialmente contengono i record ufficiali e definitivi per un dominio cercato.

Esempio

Si richiede www.amazon.it, il local DNS interPELLa la sua tabella, se è vuota chiede al **root DNS** informazioni sull'estensione di *“.com”*.

Il root server ridireziona la ricerca verso un **TLD DNS** responsabile di *“.com”*; a questo punto il DNS ricerca nella sua tabella se ha informazioni su *“amazon.com”*, Il TLD DNS non ha l'indirizzo IP preciso, però sa che *“amazon.com”* è gestito dall'autoritative DNS di Amazon; dunque, invita a fare un altro passaggio per completare la risoluzione.

Una volta contattati l'autoritative DNS di Amazon la soluzione è completa in quanto questo DNS sa esattamente tradurre il nome www.amazon.com nell'indirizzo IP corretto, supponiamo *“205.1.10.20”*



Quadrupla DNS

RR format: (name, value, type, ttl)

Se type = A:

Name è l'hostname e **Value** è l'indirizzo IP

Se *type* = NS:

Name è il dominio (pippo.com) e **Value** è hostname dell'autoritative DNS per quel dominio

Se *type* = CNAME:

Name è alias per nome canonico e **Value** è il nome canonico

Se *type* = MX:

Value è il nome del mail server associato a quel nome

Generalmente le richieste verso i server DNS sono inviate tramite richieste connection-less, quindi richieste UDP, questo per migliorare le prestazioni di questi server.

Approfondimenti

Capitolo 3

Socket TCP

Supponiamo di avviare una connessione SSH tra l'host 192.100.1.20/24 e l'host "pancrazio.cs.unibo.it" che supponiamo avere indirizzo IP 130.136.1.100 /16.

Nella fase di handshake il PC1 invia un SYN alla porta 22 (well known port per ssh) di "pancrazio". Pancrazio riceve la richiesta, la prende in considerazione e risponde con un SYN-ACK. Successivamente, PC1 invia l'ACK finale e la connessione è aperta.

Quando la connessione è aperta (established), la porta 22 non è più quella utilizzata nella quadrupla dei socket, ma viene creata una "sala riunioni privata" tra PC1 e "pancrazio" con le stesse identiche caratteristiche di SSH, ma questa "sala riunioni" viene spostata su un'altra porta. Ciò è dovuto dal fatto che la porta 22 è come la reception di un hotel, viene utilizzata solo per l "check-in" del servizio, poi il servizio (analogamente il pernottamento) viene effettuato altrove, mantenendo però tutti i comfort scelti al check-in.

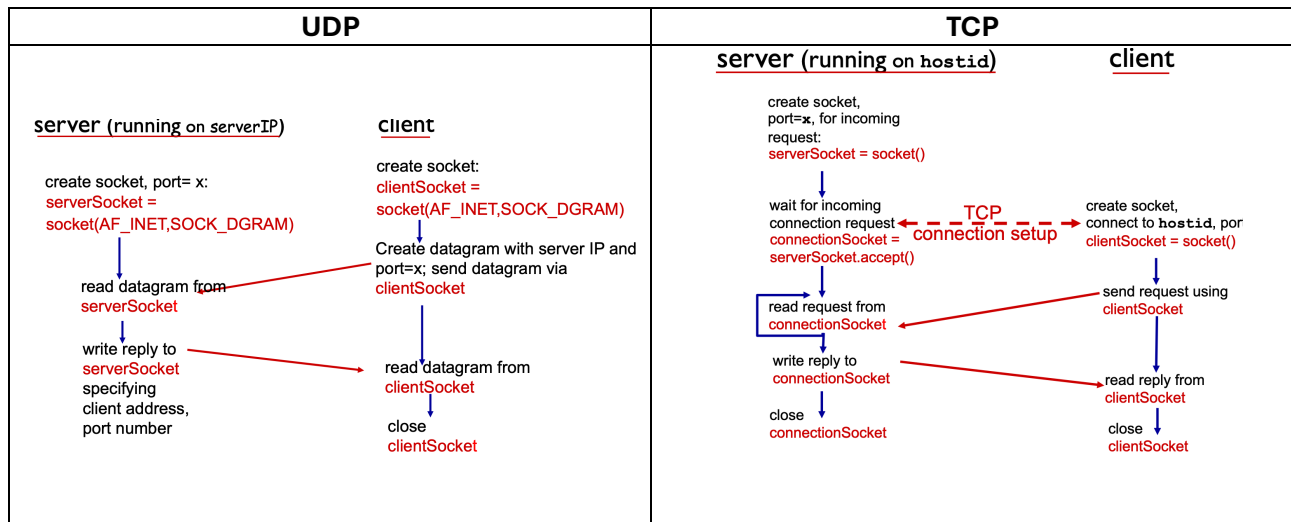
Formalmente un socket è un'astrazione software che rappresenta un endpoint per la comunicazione bidirezionale tra due processi in una rete. Intuitivamente, il processo applicativo è la casa, mentre il socket è la porta di ingresso di quella casa.

TCP/UDP

TCP e UDP non hanno encryption, quindi qualsiasi comunicazione tra client-server passa in chiaro su internet.

SSL, invece è posizionata a livello applicazione ed utilizza librerie che parlano con TCP, dunque permette una comunicazione encrypted, data-integrity e end-to-end authentication.

UDP e TCP Socket



Dettagli su ACK di TCP

In TCP gli ACK non vengono inviati uno per ogni pacchetto ricevuto, ma vengono inviati per blocchi di pacchetti. Quindi in una sliding window di 5, non mandiamo un ACK per il pacchetto 1, per il 2, ..., ma inviamo un solo ACK che dice al mittente di aver ricevuto correttamente un set di pacchetti inviati. C'è una regola che però permette l'invio di ACK riguardante un solo pacchetto, ed è quando c'è un buco nella trasmissione di un pacchetto; quindi, arrivano tutti i pacchetti fino al numero x, poi arriva il pacchetto numero x+2. Qui il destinatario manda un ACK dicendo che manca il pacchetto x+1

Livello Applicazione

l'isola blu dello stack di Internet

Architetture

Un'applicazione che funziona su diversi dispositivi ed ha come obiettivo la comunicazione tramite internet può utilizzare diverse architetture per l'implementazione.

- **Client – Server:** si hanno uno o più server (per garantire scalabilità, velocità e servizio) ognuno dei quali ha indirizzo IP permanente ed è definito “*always-on*”.
I client invece comunicano con il server in modo intermittente (non sempre la connessione client-server è attiva) tramite un indirizzo IP dinamico.
In questo tipo di architettura i client comunicano solo con i server.
- **P2P:** Non esistono server sempre accesi, ogni utente è sia client che server e la scalabilità è dovuta dal numero di utenti che utilizzano il servizio.
La comunicazione avviene direttamente tra “*peer*” fruitori del servizio. La connessione tra “*peer*” avviene in modo intermittente tra indirizzi IP diversi. Ciò ha una gestione complessa. Non esiste nessun server fisico sempre connesso, ma ogni client ha una piccola tabella contenente i “server” vicini a loro, ma tali server non sono altro che client, che vedono gli altri client come server.
Per supportare la gestione complesse dei “peer” alcuni servizi P2P possono avere un server che contiene l'elenco dei client funzionanti e partecipanti alla rete P2P.

Generalmente bisogna ricordare che un **processo cliente** è il processo che inizia la comunicazione, mentre il **processo server** è quel tipo di processo che aspetta di essere contattato.

Cosa definisce il livello Applicazione

Definire il livello applicazione non è semplice e per farlo è necessario chiarire diversi ambiti e aspetti

- **Tipologia di messaggi scambiati:** Definire le richieste e le risposte
- **Sintassi:** La sintassi utilizzata per definire quali campi evidenziano le meta-informazioni e le informazioni vere e proprie
- **Semantica:** Il significato delle informazioni espresse in una sintassi ben precisa in una parte della richiesta/risposta altrettanto specifica
- **Regole:** Definire quando e come un processo invia richieste e riceve risposte
- **Protocolli utilizzati:** Uso un protocollo proprietario, ne creo uno da zero, ne uso uno open-source oppure uso un protocollo che permette interoperabilità. Solitamente si utilizzano dei protocolli già rodati, testati ed implementati, questo permette alta interoperabilità e favorisce l'accrescimento e l'utilizzo del servizio.

Inoltre, **le esigenze del trasporto** di un app sono diverse; infatti, ci sono alcune applicazioni che richiedono che i file vengano trasmessi in modo **integro**, richiedendo il 100% di **affidabilità** (come le transazioni bancarie). Ce ne sono altre invece che preferiscono la **velocità**, altre invece che necessitano la **sicurezza** come punto fondamentale. Possiamo racchiudere questo in tre gruppi:

- **DataLoss:** Tolleranza sulla perdita di dati durante la trasmissione
- **Throughput:** Tolleranza trasmissione a livello di bit, quindi la minima banda necessaria per l'utilizzo del servizio in modo fluido e piacevole
- **Time Sensitive:** Tolleranza trasmissiva a livello di attesa, quindi la sensibilità al tempo impiegato nella trasmissione

Qui di seguito è riportata una piccola tabella che cataloga i servizi rispetto ai requisiti di data loss, throughput and time sensitive.

application	data loss	throughput	time sensitive
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5kbps-1Mbps video: 10kbps-5Mbps	yes, 100' s msec
stored audio/video	loss-tolerant	same as above	

Protocollo HTTP

HTTP è il protocollo principe per la trasmissione di risorse di rete scaricabili da siti web tramite browser, come i file HTML, i file jpeg, i file audio... Ogni oggetto che viene trasmesso con il protocollo HTTP **deve** essere individuabile tramite un URI, in particolare un URL, che però può essere usato anche per trasmettere risorse di rete con protocolli diversi rispetto ad HTTP.

La seguente struttura identifica un URL:

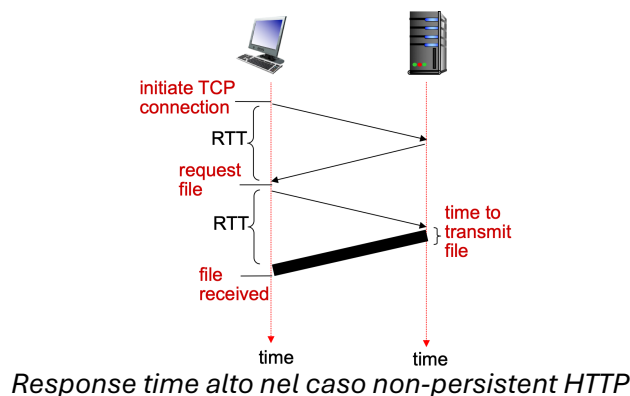
nome protocollo/hostname:portnumber/pathname che scomposto diventa

http://www.something.com:xyzk/somedepth/file

Nei browser ogni volta che si cerca qualcosa il protocollo HTTP è quello di default; quindi, non viene scritto. HTTP è un protocollo che si basa su richieste e risposte tra client e server. Il client richiede una risorsa disponibile in rete (come una pagina web) e il server tramite la response ritorna al client la risorsa richiesta. HTTP utilizza il protocollo di trasmissione **TCP**, il **welcoming socket** è sempre la porta 80, che dopo viene cambiata in una porta alta (in modo automatico) dopo l'handshake.

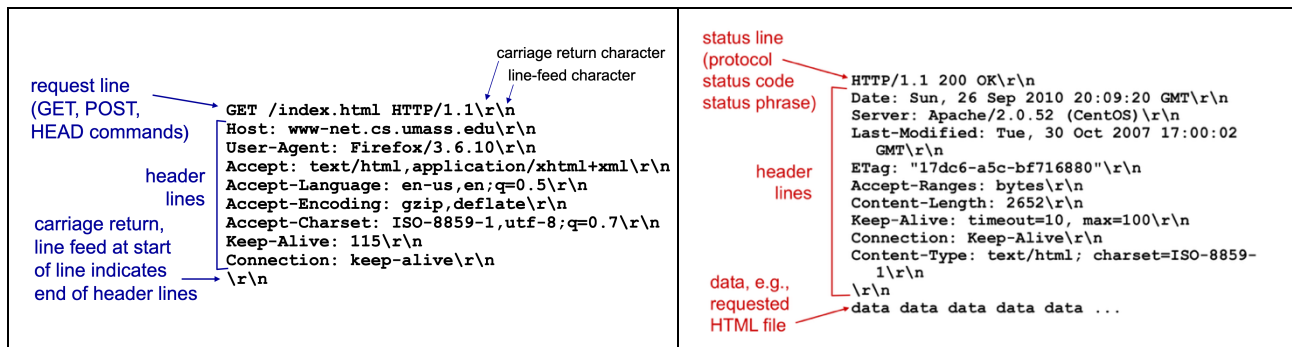
La connessione HTTP ha diverse caratteristiche:

- **Stateless**: il server non salva informazioni sulle richieste passate (oggi vengono usati cookies)
- **Non-Persistent**: di base se una risorsa richiesta dal client deve accedere ad altre risorse ogni volta la connessione tra client – server si apre e si richiude (le versioni successive sono **persistent**, ma per un periodo di tempo limitato). Questo fa viaggiare più pacchetti in rete, e non permette uno scambio estremamente veloce; tuttavia, il server non viene mai sovraccaricato di richieste continue, ne ha tante e brevi. Da qui si comprende che il tempo RTT può essere alto (inizio connessione, genero richiesta, ricevo risposta, chiudo trasmissione x tutte le volte che tale meccanismo è richiesto).
- **Metodi**: HTTP ha diversi metodi GET, PUT, POST, DELETE
- **Status Code**: usate per comunicare lo stato della trasmissione della risorsa (andato a buon fine, o errore)



Ogni richiesta e risposta è divisa in tre parti:

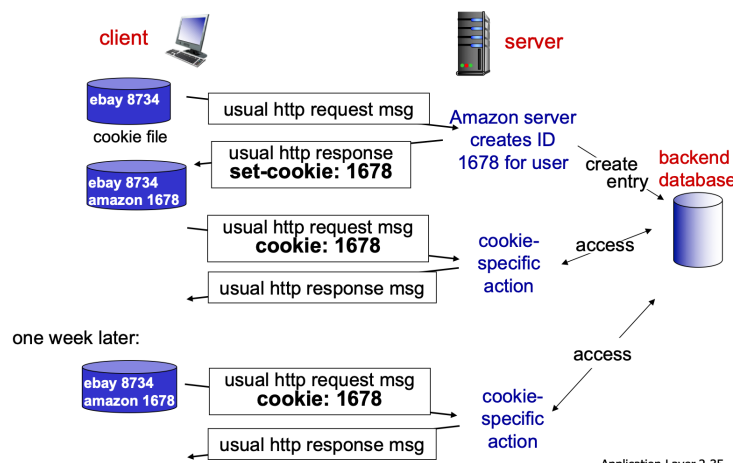
- **Request/Response Line**: viene esplicitato il metodo chiamato, l'URL richiesto, il protocollo e la versione del protocollo usato.
- **Header**: contiene le metainformazioni, come: informazioni sul browser, sulla lingua della risorsa richiesta, sull'encoding della risorsa richiesta, esplicita se la connessione deve rimanere aperta e per quanto, cookies di sessione...
- **Body**: Il corpo della richiesta e della risposta, esempio il file ritornato a seguito della richiesta



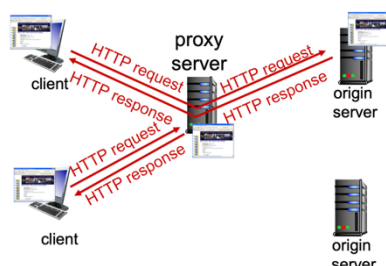
Cookie HTTP

I cookie vengono inseriti nelle header line delle richieste e risposte del protocollo HTTP e servono per effettuare un tracciamento tra client e server. HTTP di base è stateless, ma molti servizi necessitano uno storico/tracciamento per funzionare correttamente, i cookie (identificativo) quindi memorizza lo storico e lo trasmette al server, in questo modo HTTP rimane stateless, ma la comunicazione tra client e server riprende dal punto dove si era conclusa, senza dover ritrasmettere ogni informazione ogni volta. Un esempio sono le ultime ricerche di Amazon, cambiando dispositivi (computer) si hanno gli stessi suggerimenti che si avevano sul telefono.

I cookie vengono principalmente utilizzati per autorizzazioni, carrelli degli shop e-commerce e user-session state. HTTP, dunque, rimane stateless perché è il cookie trasmesso che trasmette info.



Web Cache – Server Proxy



Il proxy è un server che fa da ponte (tramite) tra client e origin server. Ogni volta che il client chiede un'informazione su internet, in realtà non la chiede direttamente all'origin server della risorsa cercata, ma al Server Proxy, il server proxy, per conto del client esegue la ricerca, la salva in cache e la restituisce al client. In questo modo, il secondo client che ricerca la stessa cosa ha l'informazione più vicino a sé (il server proxy ce l'ha già in memoria), dunque sarà il server proxy a dare una copia tale e quale all'informazione cercata, che viaggia solo in locale, senza andare ad interpellare i router e le buste

arancioni. Ad esempio, se cerco "Giallo Zafferano" il proxy che non ha l'informazione in va a cercare il sito web su Internet, interpellando l'origin server che hosta il sito web di "Giallo Zafferano". Quando il secondo client va ad effettuare come ricerca "Giallo Zafferano" il proxy restituisce immediatamente l'informazione cercata, in quanto il proxy ha già quel sito web in cache.

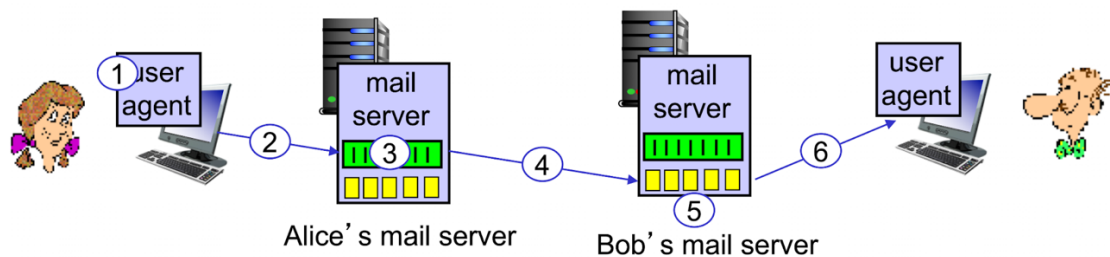
Tuttavia, se il sito web salvato in cache è troppo vecchio (mesi o anni prima), il proxy ricerca nuovamente il sito, aggiorna la sua versione in cache e poi lo restituisce al client. Il vantaggio è una

velocità migliorata (abbattimento ritardo di connessione), a discapito di un possibile attacco informatico. Infatti, se il proxy è impostato male si ha una sorta di spyware a tutti gli effetti, infatti, tutto il traffico di rete passa da quel nodo; dunque, tale nodo conterrà informazioni molto sensibili.

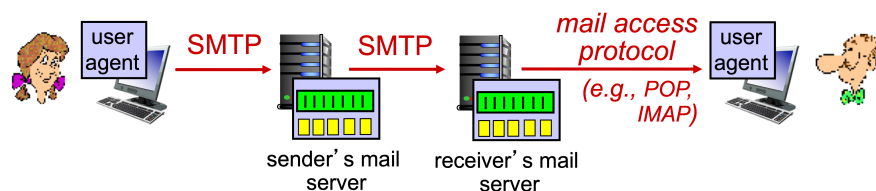
Ricapitolando, il proxy agisce sia come client che server ed è spesso utilizzato da enti grandi (come università o aziende) che vogliono ridurre il tempo di risposta e vogliono ridurre il traffico. Un proxy tuttavia fatto male avrà in cache risorse obsolete. Dunque, sono necessarie delle politiche che permettono di riportare nel proxy una pagina che ormai è diventata obsoleta.

POP3 e IMAP – Approfondimento

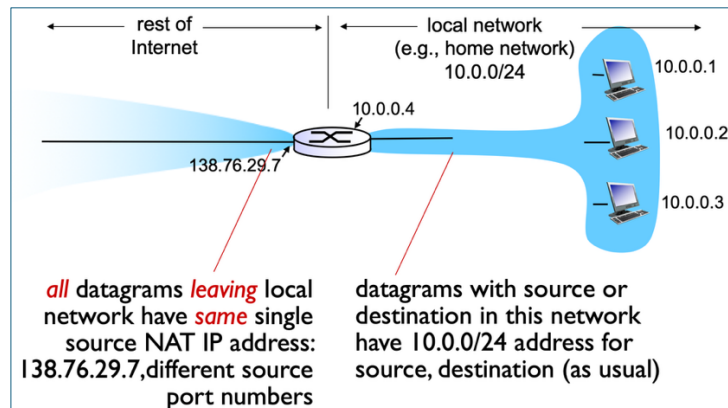
SMTP è il protocollo per il trasferimento di un messaggio di posta elettronica. SMTP per lo scambio di mail tra i mail server. Dove la mailbox è la cassetta postale e message queue è la cassetta delle mail da spedire.



POP3 e IMAP visualizzano la posta elettronica ricevuta. POP3 è più vecchio, username e password passano in chiaro, inoltre non c'è il tracciamento dei messaggi aperti sui device; quindi, se sul telefono leggo una mail, dal computer avrò la stessa mail ancora da leggere (ma in realtà io l'ho già aperta). IMAP invece è più completo con più usability, dove tutti i messaggi vengono contenuti in un server, l'organizzazione di messaggi è in cartelle e in più si ha "user state" attraverso diverse piattaforme.

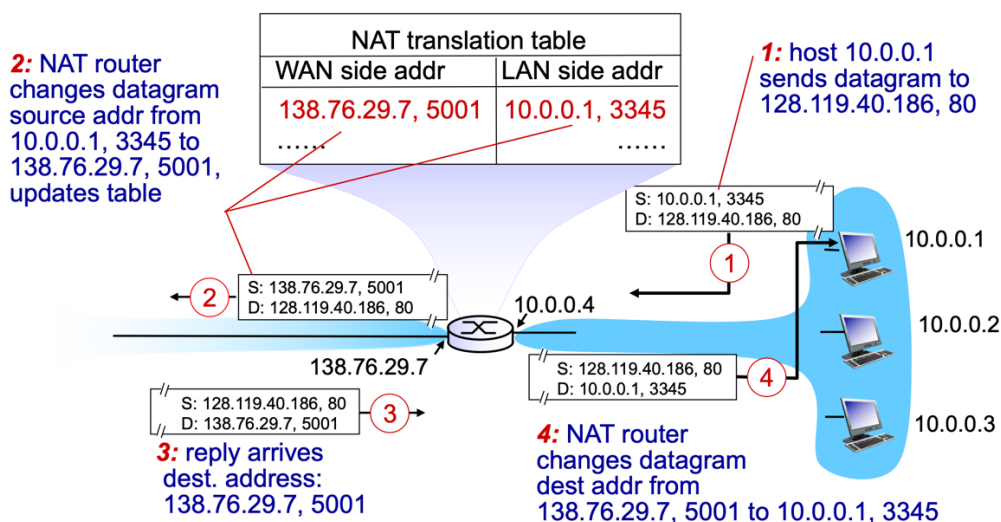


NAT – Network Address Translator



All'interno di una rete locale è di comune abitudine creare delle violazioni volute e controllate tale da "estinguere" parzialmente lo **shortage** degli indirizzi IPv4. Avendo pochi indirizzi IPv4, se tutti i dispositivi all'interno di un'abitazione (ipotizziamo 256) usassero un proprio indirizzo IP avremmo bisogno di una rete di classe C, che ci potrebbe costare svariate centinaia di euro. Con NAT, invece, ci basta comprare un indirizzo IP statico (ma anche dinamico si comporta allo stesso modo) ed associarlo al router di casa (quello che ci viene fornito dall'operatore con cui abbiamo sottoscritto il contratto). Dall'esterno, la nostra rete locale (e tutti i dispositivi connessi) è/sono visti come un unico indirizzo IP. Quello che succede dal lato interno (dentro casa), invece è completamente diverso, ovvero esiste una rete fittizia di classe A o C che assegna indirizzi IP falsi ad ogni dispositivo collegato (creando uno strato di sicurezza). Tuttavia, usare un solo indirizzo IP per definire centinaia di dispositivi, e assegnare a questi dispositivi un indirizzo IP non idoneo è un problema. Infatti, se il mio telefono collegato in rete locale (usando NAT) con un indirizzo IP pari a 10.1.1.32 provasse ad accedere alla rete esterna (quindi oltre il router di casa) dialogando con quel preciso indirizzo IP provocherebbe dei problemi, infatti si indetificherebbe in "Pippo" quando lui in realtà è "Pluto"; quindi, qualsiasi pacchetto da lui richiesto non gli verrà mai recapitato. È necessario quindi tradurre l'indirizzo IP violato con un indirizzo IP effettivo. Il router NAT (router di casa) ha quindi un compito in più, ovvero quello di prendere l'indirizzo IP del dispositivo in LAN, tradurlo in un indirizzo IP regolare, aprire la comunicazione e trasmettere le informazioni. Tuttavia, gli indirizzi IP sono sempre limitati, dunque il dispositivo in LAN acquisisce temporaneamente l'indirizzo IP del router con una porta alta, in questo modo il dispositivo comunica in WAN utilizzando un indirizzo IP completamente legale, visibile e riconosciuto.

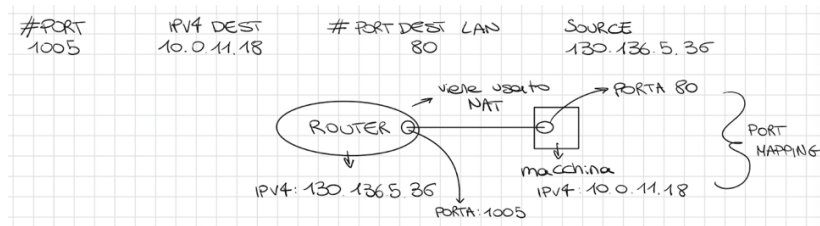
Facciamo un esempio con un immagine



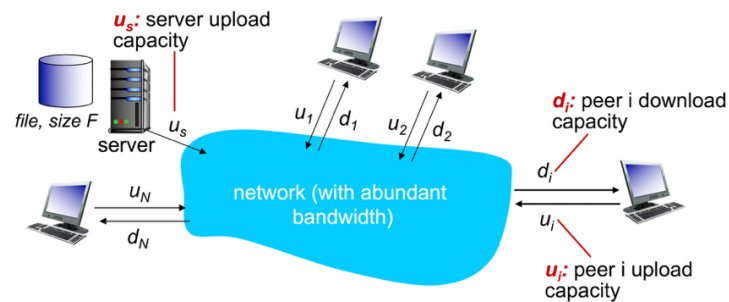
Il **port mapping** permette di accedere dall'esterno di una rete locale ad un dispositivo connesso all'interno della rete locale avendo a disposizione:

- Numero di Porta dispositivo +
- Indirizzo IP router NAT +
- Numero di Porta del Servizio della macchina a cui si vuole accedere

Il port mapping programma un comportamento esterno interno, visualizzabile nel modo sottostante:



File Distribution System



L'approccio file-distribution client-server è nettamente diverso all'approccio peer-to-peer. Analizzando il caso client-server, quando tra client invia una richiesta di download al server, i file iniziano subito ad essere scaricati dal client mentre il server fa upload. Internet non è un'area di parcheggio, i pacchetti hanno un TTL predefinito e appena viaggiano in rete devono **immediatamente** essere recapitati; dunque, quando si scarica un'immagine il client non è che aspetta a scaricare l'immagine una volta che il server ha fatto l'upload dell'immagine per intero. Una rete Client-Server è più lenta rispetto ad una P2P; infatti, può caricare solo uno, e possono scaricare in tanti, il tempo minimo impiegato è il tempo massimo necessario tra effettuare l'upload (server) e il download (client). Il collo di bottiglia è quindi l'upload del server.

$$D_{cs} \geq \max \{ NF/u_s, F/d_{min} \}$$

Dove:

F/u_s è il tempo impiegato per inviare una copia, e NF/u_s è il tempo impiegato per inviare N copie. d_{min} è il minimo download rate del client e F/d_{min} è il minimo client download time

Nelle reti P2P, invece il tempo minimo impiegato è il massimo tra l'upload del server principale (almeno una copia), la velocità di scaricamento del client, e la sommatoria di tutti i "server/client" che possono fare l'upload di parte del pacchetto.

$$D_{p2p} \geq \max \{ F/u_s, F/d_{min}, NF/(u_s + \sum u_i) \}$$

Dove:

F/d_{min} è il min client download time

$NF/(u_s + \sum u_i)$ è il max upload rate, che è la caratterizzazione specifica delle reti P2P

Come supporto, le reti P2P hanno un tracker, ovvero un server che conosce tutti gli utenti che partecipano al torrent, dove il torrent è quel gruppo di peer che si scambiano dei frammenti di file.

Il tracker è essenziale in quanto i peer che si collegano in quell'istante devono sapere quali altri client sono attivi

Le reti P2P sono ottime, ma possono collassare se tutti scaricano e nessuno fa upload; quindi, ogni persona può scaricare in misura equivalente a quanto ha fatto upload. Questo meccanismo si chiama **tit-for-tat**. Inoltre, è possibile rallentare la velocità di download di un peer che non permette gli upload agli altri peer. Queste sono solo alcune delle tante politiche per download e upload delle reti P2P.

CDN

I content delivery network sono piattaforme distribuite e sono delle vere e proprie infrastrutture di livello applicazione, sostanzialmente forniscono un servizio, il content delivery.

I **CDNs** riguardano quindi la distribuzione client-server speciale (dedicata a streaming video), ovvero i server sono geograficamente distribuiti (non un server centrale e sovraccaricato, ma tanti veloci) con diversa qualità di scaricamento (il client scarica dal server le immagini più adatte alla visualizzazione, un televisore scaricherà immagini di qualità elevata, un telefono piccolo scaricherà immagini di qualità nettamente inferiore).

DASH è esattamente il nome del servizio appena spiegato, formati diversi (chunk) con diversa qualità. DASH risparmia traffico dati che affiancato a un bitrate variabile permette di risparmiare ancora più

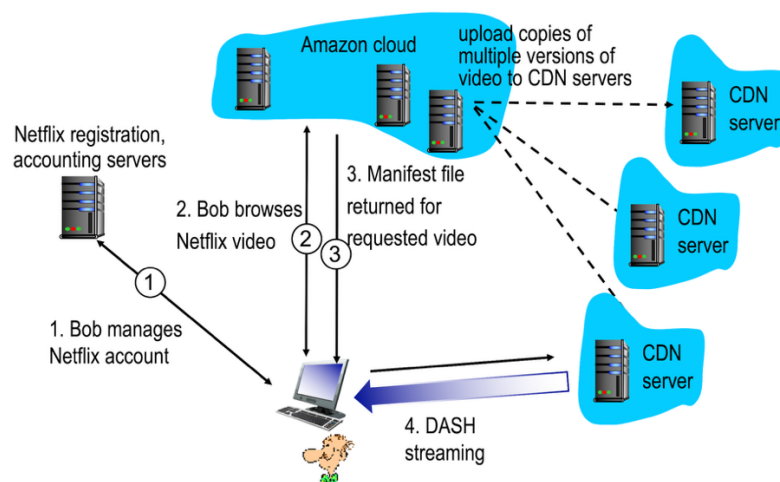
banda. DASH viene attivato anche in caso di banda scarsa, se sono sul televisore, ma ho poco bitrate si invierà un'immagine di qualità inferiore.

Spatial Coding e **Temporal Coding** sono meccanismi di invio dati che mirano a ridurre le dimensioni del pacchetto inviato permettendo una qualità visiva buona sfruttando la ridondanza presente nei dati

Lo spatial Coding si basa sulla ridondanza spaziale, ovvero le informazioni ripetute o altamente prevedibili all'interno della stessa singola immagine. Invece di memorizzare il valore esatto di ogni singolo pixel, si creano algoritmicamente degli schemi e dei blocchi di colore uniforme. Invece di inviare per 7 volte il pixel è nero, si manda un'istruzione che dice i primi 7 pixel sono neri.

La temporal coding invece si concentra sulla ridondanza temporale, ovvero le informazioni che rimangono identiche da un fotogramma al successivo fotogramma nel corso del tempo. È la politica che permette i tassi di compressione più elevati in assoluto, consentendo lo streaming fluido e molto economico (in termini di banda).

- **Bitrate Variabile:** se siamo a 24 frame/sec, e guardiamo un film, riceviamo e aggiorniamo nell'immagine solo i pixel che effettivamente cambiano secondo la politica spatial/temporal utilizzata
- **Bitrate Costante:** se siamo a 24frame/sec ogni secondo riceveremo 24frame



SICUREZZA IN RETE

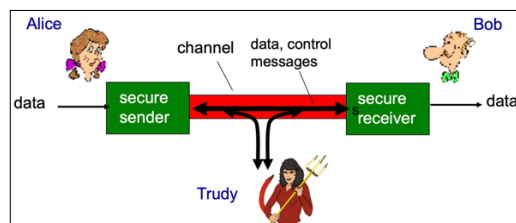
Alice, Bob e Trudy nelle comunicazioni

Per sicurezza in rete si intende:

- **Confidenzialità:** solo il mittente e il destinatario devono capire il contenuto del messaggio, per questo si ha una cifratura e decifratura del messaggio.
- **Autenticazione:** il mittente e il destinatario vogliono la conferma di star comunicando reciprocamente.
- **Integrità dei messaggi:** il messaggio scambiato deve essere integro, non alterato e non intercettato da terzi soggetti che operano malevolmente. Un messaggio non integro deve poter essere intercettato e scartato.
- **Accessibilità e disponibilità:** tutti i servizi devono essere accessibili e disponibili a qualsiasi utente

Encrypt e Decrypt

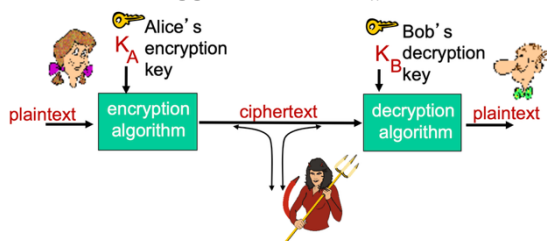
L'idea di una comunicazione sicura è avere un algoritmo conosciuto da chiunque, ma personalizzabile da utente a utente (da mittente a destinatario) che riceve un messaggio in chiaro, cifra il messaggio in chiaro e lo fa passare all'interno della rete cifrato, illeggibile e incomprensibile. Quando il messaggio giunge al destinatario, l'algoritmo funziona in modo opposto, e riesce a decifrare il messaggio, riportandolo completamente in chiaro. Questo è a grandi linee quello che viene affrontato in questo capitolo.



Ci sono diversi possibili attacchi, eccone un elenco:

- **Eavesdrop:** intercettazione dei messaggi (origliamento)
- **Injection:** messaggio modificato da un attaccante, alterandone il significato iniziale
- **Impersonation:** attaccante si finge una parte attiva della comunicazione
- **Hijacking:** attaccante si sostituisce ad una parte attiva della comunicazione
- **DoS:** crea indisposizione di servizio sovraccaricando le risorse di quella macchina

Sia m il **messaggio cifrato** e K_a una chiave di cifratura e sia anche K_b una chiave di decifratura,



i quadrati verdi sono algoritmi noti che grazie a K_a e K_b rendono il messaggio incomprensibile dall'attaccante. L'attaccante può solo fare un attacco brute-force per provare tutte le combinazioni possibili di quel messaggio. Tuttavia, questo è un attacco debole, perché più ampia è la chiave, più sono le combinazioni possibili e le permutazioni del

messaggio; dunque, l'attaccante può anche scoprire (dopo mesi o anni) la nostra chiave, ma in quel lasso di tempo noi avremo già cambiato la chiave di cifratura più volte, dunque, quello che scopre l'attaccante è inutile ed obsoleto. Il funzionamento è molto semplice:

1. Alice invia a Bob un messaggio (m) = "ciao"
2. Algoritmo cifra con K_a il messaggio: $K_a(m)$
3. Il messaggio cifrato viaggia in rete
4. Bob utilizza la chiave segreta K_b che funge da antidoto e scopre che $m = K_b(K_a(m)) = \text{ciao}$

Ci sono due metodi di crittografia:

- Chiave simmetrica: $K_a = K_b = K_s$, dove quindi la chiave segreta è uguale e condivisa sia per Bob che per Alice.
- Chiave pubblica e chiave privata.

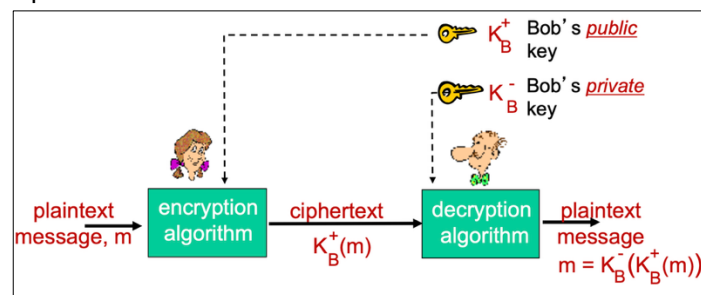
Il metodo a **chiave simmetrica** è oggi ancora molto usato e nel corso del tempo ha seguito diversi standard:

- **DES**: Data Encryption Standard. Si utilizza una chiave simmetrica da 56 bit che viene utilizzata per cifrare il messaggio a blocchi di 64 bit. Se un tempo 56 bit erano una moltitudine, oggi calcolare 2^{56} combinazioni per un calcolatore è veramente molto semplice e questione di secondi. Come standard DES si considera attaccabile in meno di un giorno.
- **3DES**: utilizza sempre il concetto di DES, quindi cifratura a blocchi, ma un messaggio viene cifrato tre volte con tre chiavi diverse. Visivamente: $K_{3a}(K_{2a}(K_{1a}(m)))$ è quello che viene inviato sulla rete, e intuitivamente molto più sicuro di DES standard
- **AES**: oggi è il vero standard, è molto più sicuro di DES e 3DES, per il semplice fatto che il messaggio viene suddiviso in blocchi da 128 bit e cifrato con chiavi da 128/192 o 256 bit. Numericamente parlando, se supponiamo DES crackable in 1 secondo con una chiave, AES impiegherebbe 149 trilioni di anni di tempo per essere crackable. È ovvio che 149 trilioni di anni è una quantità smisurata di tempo, ma supponiamo fosse possibile vivere così a lungo, in tale lasso di tempo, Alice e Bob avrebbero cambiato così tante chiavi simmetriche che l'attaccante si ritroverebbe tra le mani la primissima chiave di cifratura obsoleta e inutilizzabile

La vera soluzione di sicurezza quindi con queste chiavi simmetriche è l'utilizzo di **sessioni temporanee** che prevedono una **chiave temporanea diversa** per ogni sessione differente. Non che una chiave statica utilizzata con AES non sia sicura, ma visto che non si possono mai sapere tutte le tipologie di attacchi, e visto che non si è mai troppo sicuri, cambiare la chiave ogni tot-tempo (generalmente 30/60/90 giorni) innalza notevolmente il livello di sicurezza.

Ma la vera domanda è: come faccio a condividere la chiave simmetrica ad una persona che non ho mai visto? Sicuramente non posso inviarla in chiaro, altrimenti un attaccante potrebbe leggerla ed utilizzarla per decifrare tutte le future comunicazioni.

La soluzione è il metodo di cifratura **asimmetrico** con chiave **pubblica** e chiave **privata**, dove la chiave pubblica è nota a chiunque e la chiave privata è nota solamente al proprietario di quella chiave. La chiave pubblica viene utilizzata per cifrare, e la chiave privata è l'unico antidoto esistente per decifrare il messaggio criptato.



Immaginiamo che Alice voglia mandare il messaggio $m = \text{"pippo"}$ a Bob. Alice allora usa la chiave pubblica di Bob per cifrare $K_B^+(m)$, il messaggio dunque viaggia in rete cifrato. Arriva a Bob, che usa l'unico antidoto esistente per decifrare, facendo: $K_B^-(K_B^+(m)) = \text{pippo}$. Questo è lo standard RSA che è molto sicuro, ma che è fragile per messaggi molto brevi. Se il messaggio è molto breve, qualche carattere, e se la chiave pubblica di Bob fosse disponibile a tutti, l'attaccante potrebbe in qualche modo sottrarre la chiave pubblica al messaggio cifrato, ottenendo dunque il messaggio in chiaro, visto che le combinazioni sono poche e veloci da esplorare. Ma se il messaggio viene ingrandito con dei caratteri di padding, il messaggio diventa di conseguenza più grande, allarga lo spazio di ricerca, le permutazioni e le combinazioni da esplorare, rendendo quindi impossibile la sottrazione della chiave pubblica al messaggio. Dunque, si ha un solo risultato possibile: la chiave pubblica di un utente è

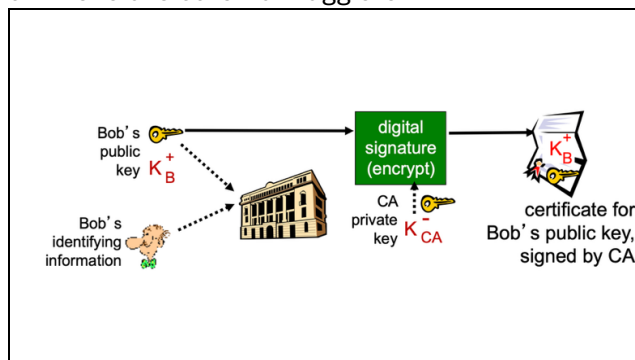
Abbiamo capito come il messaggio si può criptare. Tuttavia, vogliamo anche essere sicuri che il messaggio criptato provenga dalla persona con cui si desidera parlare, dunque il problema è come **autenticarci**, ovvero come provare al nostro interlocutore che siamo veramente noi la parte attiva della comunicazione.

The diagram illustrates the Diffie-Hellman key exchange process between Alice and Bob:

- Alice's Side (Left):**
 - Alice starts with the message m .
 - She generates a public key K_A^+ from her private key K_A^- .
 - She sends K_A^+ to Bob.
 - She receives Bob's public key K_B^+ from Bob.
 - She computes the shared secret $m = K_A^-(K_B^+(m))$.
- Bob's Side (Right):**
 - Bob starts with the message m .
 - He generates a public key K_B^+ from his private key K_B^- .
 - He sends K_B^+ to Alice.
 - He receives Alice's public key K_A^+ from Alice.
 - He computes the shared secret $m = K_B^-(K_A^+(m))$.
- Trudy's Side (Bottom):**
 - Trudy intercepts the communication.
 - She receives K_A^+ from Alice and K_B^+ from Bob.
 - She attempts to compute the shared secret $m = K_B^-(K_A^+(m))$ but fails because she does not have the private keys K_A^- or K_B^- .

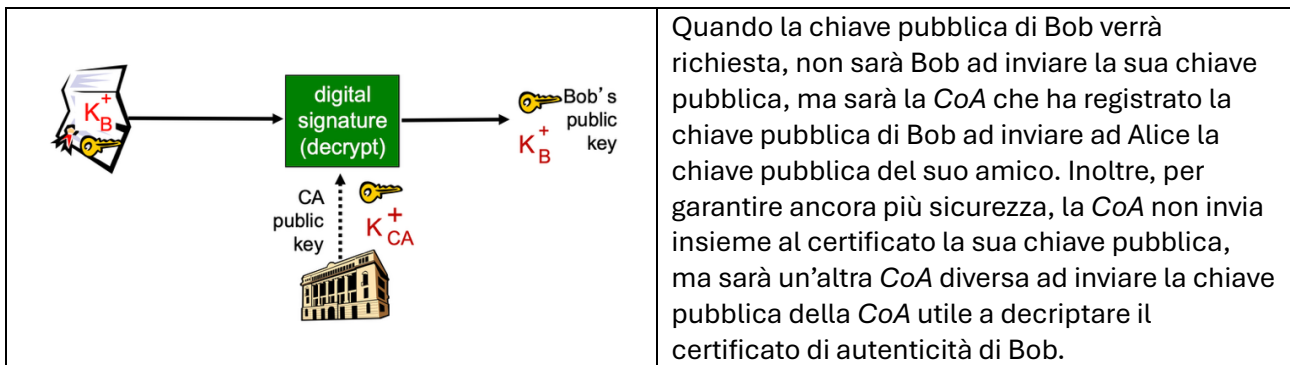
comunicando con Alice), quindi richiede ad Alice la chiave pubblica per decriptare il Nonce. Trudy lascia passare il messaggio, ma nel frattempo manda a Bob la sua chiave pubblica.

Si evince quindi che il passaggio critico, l’anello debole è l’invio della chiave pubblica. La chiave pubblica, quindi, non è da diffondere in modo spensierato, ma c’è bisogno di una azienda, chiamata “*certification of authority*” che fa da tramite garantendo l’identità digitale sia di Bob che di Alice. La certification of authority, quindi, permette di legare la chiave pubblica di una persona a quella persona in modo certo. E questo è un processo che può richiedere verifiche, che richiedono tempo, garantendo un livello di sicurezza maggiore.



La chiave pubblica di Bob viene creata dalla CoA a seguito di verifiche, in questo modo si è certi che la sua chiave pubblica gli appartenga in modo inequivocabile. Inoltre, la sua chiave pubblica viene criptata utilizzando la chiave privata della CoA, garantendo **autenticità** sulla chiave pubblica di Bob.

Ricordiamo che criptare con la chiave privata garantisce autenticità.



Nota! Solitamente le chiavi pubbliche delle CoA sono già preinstallate sul dispositivo (+ sicurezza)

In questo modo, Trudy è impossibilitata a fare da *man in the middle*.

Supponiamo ora che non ci interessi la privacy dei messaggi scambiati in rete, ma che ci interessi **l'integrità** del messaggio, ovvero che il messaggio scambiato tra Bob ad Alice o viceversa non sia stato manomesso durante il "viaggio" in internet.

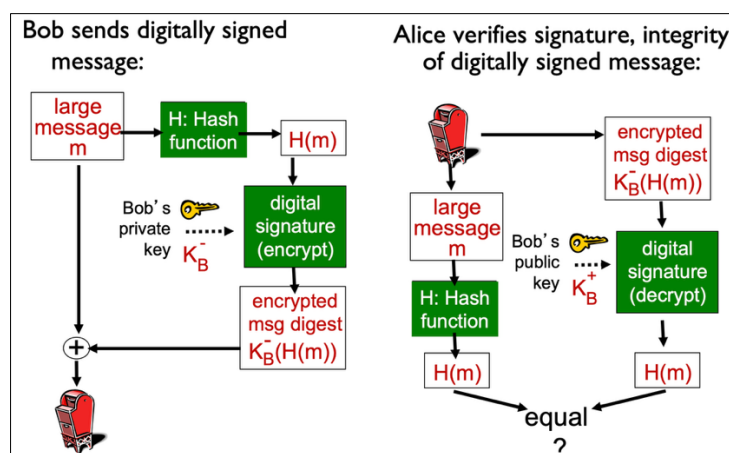
Visto che sappiamo che con RSA, cambiare un bit di un messaggio cifrato produce un messaggio decryptato privo di senso, possiamo:

1. Scrivere il messaggio da scambiare
2. Criptare con chiave privata del mittente il messaggio
3. Inviare al destinatario sia il messaggio criptato che in chiaro

Quando il destinatario riceve il messaggio deve decryptare con la chiave pubblica del mittente il messaggio criptato (che se manomesso risulterà illeggibile e privo di senso), in questo modo, se il messaggio in chiaro e il messaggio decryptato combaciano significa che la conversazione non è stata alterata in alcun modo. Tuttavia, questo metodo è molto laborioso, RSA richiede molte risorse, dunque si adotta un Hashing. In sostanza si prende il messaggio in chiaro, gli si applica una funzione di hashing (molto randomica e con proprietà matematiche potenti) che mira a restringere la lunghezza del messaggio. Successivamente:

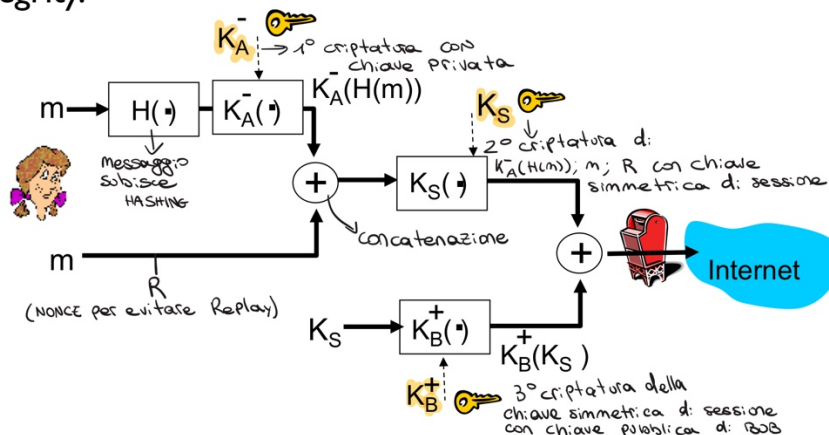
1. Viene spedito il messaggio in chiaro
2. Si invia in forma criptata il messaggio che è stato ridotto di dimensione con la funzione di hashing
3. Il destinatario, all'arrivo del messaggio applica l'hashing al messaggio in chiaro e decrypta il messaggio "hashato". Se i due messaggi combaciano, allora la comunicazione è avvenuta in modo sicuro.

Schematicamente:



Come si evince, abbiamo un messaggio firmato digitalmente. La firma digitale è fatta dunque con **hash + criptatura RSA**. Garantiamo: integrità, autenticazione e non ripudiabilità.

Adesso siamo pronti per comporre tutti i tasselli per una comunicazione: confidenziale, integra e autentica, riassunto in questo schema:



SSL – tra livello trasporto e applicazione (protegge i segreti dei processi)

Secure Socket Layer è un protocollo crittografico che si pone sopra TCP per dare *confidenzialità*, *integrità* e *autenticazione* alle comunicazioni. SSL è un protocollo di sicurezza che cifra i dati tra client e server garantendo che terzi non possano leggerli o modificarli. SSL si pone tra il livello applicazione e il livello trasporto, lavorando da intermediario impiegato nella sicurezza. SSL viene usato il **HTTPS**.

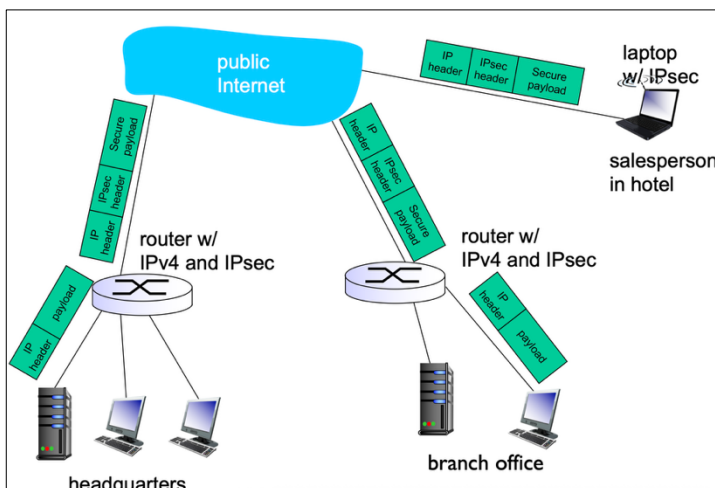
SSL in sostanza è un canale di comunicazione sicuro che avviene in quattro fasi:

1. **Handshake:** Alice e Bob usano i loro certificati per autenticarsi l'un l'altro
2. **Key Derivation:** Alice e Bob si scambiano le loro chiavi simmetriche
3. **Data Transfer:** trasferimento di dati
4. **Connection Closure:** chiusura della connessione, ogni qual volta succede (anche per errore temporaneo), la comunicazione viene riaperta e le chiavi (temporanee di sessione) vengono rigenerate

Network Layer Security – Ipsec – lavora a livello di rete (proteggendo direttamente i pacchetti IP)

Spesso aziende o istituzioni che hanno dati sensibili vorrebbero la propria rete privata cablata, ma questo non sempre è possibile, visto i costi elevati. L'alternativa a una rete propria, che garantisce una grande sicurezza è l'utilizzo di VPN (Virtual Private Network). Le VPN usano un meccanismo di sicurezza chiamato Ipsec (IPsecure) che criptano tutto il contenuto dei pacchetti (contenuto, header e trailer) e che sono logicamente separati dal traffico comune.

Ipsec lavora a livello di rete, proteggendo direttamente i pacchetti IP tra gli endpoint (client-server, host-host, ...) a prescindere dall'applicazione utilizzata (qualsiasi cosa sopra ad un indirizzo IP viene protetto). L'Ipsec è un meccanismo potente in quanto non solo fornisce un collegamento sicuro tra reti interne, ma calcolatori sotto reti effettivamente diverse, in diverse parti del mondo, si vedono come "uniti"/"raggruppati"/raggruppati sotto un'unica rete LAN.



Gli scatolotti bianchi applicano il livello di sicurezza, all'interno della rete LAN fisica i messaggi vengono scambiati normalmente, in chiaro e in sicurezza, mentre quando un pacchetto è destinato all'esterno il router che usa Ipsec cifra tutto, rendendo la comunicazione sicura. Inoltre, gli scatolotti bianchi fanno il giochino di rendere tutto una rete LAN, quindi filiali diverse, che usano NAT, non possono fornire alle macchine gli stessi IP, ma devono cambiare IP. Esempio, se nell'headquarter c'è la macchina

192.168.1.45, nella filiale non ci può essere la macchina con indirizzo IP 192.168.1.45, ci sarebbe conflitto, proprio come se fosse una rete locale.

Ipssec fornisce:

- Integrità del messaggio
- Autenticazione
- Prevenzione contro Replay Attack
- Confidenzialità

Ci sono due protocolli che mettono a disposizione diversi servizi

- Authentication Header (AH): non fornisce confidenzialità
- Encapsulation Security Protocol (ESP): autenticazione, integrità e confidenzialità

Inoltre, ci sono due tipi di incapsulamento per Ipssec

- **Transport Mode:** payload (contenuto) e header dei protocolli di livello superiore sono cifrati, mentre l'header IP originale resta visibile
- **Tunnel Mode:** Intero pacchetto originale (IP + Payload + header + trailer) vengono criptati e incapsulati in un nuovo pacchetto IP. Questa modalità è tipicamente utilizzata con le VPN.

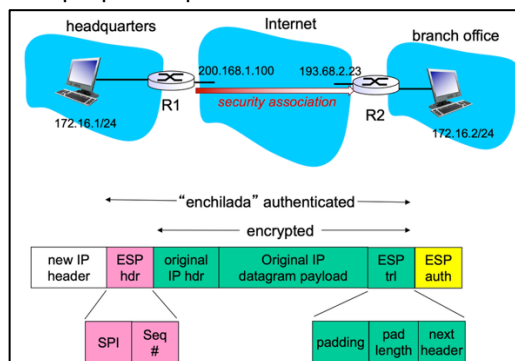
Transport Mode: utilizzato nelle comunicazioni dirette da host a host

Originale: *IP | DATI*

Transport: *IP | ESP – Header | DATI | ESP – Trailer*

Tunnel mode: standard delle VPN, utilizzato nelle comunicazioni da Gateway a Gateway o da host a gateway. I reali indirizzi IP vengono nascosti, evitando un net scanning/mapping.

Tunnel: *New – IP | ESP – Header | IP | DATI | ESP – Trailer*



Schema Grafico di Tunnel Mode

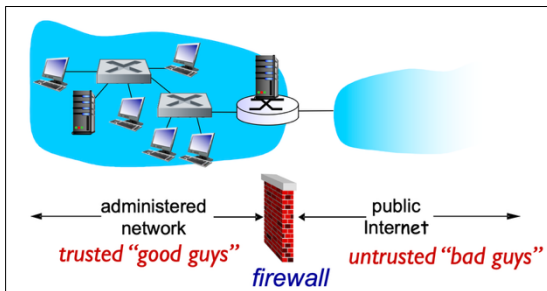
Il tunnel tra router R1 a router R2 si chiama security association ed è un vero proprio tunnel.

SPI è il security Index, mentre SEQ# è il numero di sequenza. La sezione verde invece è il pacchetto IPv4 normale e standard. Il MAC (Meccanismo autenticazione Contenuto) è posto in ESP auth ed è creato con una chiave simmetrica.

Nota! IP è connectionless, mentre Ipssec è connection-oriented

Firewall

I firewall sono soluzioni adottate da organizzazioni che vogliono mantenere la loro rete LAN interna il più sicura possibile. Il firewall, come dice la parola stessa è un muro di fuoco che separa la rete aziendale interna da internet. I firewall agiscono a livello di rete e di trasporto.



A sinistra del firewall troviamo la rete aziendale, che si presume essere “sicura”, ovvero tutto quello che circola all’interno della LAN non è di competenza del firewall, e si presume essere sicuro. A destra del Firewall troviamo internet che per definizione è insicuro. Il firewall ha il compito di lasciar transitare sia in ingresso che in uscita determinati pacchetti, discriminati utilizzando diversi algoritmi e modelli. Però il firewall a volte può essere sia troppo potente, che minare la sua sicurezza.

Immaginiamo che il sito dell’azienda sia hostato da una macchina all’interno della LAN. Se un utente accede al sito (concesso dal firewall) e in qualche modo riesce a far partire un attacco, la sicurezza interna viene a mancare, e il firewall non può farci nulla, la LAN non è di suo interesse. Dunque, solitamente si aggiunge una terza strada “verso il basso” che permette di accedere a una “DMZ” (zona de militarizzata) che non è sicura come la LAN, ma è un luogo dove ci devono stare tutti quei servizi che in qualche modo devono essere accessibili da terzi (come la macchina che fa hosting del sito web). In questo caso, i terzi non accederanno mai alla LAN, ma a questa DMZ.

I firewall sono quindi strumenti per evitare attacchi come:

- **DoS:** il firewall fa discriminazione di ciò che entra e come entra, dunque se vede attività sospette, tramite opportuni algoritmi blocca tutte le successive richieste da quell’host, escludendolo (droppando ogni suo pacchetto) che viene ricevuto.
- **Modifiche non concesse o non autorizzate** da parte di terzi a servizi nevralgici della rete
- **Ingresso di persone non conosciute o non autorizzate**, escludendo un set di host.

e possono essere di tre tipi:

- **Stateless packet filters:** guardano e analizzano pacchetto per pacchetto senza considerare il contesto che quel pacchetto si porta dietro. Decidono se inoltrare o lasciare il pacchetto in base a:
 - Indirizzo IP mittente e destinatario
 - Sorgente TCP/UDP e numeri di porte di destinazione
 - Tipi di messaggi ICMP
 - SYN (Sincronizzazione) e ACK (Acknowledgment) bits in TCP
- **Stateful packet filters:** tengono conto del contesto, guardano quindi le sessioni, e una volta che la sessione è sicura, tutti i pacchetti vengono considerati sicuri. Decide se attivare o meno connessioni tra client-server, ad esempio:
 - Blocca tutto il traffico TCP da interno ad esterno che prova ad effettuare connessioni http a porte specifiche. Esempio: tutte le richieste alla porta 22 (SSH) si scartano.
- **Application gateways:** applica le regole definite dalla semantica del protocollo applicazione

Esempi di stateless packet filtering

<i>Policy</i>	<i>Firewall Setting</i>
No outside Web access.	Drop all outgoing packets to any IP address, port 80
No incoming TCP connections, except those for institution's public Web server only.	Drop all incoming TCP SYN packets to any IP except 130.207.244.203, port 80
Prevent Web-radios from eating up the available bandwidth.	Drop all incoming UDP packets - except DNS and router broadcasts.
Prevent your network from being used for a smurf DoS attack.	Drop all ICMP packets going to a "broadcast" address (e.g. 130.207.255.255).
Prevent your network from being tracerouted	Drop all outgoing ICMP TTL expired traffic

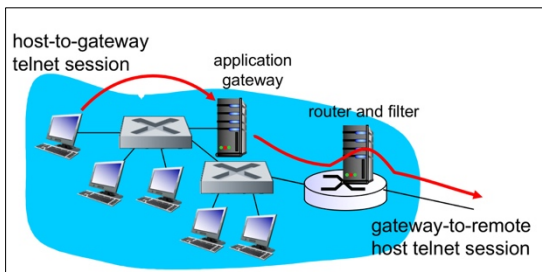
La Access Control List (ACL) è una tabella di impostazione del firewall, da regole più importanti (che vengono applicate per prime) a regole meno importanti (che nella cascata vengono applicate in seguito). In sostanza, entra un pacchetto, se il pacchetto è racchiuso nella prima categoria allora si consente o no il suo passaggio, se il pacchetto non è racchiuso nella prima categoria si passa alla seconda, e così via. Se il pacchetto non è in nessuna categoria, allora si ha una situazione “default” con la decisione finale da intraprendere per qual pacchetto (solitamente, il default è un denial).

action	source address	dest address	protocol	source port	dest port	flag bit
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80	any
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	---
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	----
deny	all	all	all	all	all	all

Solitamente, viene preferita una selezione **stateful** per due motivi. In primis si ha un filtraggio più consapevole, rapido e veloce. Inoltre, il metodo stateless è molto pesante, ovvero richiede tante computazioni, ma richiede anche di aprire completamente il pacchetto, quindi guardare completamente il contenuto del pacchetto. Nel modello stateful, inoltre la ACL può

anche effettuare un controllo della condizione, ovvero si va a vedere se il pacchetto, di quella connessione è coerente con tale connessione, se è coerente bene, altrimenti i pacchetti vengono droppati. Le stateful ACL hanno anche un campo **check condition** dove viene detto se controllare se il pacchetto è coerente con il flusso della connessione.

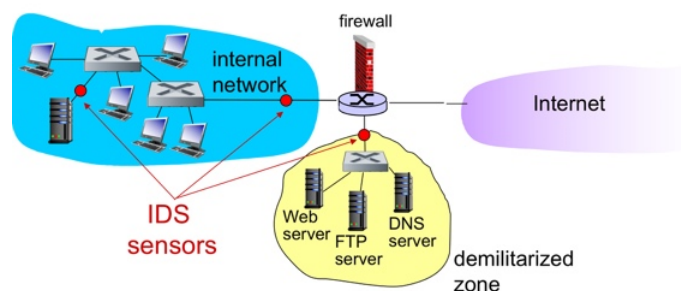
Se i firewall oltre a guardare cosa entra guardano anche cosa esce la connessione aziendale può subire rallentamenti. L'idea allora è quella di creare una struttura simile a quella dell'immagine, ovvero avere un gateway che si occupa di gestire le richieste client-server. Se tali richieste vengono approvate, allora il firewall butta fuori tutti i pacchetti che gli arrivano dall'interno e li spedisce verso l'esterno senza problemi. Se le richieste vengono rigettate i pacchetti non passano neanche dal



firewall. Dunque, per il firewall la rete interna è sempre sicura e tutto può transitare verso l'esterno, questo perché il lavoro sporco viene fatto dal gateway. D'altro canto, il firewall si preoccupa solo di gestire la situazione: *esterno* → *interno*.

I firewall aumentano la sicurezza, ma non sono invincibili, se un indirizzo IP considerato sicuro accede,

il firewall lo fa passare, ma se tale indirizzo IP era stato rubato con uno spoofing e ad entrare è una persona disonesta, la sicurezza viene a mancare. Ecco perché all'interno della rete LAN vengono inseriti degli IDS (Intrusion Detection Systems) che effettuano uno scan molto approfondito, e magari anche invasivo dei singoli pacchetti che passano nella rete LAN. Gli IDS guardano proprio il payload e esaminano la correlazione tra pacchetti multipli, cercano degli eventuali port scanning, network mapping e DoS attack.



Ci sono anche altre limitazioni dei firewall e dei gateway; infatti, gli host devono conoscere il gateway, devono conoscere l'indirizzo IP, altrimenti non riescono a comunicare.

Approfondimenti

Capitolo 4

Rompere degli schemi di crittatura

- Attacco con solo il testo cifrato
 - Tentativo lungo, ma che per un numero di tentativi che tende all'infinito ha sempre una soluzione. Questo è il modello brute force.
 - L'analisi statistica invece è un altro modello di aggirare la crittatura e si basa sullo studio analitico dei pattern nel testo cifrato per **dedurre** informazioni sul testo in chiaro o sulla chiave.
- Attacco con testo in chiaro noto
 - Trudy ha a disposizione un testo in chiaro corrispondente a un testo cifrato; dunque, Trudy in qualche modo riesce a determinare le corrispondenze (accoppiamenti per sostituzione= per le specifiche lettere note derivate dal messaggio).
- Attacco con testo in chiaro scelto
 - Trudy ha la capacità di ottenere il testo cifrato per un testo in chiaro da lei appositamente scelto.

DES standard – algoritmo

64 bit di testo e 64 bit di chiave, di cui 8 sono riservati per il controllo; dunque, effettivamente si hanno 56 bit di chiave. Des ripete un ciclo di operazioni per 16 volte.

Dopo una permutazione iniziale, il testo da 64 bit viene diviso in due metà uguali da 32 bit. Ad ogni iterazione del ciclo, la parte di sinistra diventa la metà destra prima di subire la crittatura, mentre la metà di destra viene calcolata utilizzando un XOR tra la parte sinistra e l'output di una funzione di trasformazione sulla parte destra.

Algebricamente: $L_i = R_{i-1}$ e $R_i = L_{i-1} \text{ XOR } f(R_{i-1}, K)$

Una volta finiti i 16 round di cifratura la parte sinistra e destra vengono scambiate nuovamente e viene ricomposto l'output a 64 bit.

RSA – Tecnica Padding

L'algoritmo RSA è puramente deterministico (funzione matematica), ciò significa che un testo piccolo cifrato sarà sempre lo stesso identico blocco di numeri, ogni singola volta che viene cifrato con quella specifica chiave pubblica.

Visto che la chiave pubblica di una persona è nota a chiunque, si ha la vulnerabilità di un attacco a dizionario, ovvero:

- Trudy intercetta il messaggio
- Sospetta in un qualcosa
- Si scarica una mappa di tutte le parole possibilmente plausibili
- Le cifra una ad una e prima o poi beccherà il messaggio che è stato inviato.

Con l'aggiunta del padding il determinismo di annulla, generando un testo cifrato completamente diverso ad ogni esecuzione. In sostanza, il padding (supponiamo a 128 bit) aumenta lo spazio di ricerca, dunque ogni singola parola (anche corta) inviata dovrebbe essere testata con tutte le 2^{128} possibili valori del padding. Brute force è sempre possibile, ma tremendamente lungo.

Idea di RSA

Carattere = bit $\rightarrow$ bit = numero $\rightarrow$ la crittatura di caratteri è equivalente a criptare dei numeri.

Presi due numeri primi p e q molto grandi da 1024 bit l'uno.

Sia $n = p * q$ e $z = (p-1)*(q-1)$.

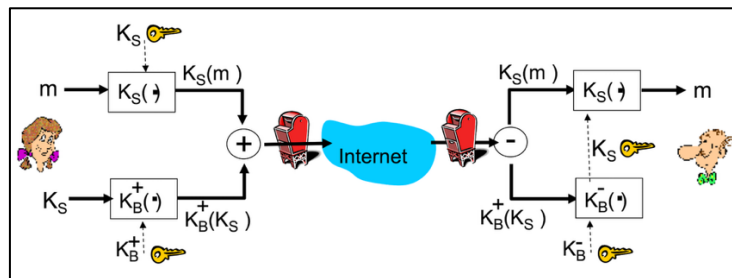
Sia scelto un numero e minore di n che non ha fattori comuni con z ; dunque, il massimo comun divisore tra z ed e è 1.

Scelto un numero d tale che $ed \% z = 1$ si ha:
Chiave pubblica: (n, e) , chiave privata (n, d)

Sia scelto un messaggio $m < n$:
Per criptare: $c = m^e \% n$
Per decryptare: $m = c^d \% n$

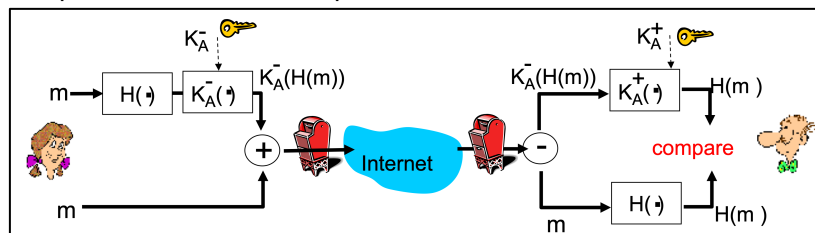
Schemi default per obiettivi diversi nello scambio di messaggi

Invio di una e-mail **confidenziale** da Alice a Bob

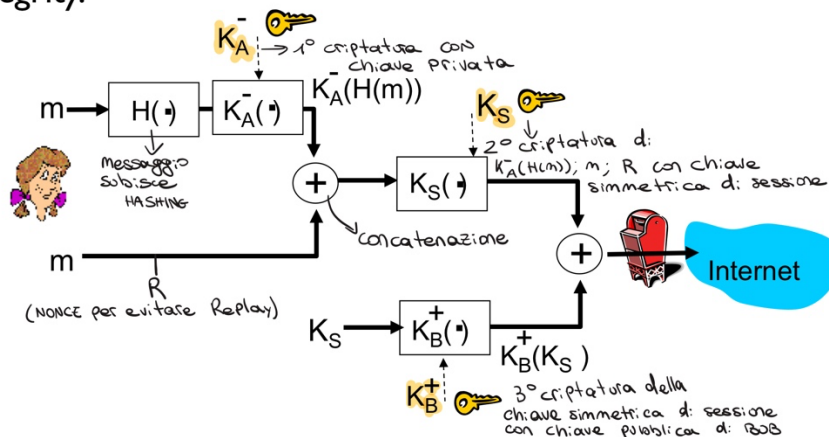


Invio di una mail con **firma digitale** del mittente e **integrità** del messaggio, **non ripudiabilità**.
Questo è lo schema (invio di Alice) di una **firma digitale**.

Se si vuole aggiungere **autenticazione** del mittente si usa la **challenge-response**, con la conseguente aggiunta del nonce, criptato con chiave **RSA** privata del mittente



Invio di una mail **confidenziale**, con **autenticazione** e **message integrity**



Nota! R si aggiunge solo per evitare il replay attack o per Autenticazione

Qualsiasi altro utilizzo di R è da considerarsi eccessivo

Supportare sempre l'utilizzo di AES per chiave simmetrica e RSA per chiave pubblica e privata.
In quanto questi metodi sono gli unici che aggiungono automaticamente del padding al messaggio cifrato.

SSL dettagli

SSL dispone di una serie di cipher suite, ovvero il set dei vari algoritmi di cifratura. Logicamente, il client e il server devono mettersi d'accordo sulla cipher suite da utilizzare, altrimenti la comunicazione non è comprensibile. Solitamente, il client offre le proposte e il server ne sceglie una tra quelle proposte.

Handshake è la parte cruciale, dopo si è sicuri di una comunicazione protetta

- Autenticazione del server
- Negoziazione sul cipher da utilizzare
- Scambio di chiavi
- Autenticazione del client (opzionale)

Nell'handshake si inviano anche una copia criptata di tutti i messaggi scambiati tra client e server durante handshake, questo serve ad evitare che qualcuno abbia fatto da man in the middle e abbia rimosso i cipher sicuri e mantenuto solo i cipher deboli.

NETWORK LAYER

Routing dei Router

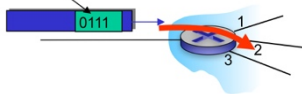
Forwarding e Routing

Forwarding è quando un pacchetto arriva da una porta di ingresso del router ed esce dalla porta in uscita. In sostanza, il forwarding di un pacchetto è il passo che compie il pacchetto per andare dalla porta in input del router A alla porta di output del router A. Questo è implementato nel **data plane**. Il Routing è la decisione intrapresa dal router per scegliere la strada da far fare al pacchetto. Il routing è in sostanza la visione della rete che ha un router. Questo è implementato nel **control plane**.

Data Plane opera nell'ambito del forwarding, quindi è una funzione locale del router, in quanto determina su quale porta far uscire un determinato pacchetto, con un determinato header.

values in arriving
packet header

immagine a sinistra è il Data Plane



Il **Control Plane**, invece opera network-wide, quindi determina come il datagram è instradato tra gli altri router lungo il percorso end-to-end da mittente a destinatario. Ci sono due diversi tipo di approcci Control Plane,

il primo è l'approccio locale, mentre il secondo è l'approccio centralizzato SDN.

Intuitivamente, il Data Plane è il singolo svincolo autostradale, mentre il Control Plane è il navigatore, che suggerisce la strada migliore per arrivare alla destinazione.

In ogni router si ha la forwarding table. Questa tabella viene consultata ogni qual volta il pacchetto giunge alla porta di input del router, e in base alle informazioni contenute nei primi bit dell'header del pacchetto, il router decide su quale porta di uscita eseguire il **forwarding** del pacchetto. In sostanza, il control plane e il data plane sono due funzioni di supporto, in quanto il **control** ha la visione della rete, e decide/attua il cambiamento della visione della rete in base alle irregolarità della rete stessa (colli di bottiglia, malfunzionamenti brevi o persistenti), ma tali informazioni relativi ad "errori" della rete vengono comunicate dall'hardware di rete che se ne accorge e lo comunica al Control Plane. Il Data Plane è progettato per essere basilare e veloce, prende i pacchetti, ne legge l'header, guarda la sua tabella, e li spara sulla porta di output. Non ha logica per dialogare sui malfunzionamenti.

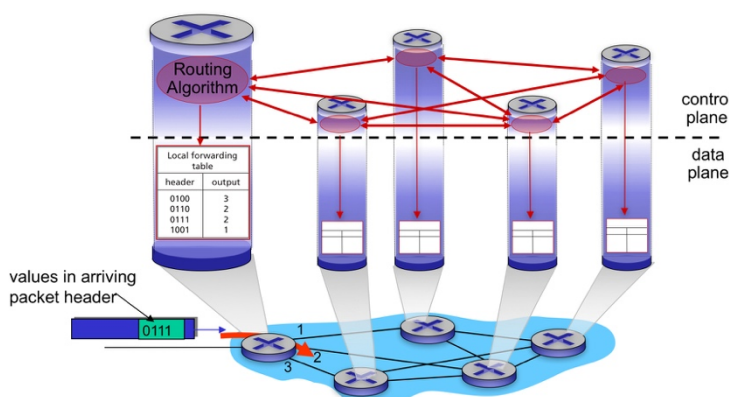
Riassumendo, i due plane lavorano in totale sinergia, il **Control Plane monitora** lo stato della rete, quando rileva un cambiamento della topologia (dovuto a guasti o colli di bottiglia), ricalcola i percorsi e aggiorna la forwarding table del Data Plane (continuando ad instradare pacchetti a velocità folli, seguendo le nuove direttive).

Control Plane locale/SDN

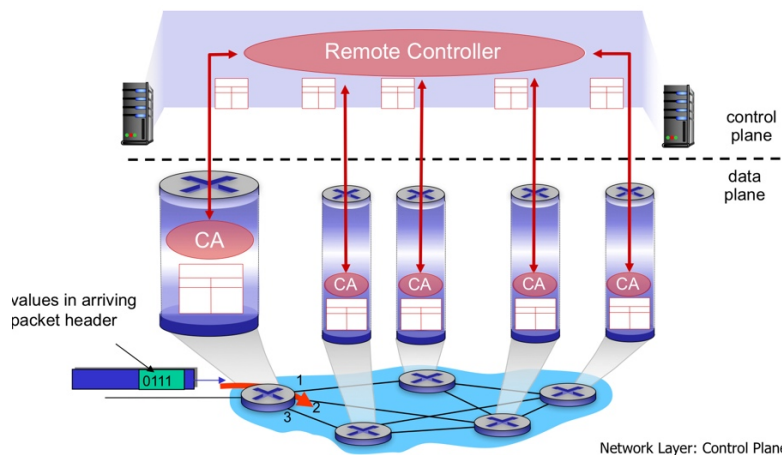
L'immagine qui accanto raffigura l'approccio tradizionale, l'algoritmo di routing e forwarding sono

elaborati localmente sul router. Questo approccio locale è quello tradizionale, ogni router è una macchina intelligente che calcola le rotte in modo autonomo. Il primo vantaggio è la robustezza, se un router si guasta, gli altri se ne accorgono immediatamente (tramite passaparola), ricalcolano i percorsi e la rete sopravvive sempre. Inoltre, questo meccanismo è User-Friendly, basta inserire un nuovo router nella rete, configurare il protocollo utilizzato e in automatico il nuovo router si integra

automaticamente nella rete. Lo svantaggio è che ogni router ha la **propria** visione delle rete, e ogni router calcola il percorso più breve facendo localmente dei calcoli per conto suo. Questo implica che

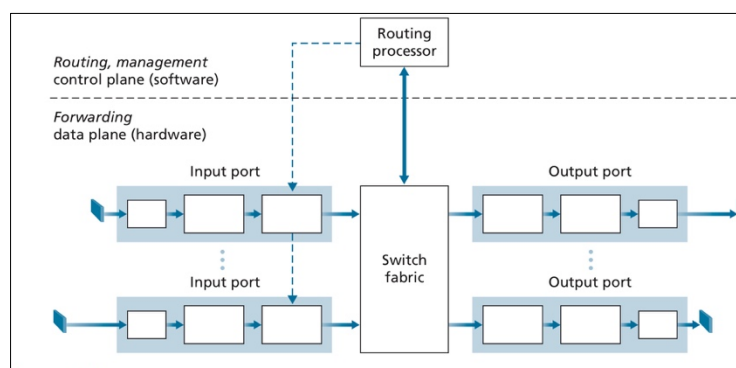


tutti i router potrebbero scegliere la stessa autostrada, intasandola e lasciando vuote strade più lunghe, ma più veloci. Inoltre, quando c'è un guasto serve del tempo affinché il passaparola tra tutti i router venga portato a termine. Inoltre, utilizzare questi algoritmi in locale necessita di hardware sofisticato, quindi CPU potenti e più RAM.



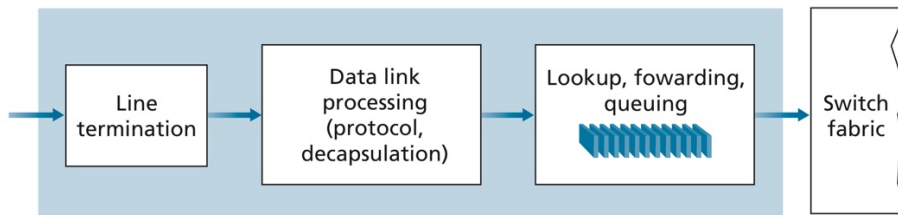
Tuttavia, oggi è molto di moda una tabella di forwarding condivisa (SDN), dove tutti i router hanno una copia in locale di una tabella di forwarding calcolata globalmente da un server esterno che prende informazioni di stato della rete da una serie di router. Con questo approccio si ha una visione globale perfetta e un'ottimizzazione globale perfetta, il controller remoto vede l'intera rete in tempo reale. Per aggiunta i router diventano economici, eseguono solo ordini; dunque, il loro hardware è meno costoso. Inoltre, si ha una distinzione tra politiche e meccanismi, il meccanismo è del router, ma la politica è del remote controller; quindi, se si vuole cambiare il modo in cui funziona la rete basta cambiare la politica del remote controller e sistematicamente tutti hanno le nuove regole intraprese. Il contro è parecchio evidente, se il controller centrale si blocca, o la connessione tra i router cade, la rete perde il cervello, i router sono in panne, non sanno più cosa fare. Per risolvere questo problema è necessario fare un'infrastruttura di backup (costosa e difficile). Infine, si ha latenza nella copia delle tabelle di forwarding calcolate dal remote controller.

Analisi della struttura di un router



Le porte di ingresso sono costituite da tre blocchetti, il primo è il layer fisico, dove arrivano i segnali (bit-level reception), il secondo è il layer di binding, da segnale a bit, il terzo invece è la coda dei pacchetti arrivati e in attesa di essere instradati su una porta. Le porte di uscita sono uguali, soltanto speculari alle porte di entrata. Lo switch fabric è invece è collegamento tra le porte di entrata e di uscita, è contenuto nel router ed è una sorta di rete dentro la rete, perché smista i pacchetti in arrivo sulle porte di entrata alle porte di uscita. Il routing processor invece è control plane, che nei router tradizionali è fisico ed individuale, mentre nei router moderni è centralizzato.

Guardando con uno zoom la porta di ingresso si vede la seguente struttura:



La line termination è dove fisicamente il cavo si innesta al router; quindi, è il punto dove terminano i segnali, e inizia i bit. Il data Link è l'esatto punto in cui si passa da segnale a bit e dove si fa un lookup; quindi, si sceglie già la strada che si dovrà intraprendere quando si entra nello switch fabric. E poi c'è la coda di attesa dei vari pacchetti. Lo switch fabric quindi applica l'instradamento grazie al lookup eseguito precedentemente, e si occupa di smistare il pacchetto verso la porta di uscita utilizzando un'interfaccia che applica delle discriminazioni sui lookup dei pacchetti. Le discriminazioni possono essere di due tipi: **destination-based**, dove i pacchetti vengono instradati guardando l'indirizzo IP del destinatario, o **generalized forwarding**, dove il router si comporta da ispettore doganale, dove può quindi vedere, analizzare e decidere in base a molti campi del datagram arrivato. Noi prendiamo in considerazione il **destination-based** e lo studiamo in due modalità: Address Range e **longest prefix method**.

Destination Address Range	Link Interface
11001000 00010111 00010000 00000000 through 11001000 00010111 00010111 11111111	0
11001000 00010111 00011000 00000000 through 11001000 00010111 00011000 11111111	1
11001000 00010111 00011001 00000000 through 11001000 00010111 00011111 11111111	2
Otherwise	3

La tabella precedente mostra un **destination based forwarding**. Quando il pacchetto arriva, il router cerca in quale range della sua tabella l'indirizzo IP di destinazione risiede e procede nell'indirizzamento. C'è anche un'altra metodologia di instradamento verso la porta di uscita, che prende il nome di **longest prefix method**. Tale metodo non è molto utilizzato in quanto non applica una buona discriminazione e quindi può generare dei conflitti. In sostanza, in quest'ultimo intervallo non c'è un intervallo di indirizzo, ma ci si basa su prefissi. Tale approccio in alcuni casi può essere non deterministico.

Destination Address Range	Link interface
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
otherwise	3

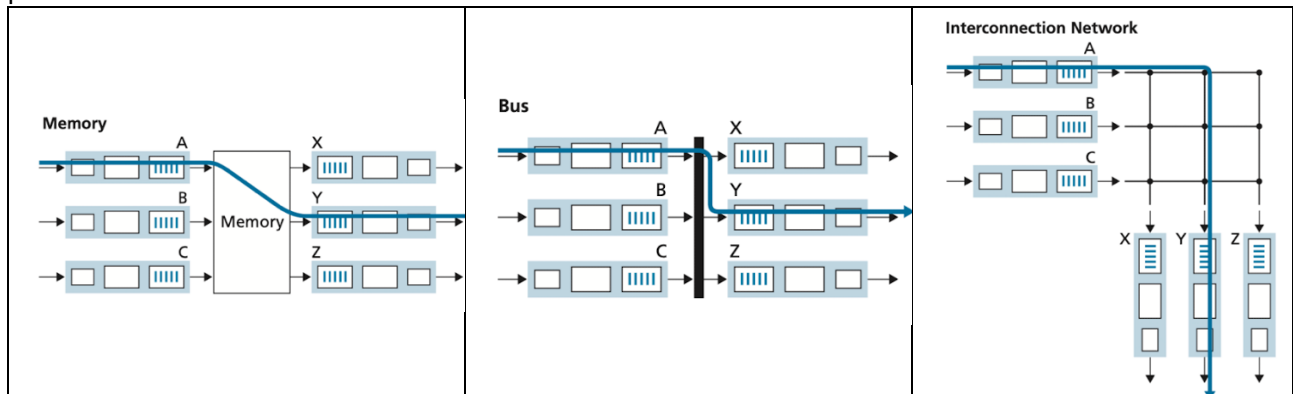
Se prendiamo i due pacchetti:

11001000 00010111 000**11110 10100001**
 11001000 00010111 000**11000 10101010**

Il secondo pacchetto può essere mandato sia nel link 1 che nel link 2, quale si sceglie?

Non solo ci sono diversi modelli di instradamento all'interno della switching fabric, ma ci sono anche diverse tipologie di switching fabric, ognuna con dei pro e dei contro. In generale, l'obiettivo che si vuole raggiungere è un'ampia velocità di switching rate (tempo impiegato da porta di input a porta in output) garantendo il massimo parallelismo possibile (in due dei tre modelli non è raggiungibile per questioni architetturali).

Ad esempio, la memory fabric costa poco ed è semplice da realizzare, tuttavia è lenta e nei picchi di burst può portare ad importanti colli di bottiglia. La Bus Fabric, d'altro canto è molto più veloce, è efficiente e non richiede il "copia e incolla della memory", tuttavia si ha concorrenza, il bus può essere occupato da più di un pacchetto alla volta. L'interconnection fabric d'altro canto è veloce, rapida e dispone di parallelismo, tuttavia si richiede un hardware dedicato, che comporta costi elevati di produzione.



È errato pensare che avere una switching fabric estremamente veloce sia una cosa sempre positiva, questo perché è necessario che la coda di output sia altrettanto potente e veloce da non perdere pacchetti. Infatti, se la switching fabric processa pacchetti molto più velocemente di quello che la porta di output è capace, i buffer di output si riempiranno in fretta, comportano una perdita di pacchetti dopo k pacchetti in coda. Dunque, in un **router è necessario**:

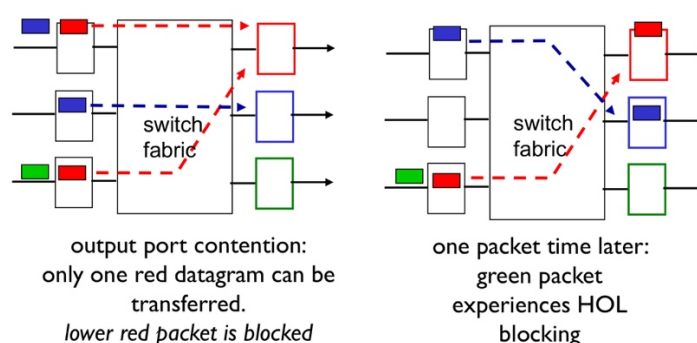
- Avere meno Head-of-Line Blocking (**HOL**) possibile, dovuta a una switching fabric lenta, che non riesce in un tempo ragionevole a spostare i pacchetti dalle porte di input a quelle di output. Una contesa della porta può portare a HOL.
- **Buffering ridotto**, ovvero la porta di uscita riesce a stare al passo con la switching fabric, riuscendo a non andare mai in over-buffering, che comporta la perdita dei pacchetti. Con un buffer ampio la porta di uscita può essere leggermente più lenta della switching fabric, in questo modo, si possono parcheggiare per qualche istante i pacchetti prima di spedirli.

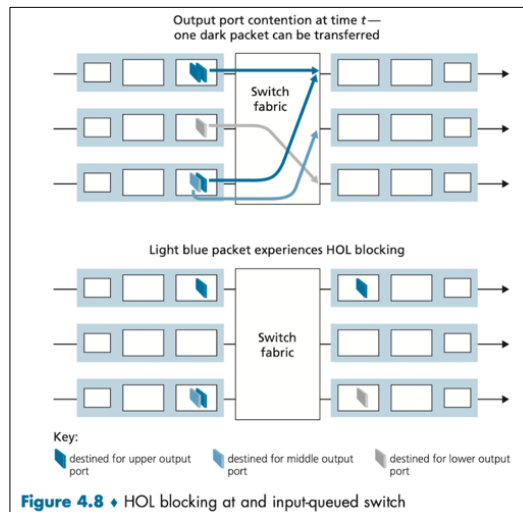
Il buffering necessario è dato da questa formula:

$$\frac{RTT * C}{\sqrt{N}}$$

Dove C = capacità; N = numero di flussi

- Un buon algoritmo di **scheduling** che riesce a gestire nel modo più rapido ed efficiente la coda nella porta di uscita.





Ci sono moltissimi algoritmi di scheduling, proprio come nei sistemi operativi; infatti, gli algoritmi di scheduler sono molto simili, infatti per analogia, i processi per un SO sono i pacchetti per il router. Tra i processi di scheduling troviamo i seguenti algoritmi:

- **FIFO Scheduling**, First In First Out con discard policy da scegliere in caso di coda piena
 - Tail Drop: l'ultimo pacchetto arrivato vien droppato
 - Priority: drop/rimozione di un pacchetto su base prioritaria
 - Random: drop/rimozione in modo randomico
- **Priority Scheduling**, dove vengono processati per prima i pacchetti che hanno priorità alta

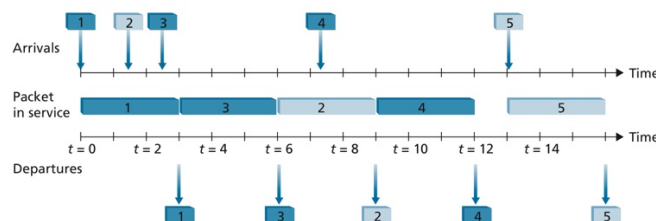


Figure 4.14 ♦ The priority queue in operation

- **Round Robin** dove i pacchetti vengono ordinati per classe e per priorità. Lo scheduler estrarrà quindi dalla coda di uscita il primo pacchetto della prima classe, poi il primo pacchetto della seconda classe, e poi tornerà ad occuparsi della prima classe, della seconda e così via. Questo meccanismo, tuttavia, può avere classi che rubano larghezza di banda ad altre classi. Ad esempio, se la prima classe ha un pacchetto che richiede molto tempo ad essere inviato, i pacchetti della seconda classe aspetteranno molto; dunque, le classi non sono fair
- **WFQ** (Weighted Fair Queuing) ha lo stesso principio dello scheduler Round Robin; tuttavia, ad ogni classe assegna una priorità, in questo modo a classi con priorità più alta verrà assegnata una larghezza di banda maggiore

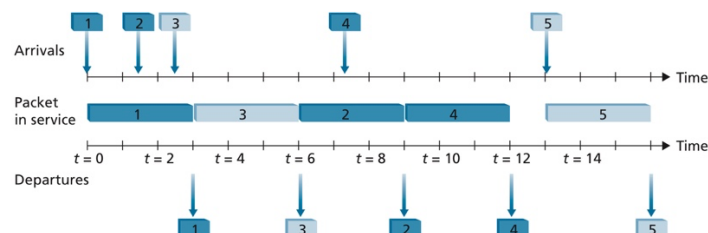


Figure 4.15 ♦ The two-class robin queue in operation

IPv4

Entriamo ora in un argomento già trattato, ma iniziamo a fare qualche precisazione ed approfondimento. Consideriamo la struttura di un pacchetto IPv4, che è composto come si vede nella tabella qui sotto (*tutti i dettagli di ogni sezione sono visibili dal libro*).

32 bits				
Version	Header length	Type of service	Datagram length (bytes)	
16-bit Identifier			Flags	13-bit Fragmentation offset
Time-to-live		Upper-layer protocol	Header checksum	
32-bit Source IP address				
32-bit Destination IP address				
Options (if any)				
Data				

Queste informazioni, dette in modo più tecnico IPv4 datagram sono la struttura del pacchetto IPv4. Queste informazioni possono pesare molto, quindi solitamente la rete non riesce a trasmettere un pacchetto di dimensioni esorbitanti. Dunque, le informazioni vengono frammentate in pacchetti, dunque non si spedisce in rete una cosa grande, ma si spedisce la stessa cosa frammentata. Questa frammentazione viene effettuata dagli host, gli stessi che hanno il compito di ricomporre nell'unico grande pacchetto la frammentazione eseguita dall'host mittente.

Pariamo ora di interfacce di rete, che sono visibili come il confine che divide host e livello fisico. E per precisione, si può dire che l'interfaccia di rete è in qualche modo rappresentata da un indirizzo IP, il quale ha il compito di riunire il confine diviso tra host e rete. Un host normale ha solitamente due interfacce di rete (una wireless e una LAN, oggi potrebbero essere anche uguali). Un router, invece, ha più interfacce, ognuna delle quale serve una sottorete che è collegata in modo diretto alla porta del router.

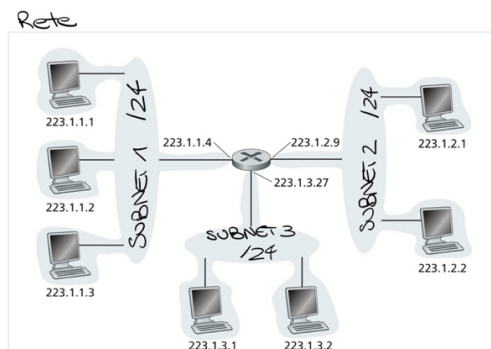
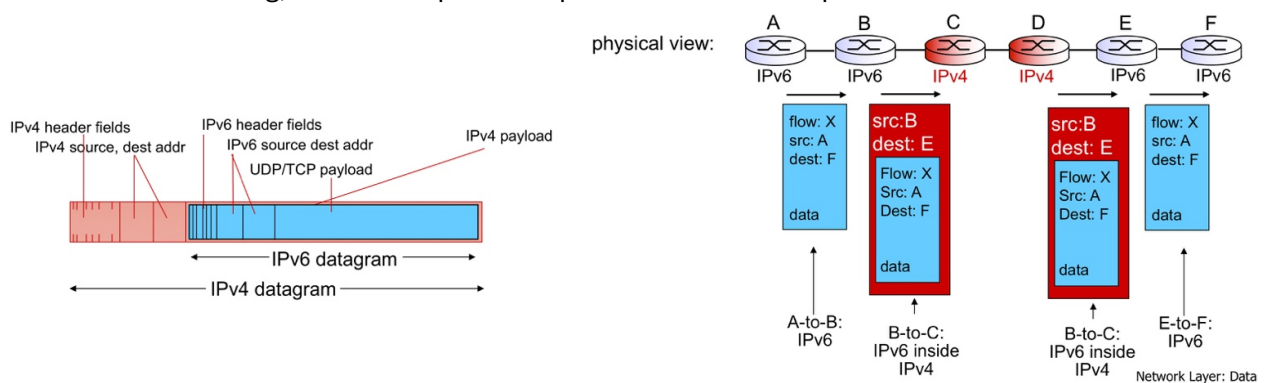


Figure 4.18 • Interface addresses and subnets

IPv6

IPv6 non è una transizione immediata, ma è una transizione lenta; dunque, far girare n rete un pacchetto IPv6 è possibile fino a che si incontrano router IPv6, ma l'ultimo router prima di un router IPv4 deve fare tunnelling, ovvero incapsulare il pacchetto IPv6 in un pacchetto IPv4.



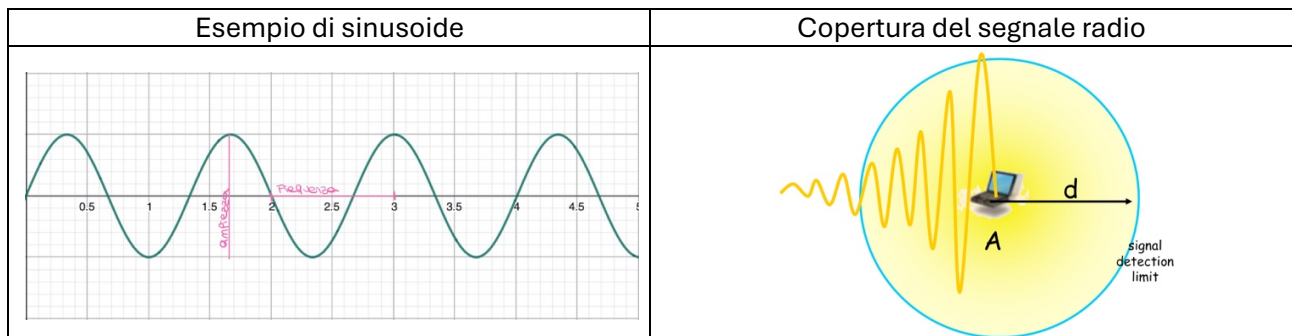
RETI WIRELESS

Le reti wireless nelle comunicazioni

Terminologia base

- **Ampiezza:** differenza tra picco positivo e picco negativo della sinusoide
- **Frequenza:** numero dell'alternarsi di picchi negativi e positivi in un certo intervallo di tempo
- **Fase:** shift dell'onda verso destra o verso sinistra
 - o **Fase positiva:** Shift a sinistra – early wavefront
 - o **Fase negativa:** Shift a destra – late wavefront
- **Lunghezza d'onda:** c/GHz dove $c = \text{velocità luce in metri}$

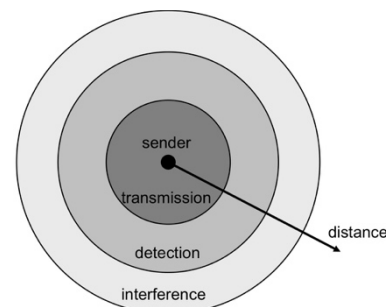
Le antenne, ovvero quegli strumenti in grado di convertire la corrente in radiofrequenze, e viceversa funzionano al massimo delle performance con misura di lunghezze d'onda pari a 1, ½ e ¼



La copertura del segnale radio è una funzione, verosimilmente è riconducibile ad un esponenziale.

La copertura di propagazione è distinguibile in tre possibili casi:

- **Transmission Range:** La comunicazione tra mittente è destinatario è possibile e c'è generalmente poco errore
- **Detection Range:** la rilevazione di segnale è possibile, ma non è possibile la comunicazione. In sostanza si rilevano segnali ma c'è possibilità di disturbo altrui; quindi, la comunicazione è compromessa
- **Range di Interferenza:** I segnali possono non venire rilevati e l'insieme dei segnali forma un rumore di sottofondo, come un ronzio



Nota! Le coperture dipendono dalla sensibilità del ricevente

Generalmente, le frequenze alte sono buone per le corte distanze, e sono anche condizionate dagli ostacoli, mentre frequenze basse sono migliori per lunghe distanze e meno condizionate dagli ostacoli. Considerando che gli ostacoli stessi possono assorbire o riflettere le onde, in base al materiale di cui sono composti.

In generale:

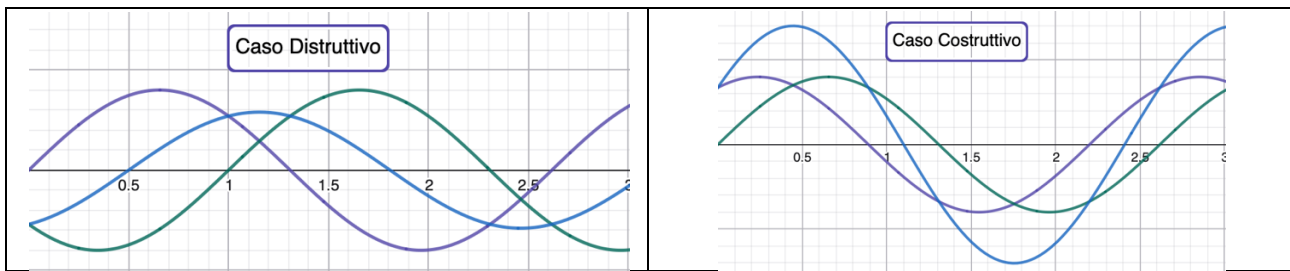
Caratteristica	Bassa Frequenza (es. 2.4 GHz)	Alta Frequenza (es. 5-6 GHz)
Velocità Dati	Minore	Maggiore
Copertura/Raggio	Maggiore (attraversa i muri)	Minore (raggio limitato)
Interferenze	Molte (Bluetooth, microonde)	Poche (molto più spazio)

Come la fase entra in gioco per la manipolazione del segnale

Equazione **viola** e **verde** è la radiofrequenza (stessa ampiezza e frequenza, ma fasi diverse).

Equazione **blu** è la somma delle due radiofrequenze.

Guardare la differenza nella tabella sottostante

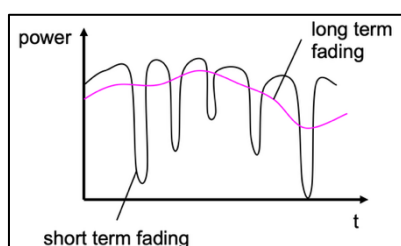
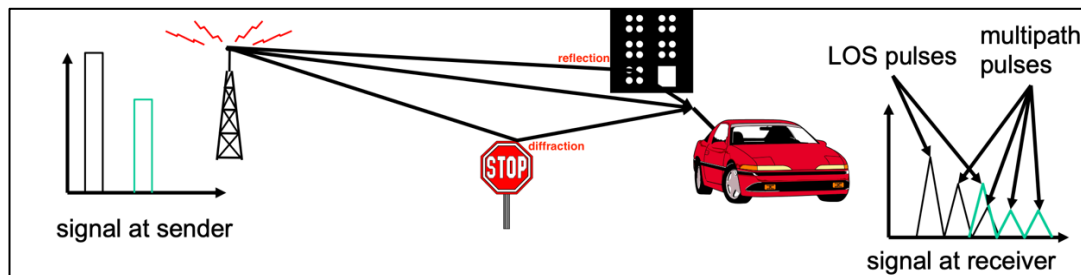


La **Polarizzazione** invece riguarda il posizionamento dell'antenna. Il campo magnetico è perpendicolare all'antenna, mentre il campo elettrico è parallelo all'antenna. Questo orientamento permette di intercettare radio frequenze diverse, oppure, di riceverle meglio.

Multi Path Propagation

I segnali quando emessi da un'antenna possono intraprendere diversi percorsi (dovuto a *shadowing*, *reflection*, *refraction*, *scattering*, *diffraction*). Quando un segnale parte di sparge quindi alla velocità della luce in tutte le direzioni e tale segnale arriva al mittente con intensità diversa a seconda della strada intrapresa. Si parla di **Intra Symbol Interference** quando lo stesso segnale partito allo stesso istante 't' dal mittente giunge all'istante 'k' e all'istante 'k+1' al destinatario creando una somma dello stesso segnale sottrattiva, peggiorando quindi la ricezione. Si parla di **Inter Symbol Interference** quando si ha interferenza tra simboli consecutivi di "colore" diverso. Quest'ultimo problema è arginato diminuendo il bit-rate.

L'immagine sotto riportata da idea dell'intensità con cui arriva lo stesso segnale che segue percorsi diversi allo stesso destinatario.



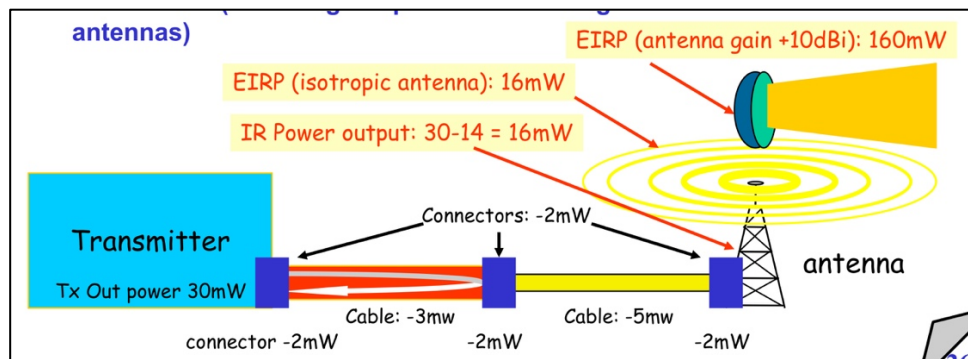
Si parla di "*fading*" quando due segnali sfasati entrano in sottrazione. Le fasi sottrattive si possono definire come "*short term fading*" quando sono molto forti, ma durano poco (segnale scende a picco e risale anche a picco) o "*long term fading*" quando il segnale tende a scendere verso il basso non nell'immediato, ma nel tempo.

Voltage Standing Wave Ratio (VSWR)

La **VSWR** accade quando ci sono in dialogazione diversi dispositivi che hanno diversa **impedenza** (misurata in *Ohm*), ovvero che hanno diversa resistenza alla corrente AC che scorre tra il trasmettitore e l'antenna. Di base si vuole un VSWR ratio pari a 1:1, ma non è sempre possibile, quindi, a volte si ha della dispersione di energia (solitamente calore) che genera rotture di schede o circuiti, prevalentemente surriscaldamento e livelli di corrente in output non stabili. L'impedenza è quindi un ostacolo, ovvero esprime la fatica di quell'oggetto a far passare la corrente.

Il VSWR è un rapporto misurato tra l'impedenza dell'oggetto alla posizione $n - 1:n$.

Nelle reti, lo standard di impedenza è 50 Ohm, questo permette la più bassa perdita di segnale (gli standard servono ad arginare il problema di VSWR molto alti ed inefficienti).



IR Power Output: Dato un trasmettitore che trasmette 30 milliWatt, e data una serie di connettori e cavi, magari con diversa impedenza e con un VSWR non rapportato 1:1 misura quanta potenza reale arriva all'antenna. Nell'esempio, vediamo che il trasmettitore ha una potenza di 30, e l'antenna riceve 16, dunque la potenza che arriva all'antenna è di 16, e si sono perse 14 unità nel tragitto fisico trasmettitore-antenna. Questo valore è quindi **influenzato** da quello che c'è fisicamente tra il trasmettitore e l'antenna.

IR Power Out è quindi la potenza in output dell'ultimo connettore appena prima dell'antenna.

EIRP invece è la potenza radiata dall'antenna (incluso il vantaggio passivo guadagnato dalle antenne direzionali). In sostanza sappiamo che l'antenna potrebbe emettere in qualsiasi direzione il segnale, ma questo non sempre è utile (se sono a Milano e voglio comunicare con uno a Parigi non propago l'onda radio verso Palermo), infatti in base alla posizione del destinatario si gira il fascio dell'antenna direzionale (guadagnando potenza grazie alla concentrazione del fascio delle onde radio). Quando si costruiscono quindi dei sistemi bisogna tenere in considerazione diversi fattori (puramente da un punto di vista ingegneristico) tra cui:

- Potenza del mezzo trasmettitore
- Loss and Gain dei mezzi di collegamento (cavi, connettori, amplificatori...) → Invio
- IR Power Output
- EIRP
- Proprietà di propagazione dei mezzi
- Loss and Gain dei mezzi di collegamento → Ricezione

Power Measurement

Watt: unità di misura elettrica

$$1 \text{ Watt} = 1 \text{ Ampere} * 1 \text{ Volt} \rightarrow P = (V * I)$$

Corrente *Ampere* è la quantità di carica (elettroni) che si spostano come corrente in un cavo

Voltaggio *Volt* è la "pressione" applicata al flusso della carica

Resistenza, ovvero impedenza è l'ostacolo all'attuale flusso della carica

Unità di misura fortemente utilizzate **Watt** e **dBm** per la misurazione della potenza in termini assoluti

Decibel (dB): unità di misura utilizzata per esprimere la **perdita** di potenza.

Un *dB* è pari a $\frac{1}{10}$ BEL

Indice	milliWatt	Logaritmo	BEL	Decibel
A)	1	$10^0 = \log_{10} 1 = 0$	0	0
B)	10	$10^1 = \log_{10} 10 = 1$	1	$10 = \Delta B - A$
C)	100	$10^2 = \log_{10} 100 = 2$	2	$20 = \Delta C - A$
D)	1000	...	3	$30 = \Delta D - A$

Tuttavia, il *Decibel (dB)* è un'unità che indica il guadagno o la perdita di potenza, dunque all'affermazione "Quanto è forte un segnale da 10dB" la risposta è: dipende dal segnale di riferimento.

Il decibel, quindi, è utile per indicare la perdita o l'incremento di potenza del segnale.
Dove con decibel positivo si indica **l'incremento** e con decibel **negativo** si indica la perdita.

$$Power\ Difference\ (dB) = 10 * \log \left(\frac{Power\ Rx\ (Watt)}{Power\ Tx\ (Watt)} \right)$$

$$Power\ Rx\ (Watt) = Power\ Tx\ (Watt) * 10^{\frac{Power\ Difference\ (dB)}{10}}$$

$$Power\ Tx\ (Watt) = Power\ Rx\ (Watt) * 10^{-\frac{Power\ Difference\ (dB)}{10}}$$

Dove Rx è il segnale ricevuto e Dx è il segnale trasmesso

Visto che condividere il watt di invio e ricezione è scomodo, viene usato il decibel come misura, che utilizzato nel modo opportuno può aiutare a capire con quanta intensità arriva il segnale.
La tabella sottostante è utile per effettuare dei calcoli:

- 3dB	$1/2$ potenza in mW
+ 3dB	$2 * $ potenza in mW
-10 dB	$1/10$ potenza in mW
+10 dB	$10 * $ potenza in mW

Esempio

$$100\ mW + 7dB = 100mW + 10dB - 3dB = 100mW * \frac{10}{2} = 500mW$$

Visto che il decibel è una misura per esprimere l'incremento o la perdita della potenza del segnale è possibile introdurre una nuova unità di misura il: dBm , ovvero il **decibel-milliWatt**.

Il decibel-milliWatt è la misura assoluta della potenza del segnale.

Viene usato per rappresentare guadagni e perdite in cascata lungo una catena

Dunque, si ha l'assunzione che $1mW = 0\ dBm$

$$P_{dBm} = 10 * \log (P_{mW})$$

$$P_{mW} = 10^{\frac{P_{dBm}}{10}}$$

Questa unità di misura è molto utile, infatti quando si usano dBm e dB si possono direttamente sommare e sottrarre le perdite senza passare per i milliWatt.

Consideriamo il seguente **esempio**:

$$Tx = 30mW ; \text{cavo} - 2\ dB; \text{amplificatore} + 9\ dB$$

$$P_{dBm} = 10 * \log(30) = 14,77\ dBm$$

$$\text{Sommando guadagni e perdite si ha: } 14,77\ dBm - 2dB + 9dB = 21,77\ dBm$$

$$\text{che equivale a } P_{mW} = 10^{21,77\ dBm / 10} = 150,31\ dBm$$

usare quindi i milliWatt è più comodo in quanto si possono sommare le stesse quantità al posto di moltiplicarle. Se avessimo seguito lo schema precedente della moltiplicazione e divisione tra mW e dB ci avremmo messo più tempo.

Ricorda! Sommare potenze in dBm è come moltiplicare le potenze in mW; dunque, non si può fare la somma di valori in dBm e convertire il risultato in mW, ma per una **somma esatta** tra dBm per portarla a milliWatt è necessario: convertire tutto a mW e poi sommare (se si usa la tabella sottostante)

dBm	Valore esatto
-12	63,1 μ W
-11	79,4 μ W
-10	100 μ W
-9	125,9 μ W
-8	158,5 μ W
-7	199,5 μ W
-6	251,2 μ W
-5	316,2 μ W
-4	398,1 μ W
-3	501,2 μ W
-2	631,0 μ W
-1	794,3 μ W
0	1 mW

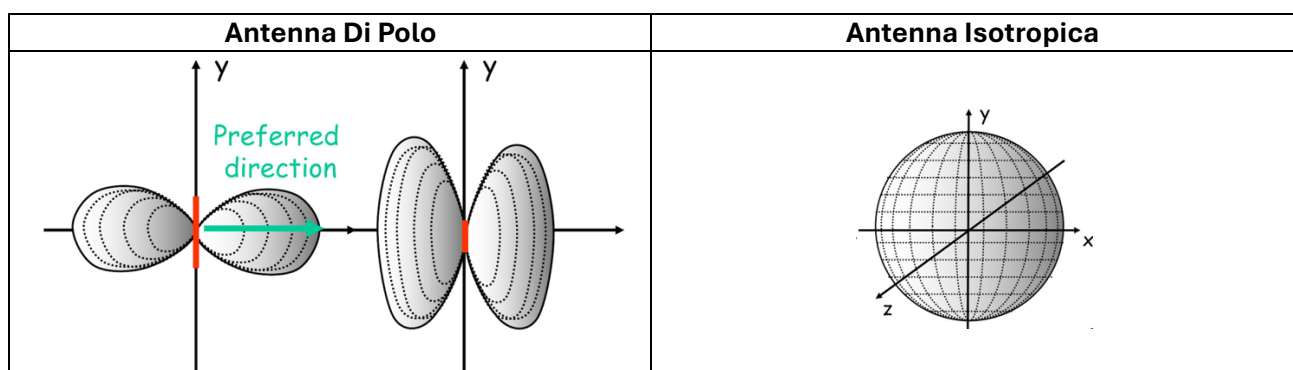
0	1 mW
+1	1,259 mW
+2	1,585 mW
+3	1,995 mW
+4	2,512 mW
+5	3,162 mW
+6	3,981 mW
+7	5,012 mW
+8	6,310 mW
+9	7,943 mW
+10	10 mW
+11	12,59 mW
+12	15,85 mW
+20	100 mW
+30	1 W
+40	10 W

dbM	mw
(+) (-)	(*) (/)
10	10
3	2
1	1,25
2	1,56

Il decibel-isotropico è la misura normalizzata del guadagno passivo di un antenna.

I dBi misurano quindi l'effetto concentrazione rispetto ad un'antenna isotropica.

L'antenna **isotropica** è l'antenna (impossibile da costruire) che ha massima efficienza nel radiare il 100% dell'energia in modo uniforme in ogni direzione. Un'antenna isotropica è visibile letteralmente come una sfera. Un'antenna non isotropica, più diffusa è l'antenna **di polo** ha quindi un guadagno passivo in quanto concentra la sua potenza in un punto specifico, dunque la sua potenza nel punto X sarà più alta della potenza dell'antenna isotropica nello stesso punto X.



Come si vede dall'immagine l'antenna di polo ha una direzione preferita. Supponiamo che in quella direzione, quell'antenna di polo abbia un guadagno pari a 7dBi e che all'antenna arriva energia pari a 2 mW , possiamo calcolare l'**EIRP**, ovvero la vera potenza dell'antenna.

Il calcolo è molto semplice: $EIRP = \text{Potenza Tx (dBm)} + \text{guadagno (dBi)}$.

Dunque, come si vede dall'immagine le antenne hanno sempre una direzione preferita dove la potenza emessa è maggiore dell'antenna isotropica, questo perché il guadagno è sempre positivo nella direzione preferita.

Le Antenne

Frequenza (f) : quante volte la sinusoide completa un ciclo di oscillazione in un secondo. Il concetto è l'unità temporale. L'unità di misura sono gli Hertz (Hz)

Lunghezza d'onda (λ) : rappresenta la distanza fisica che l'onda percorre per completare un singolo ciclo completo. Il concetto è lo spazio, e l'unità di misura sono i metri.

Visto che $c = 300.000.000 \text{ m/s}$ (velocità luce) si ha che:

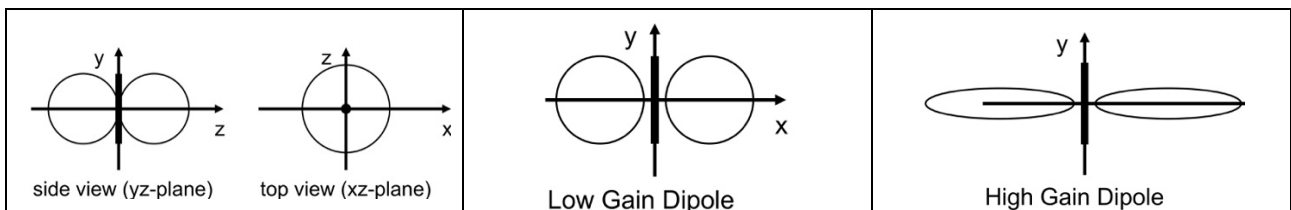
$$c = f (\text{tempo}) * \lambda (\text{spazio})$$

$$\lambda = \frac{c}{f}$$

$$f = \frac{c}{\lambda}$$

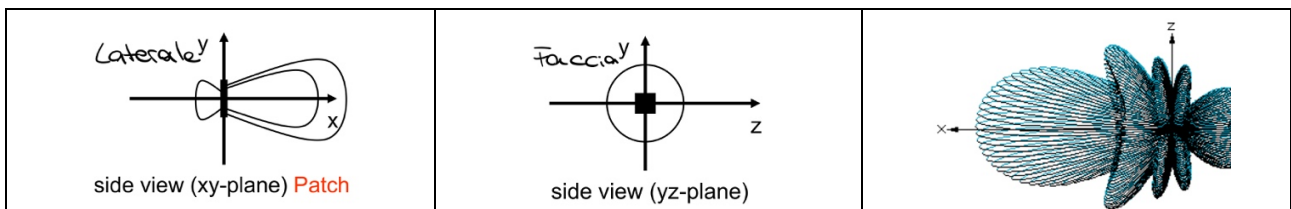
L'idea dietro alle antenne è quindi quella di avere una dimensione dell'antenna pari alla lunghezza d'onda, in questo modo si riesce contemporaneamente a trovare il picco positivo e il picco negativo, acquisendo la differenza di potenziale maggiore e il maggior bitrate. Solitamente le antenne sono della dimensione di: $\frac{\lambda}{2}$ o $\frac{\lambda}{4}$. Le antenne possono essere omni-direzionali (di polo), semi-direzionali o altamente-direzionali (le parabole).

Antenne omni-direzionali: irradiano di radiofrequenze l'ambiente circostante in ugual modo attorno l'asse verticale.

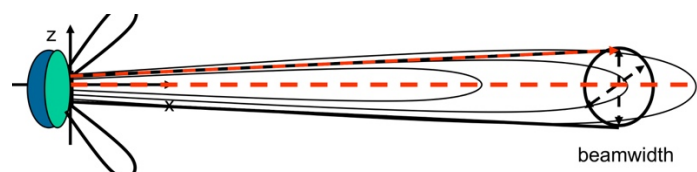


La visualizzazione delle radiofrequenze emesse nell'immagine vale per il caso outdoor con pochi ostacoli, ma nel caso indoor questo non vale, in quanto ci sono pareti, muri di cemento armato, scrivanie, oggetti e tavoli; quindi, il segnale rimbalza ed è molto disturbato.

Antenne semi-direzionali: irradiano di radiofrequenze l'ambiente circostante principalmente in una direzione specifica, avendo però dei lobi più lievi in altre direzioni

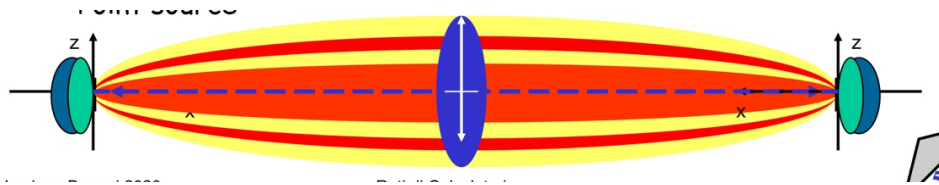


Antenne altamente-direzionali: irradiano di radiofrequenze l'ambiente circostante solamente in una direzione specifica. Sono molto suscettibili alle oscillazioni e all'inclinazione con cui sono messe tra loro in comunicazione



Le antenne altamente-direzionali come già detto sono altamente suscettibili alle oscillazioni e all'inclinazione con cui sono messe in comunicazione tra loro. Avendo una larghezza del fascio compresa tra i 4 e 25 gradi è essenziale che queste tipo di antenne siano posizionate nel modo più corretto possibile, con una linea di visuale tra le antenne il più pulita possibile. La linea di visuale tra le antenne deve essere pulita, poche ostruzioni, se non nulle. Ciò permette una buona ricezione. Come si può vedere, infatti, molte parabole sono posizionate proprio in punti alti proprio per avere una posizione privilegiata e prima di ostruzioni.

Perché tutti questi accorgimenti? Perché le antenne trasmettono radiofrequenze (trasmettono energia) nello spazio e lo spazio tende a "dissipare/diffondere" il modo nel quale l'energia viene trasmessa. Dunque, le antenne altamente direzionali devono essere posizionate strategicamente in quanto devono aver un raggio, detto raggio di Fresnel o **Fresnel Zone** il più libero possibile, questo perché tale zona è la zona in cui i segnali sono additivi, e dunque, consentono la maggior qualità ricezione/trasmmissione possibile.



In blu viene raffigurata la **Fresnel Zone**, tale circonferenza non deve essere ostruita, in quanto anche un minimo ostacolo causa le radiofrequenze a rimbalzare e dunque a distruggere le fasi additive che andranno a seguire percorsi diversi e dunque rendendo la trasmissione/ricezione peggiore, se non pessima o impossibile.

Il raggio della Fresnel Zone deve essere almeno pari al 60% per una comunicazione accettabile, ma è meglio se è pari al 100%.

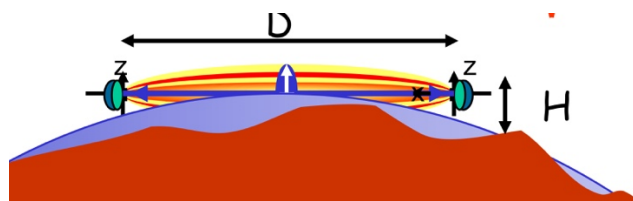
$$R_{60\%}(Ft) = 43.3 * \sqrt{(d/4f)}$$

$$R_{100\%}(Ft) = 72.2 * \sqrt{(d/4f)}$$

Dove: d = distanza in miglia; f = frequenza segnale in GHz; Ft = distanza in piedi

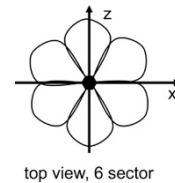
Nota! La Fresnel Zone dipende **solo** dalla distanza tra le antenne e dalla frequenza del segnale, **non** dipende dalla potenza intrinseca del segnale. Questo è da tenere ben a mente. Lo si evince dalle formula sopra riportate.

Nota (2)! La Fresnel Zone riguarda solo situazioni outdoor (indoor ci sono troppi rimbalzi inevitabili) e tiene ovviamente in considerazione della curvatura terrestre. Se la Fresnel Zone si trova sottoterra, questo è un problema, la terra non diventa invisibile.



Antenne direzionali settorizzate

Queste antenne hanno la particolarità di essere disposte come un fiore, dove c'è un array della stessa tipologia di antenna che trasmettono alla stessa frequenza. In questo modo, viene utilizzata sempre la stessa frequenza per diffondere segnali su canali diversi.



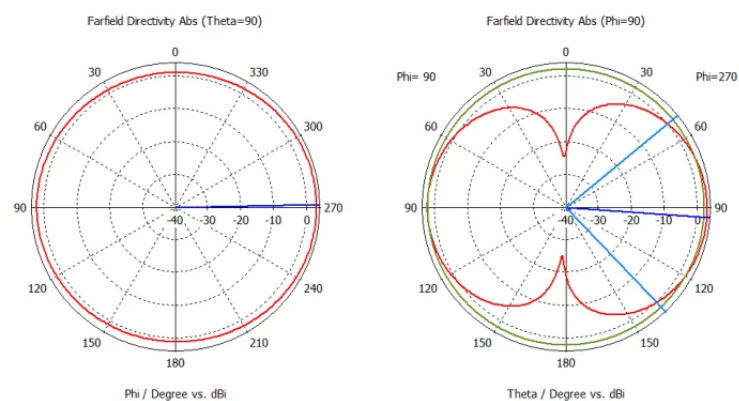
Azimut e Elevation antenna Charts

Sono grafici utilizzati per comprendere il pattern della copertura dell'antenna.

Il grafico **azimut** è ottenuto vedendo l'antenna da davanti/destra/dietro/sinistra/davanti; dunque, è ottenuto ruotando a 360 gradi sull'asse orizzontale attorno all'antenna.

Il grafico **elevation** è ottenuto vedendo l'antenna da davanti/sopra/dietro/sotto/davanti l'antenna; dunque, è ottenuto ruotando a 360 gradi sull'asse verticale attorno all'antenna.

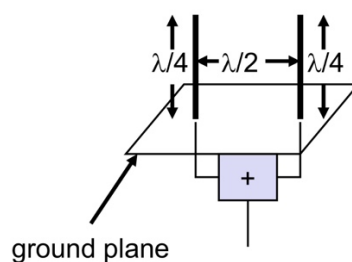
Questi grafici sono molto comodi in quanto dato un dato "standard" di potenza trasmissiva permettono di capire di quanti **dBi** si perdono o si guadagnano ruotando attorno all'antenna.



A sinistra azimuth chart, a destra elevation chart di una antenna di polo

Diversità di antenne

Abbiamo detto che un'antenna è solitamente della dimensione di $\lambda/4$ o $\lambda/2$, questo come abbiamo detto deriva dal fatto che le antenne devono avere più o meno una frazione accettabile della lunghezza d'onda, questo infatti punta a beccare più fase additiva della radiofrequenza. Ma visto che spesso ci possono essere dei rimbalzi, dei ritardi e degli effetti che non permettono correttamente di leggere il segnale dalla radiofrequenza, la cosa migliore è quella di disporre più antenne sfalsate nello spazio di un certo valore, come nell'immagine.



Questa diversificazione favorisce la diversità, riducendo drasticamente il fenomeno del fading; infatti, se la prima antenna ascolta una fase sottrattiva, la seconda antenna, che è sfalsata, matematicamente (per l'andamento della sinusoide) ascolterà una fase additiva, dunque capterà meglio il segnale, favorendo una comunicazione migliore. Ciò può essere fatto con più antenne, più antenne, maggior precisione. Questo però non viene gratis, c'è bisogno di un processore o co-processore che ascoltano ed effettuano calcoli tali da capire il segnale migliore e sfruttarlo. Tecnicamente con antenne sfalsate si riesce ad ottenere la differenza di potenziale maggiore.

Path Loss

La dispersione (attenuazione) dovuta alla distanza è un problema serio per le radio frequenze. Qui di seguito ci sono due formule per la perdita in decibel del segnale, una con costante di attenuazione minore per giornate migliori, e una con costante di attenuazione maggiore per giornate peggiori.

$$Loss (dB) = 36.6 + (20 * \log_{10}(F) + (20 * \log_{10}(D)) \rightarrow \text{Formula ottimista}$$

$$Loss (dB) = 32.4 + (20 * \log_{10}(F) + (20 * \log_{10}(D)) \rightarrow \text{Formula pessimista}$$

Dove F = Frequenza in MHz e D = distanza in miglia mittente – destinatario

Tenendo sempre a mente, la seguente tabella, che dice che al raddoppiare della distanza, la perdita è pari a -6dB per ogni “passo” compiuto

100 meters	- 80.23 dB	-6 dB
200 meters	- 86.25 dB	
500 meters	- 94.21 dB	-6 dB
1000 meters	- 100.23 dB	
2000 meters	- 106.25 dB	-6 dB
5000 meters	- 114.21 dB	
10000 meters	- 120.23 dB	-6 dB

Link Budget

Il link budget è il segnale in più rispetto al minimo necessario, in altre parole, è l'eccesso di segnale utilizzato tra il mittente e il destinatario. Il link budget può essere misurato sia in dBm che in dB. Inoltre, il LB deve essere compreso, o maggiore del fade margin (FM). Visto che le onde radio trasmesse subiscono rimbalzi ed effetti non voluti in base alle condizioni atmosferiche è necessario che il $LB \geq FM$ in questo modo, anche nelle giornate peggiori, con eventi atmosferici avversi o non ideali la comunicazione riesce sempre nel migliore dei modi. Il peggio di costruire un'infrastruttura per la comunicazione radio è avere il Link Budget uguale alla receive sensitivity (RS) del destinatario.

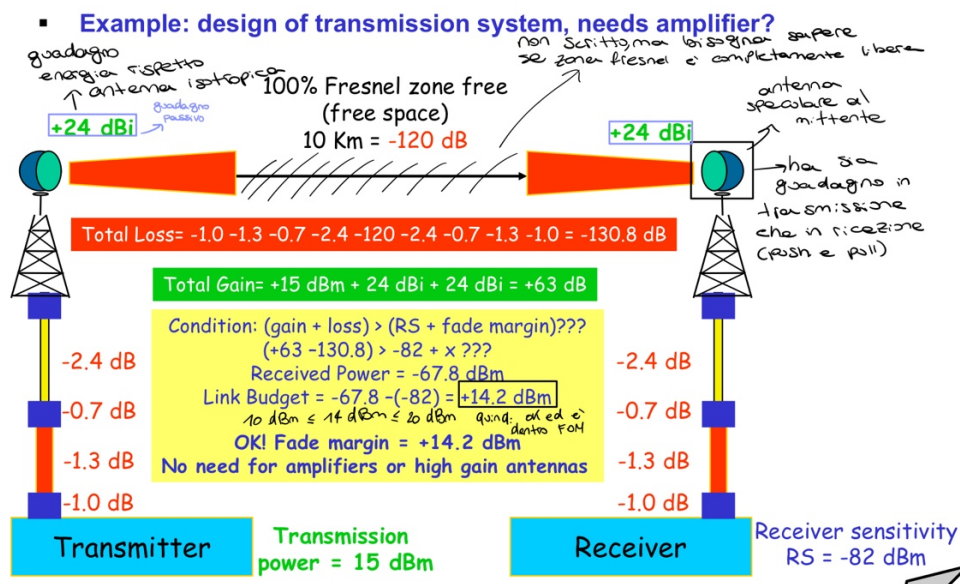
$$\text{Link Budget (dBm)} = \text{Received Power (dBm)} - \text{RS (dBm)}$$

$$\text{Received Power (dBm)} = \text{Total Gain (dBm)} + \text{Total Loss (dBm)}$$

$$\text{Fade Margin: } 10 \text{ dBm} \leq LB \leq 20 \text{ dBm}$$

Esempio: $RS = -82 \text{ dBm}$; $\text{Received Power} = -50 \text{ dBm}$

$\text{Link Budget} = -50 - (-82) = +32 \text{ dBm}$. Questo significa che il segnale ha un margine di 32 dBm prima di diventare irricevibile.



Larghezza di Banda e Spettro

Lo **spettro** (spettro elettromagnetico) è l'insieme completo di tutte le frequenze (GhZ, MhZ...) possibili nell'universo.

La **larghezza di banda** è un "pezzo" o un intervallo di frequenze all'interno dello spettro che viene utilizzato per una specifica trasmissione. Matematicamente, è la differenza tra la frequenza più alta e quella più bassa utilizzate.

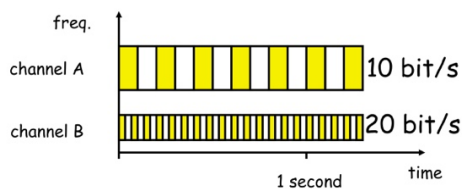
Esempio: canale trasmette da 90 MhZ a 110 MhZ; dunque, la sua larghezza di banda è pari a 20 MhZ. Più la larghezza di banda è grande, più dati si possono trasmettere simultaneamente in un dato momento.

Di base, i bit viaggiano sempre alla velocità della luce, non esiste una frequenza che li fa viaggiare più forti o più lenti, ma è possibile unire più frequenze (quindi usare un intervallo dello spettro più ampio, dunque avere più larghezza di banda) in modo tale che nel canale trasmissivo si possa impiegare meno tempo per trasmettere un singolo bit.

Il **simbolo** è quella manopola, ovvero quella caratteristica dell'onda radio trasmessa ad una determinata frequenza che può aumentare o diminuire il bit-rate. In sostanza, se impostiamo il simbolo ad essere "lungo" in un secondo trasmetteremo pochi bit, ma questi bit saranno molto chiari e ben interpretabili dal destinatario. Se invece "accorciamo" il simbolo, in un secondo verranno trasmesse più informazioni, più bit, ma il tempo di comprensione per il destinatario sarà ovviamente più piccolo. Dunque, è necessario variare i simboli in base alle condizioni meteorologiche.

Cosa non bisogna confondere:

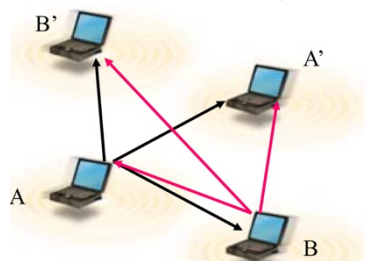
- Larghezza di Banda (altezza canale): Intervallo dello spettro di frequenze utilizzato, più è grande più spazio si ha per trasmettere dati contemporaneamente senza accavallamenti
- Simbolo (grandezza quadrato): quanto tempo far durare l'alterazione della sinusoide per la trasmissione del dato.



Transmission Coverage (Copertura Trasmissiva)

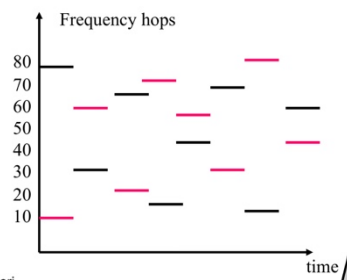
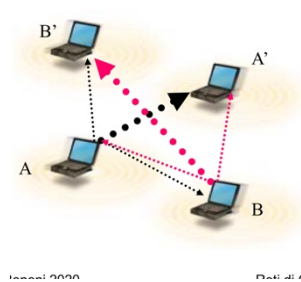
- Unidirezionale: tra due interlocutori, solo una persona riesce a ricevere segnale e solo una persona riesce a trasmettere il segnale (dovuto a potenza del segnale emesso e receiver sensitivity dei due interlocutori)
- Bidirezionale: entrambi interlocutori riescono a trasmettere e ricevere
- Asimmetrico: bidirezionale, ma in un verso si ha meno velocità trasmissiva e in un verso si ha più velocità trasmissiva (Esempio $\rightarrow 10$ Mbps e $\leftarrow 1$ Mbps)
- Simmetrico: bidirezionale e stessa velocità (Esempio $\rightarrow 10$ Mbps e $\leftarrow 10$ Mbps)

Narrowband Radio System



Trasmettitore e ricevitore usano una singola frequenza con licenza. Il cross-talking tra frequenze non esiste in quanto ogni canale è sotto-licenza e correttamente coordinato. Ha un grandi svantaggi in ambito di sicurezza, infatti nessuno può impedire a terzi di ascoltare tale frequenza, inoltre, un malintenzionato potrebbe volutamente disturbare il canale trasmissivo di una frequenza, inoltre se un segnale è disturbato per effetti terzi, solo chi usa quella frequenza è disturbato.

Frequency Hopping



Mittente e destinatario saltano di frequenza in frequenza e ci permangono per un tempo limitato. Questa è la tecnologia di condivisione dello spettro di frequenza ed è la tecnologia utilizzata dal bluetooth. Qui l'ascolto di un terzo nella comunicazione è impossibile, perché l'algoritmo di salti è deciso bilateralmente tra mittente e

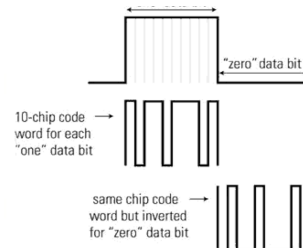
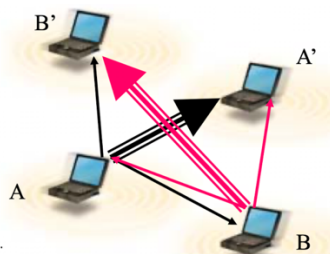
destinatario di quella precisa comunicazione e mantenuto valido da un algoritmo. L'unica cosa "pericolosa" è quando i clock del mittente e del destinatario non sono ben allineati e non mantengono correttamente l'allineamento, in quel caso, la comunicazione diventa impossibile. Abbiamo detto che l'ascolto di terzi è difficile, ma anche la fairness è molto più equa, infatti se esiste un canale disturbato, questo canale non viene usato sempre e solo da una persona, ma viene condiviso tra tutti gli utenti che attualmente utilizzano la tecnologia bluetooth in quel luogo utilizzando quello spettro di frequenze.

Direct Sequence Spread Spectrum

Questo meccanismo è veramente particolare, perché fino ad ora abbiamo parlato di casi critici, ovvero di interferenze che compromettono il messaggio rendendo impossibile la comunicazione. Adesso invece, introduciamo un sistema che permette e che garantisce una corretta comunicazione anche se più persone utilizzano la stessa frequenza per comunicare.

In questo meccanismo, dunque, non esiste una separazione dello spettro, ma diversi mittenti e diversi destinatari ascoltano lo stesso canale (stessa frequenza) e tramite un meccanismo "chip code" ci si riesce ad isolare dal rumore e si riesce ad estrarre l'informazione trasmessa.

Questa è una **TAAC**, ovvero una tecnica di accesso a condivisione del codice.

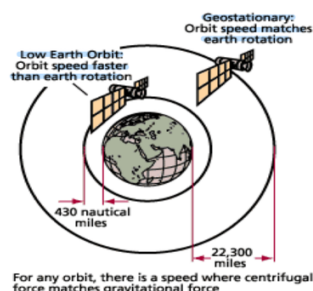


Con il chip-code è quindi possibile trasmettere l'informazione che vogliamo, in quanto il destinatario interessato a quel particolare mittente riuscirà ad ascoltare quell'esatto chip-code.

La lunghezza del chip-code può essere variabile, ad esempio, 10 bit di chip-code possono essere 1 bit di informazione, oppure 5 bit di chip-code 1 bit di informazione. Questo dipende da quanto è rumoroso il canale.

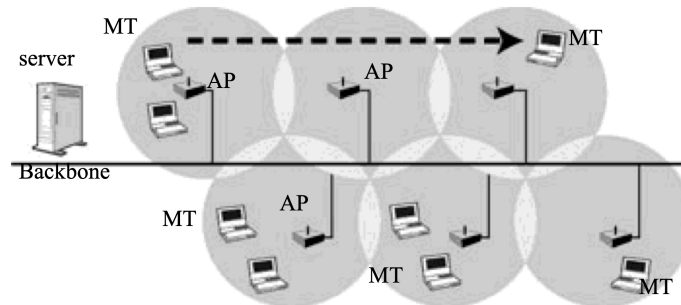
Satelliti

I satelliti a **bassa orbita terrestre** sono più veloci della rotazione terrestre (usati per reti mobili e cellulari), mentre satelliti **geostazionari** sono tanto veloci quanto la rotazione terrestre (usati per le telecomunicazioni).

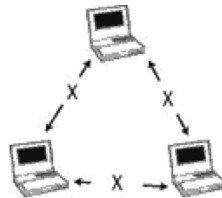


WWAN e WMAN

Una possibilità per la creazione di reti wireless è avere una dorsale, collegata ad un server e agli access point, gli access point si occupano di interpretare le onde radio, per poi trasferirle sulla rete standard, quindi sulla rete cablata.



Un'altra possibilità è quella di creare una comunicazione P2P completamente wireless, quindi senza costi di setup, come nel caso precedente (dove bisognava tirare dei fili). Questa soluzione si può mettere in piedi in poco tempo e non ha un costo vero e proprio. L'idea di questo tipo di rete è la connettività che hanno le auto del "futuro" a guida autonoma, dove ogni peer scambia delle informazioni ad altri peer.



Sostanzialmente in un'infrastruttura (da cablata a wireless) e viceversa, l'access point permette di fare da ponte, agendo sia lato wireless che lato fisico.

Svantaggi Wireless e Problemi

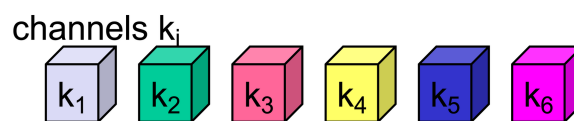
Come è possibile intuire le reti wireless sono limitate da una serie di diversi fattori, che portano con sé una serie di diversi problemi. Questi problemi possono dare dei buoni spunti di riflessione

- Capacità del canale ridotta: la rete cablata viaggia a Gigabit/s, mentre la rete wireless è più limitata, si parla generalmente di Megabit al secondo.
- Spettro di frequenze eterogeneo limitato e ridotto
 - Bisogna rispettare dei piani di allocazione delle frequenze nazionali ed internazionali
 - Si ha la necessità di riutilizzare le frequenze (proprio per il motivo della scarsità)
- Energia Limitata delle batterie
- Rumore ed interferenze: hanno un grande impatto sulle performance del canale wireless e hanno anche un grande impatto sul design dei sistemi wireless. Questo è parzialmente risolto dalla quantità di energia emessa, più energia, più il segnale è chiaro e meno bit sono disturbati, ma non sempre questo è concesso (leggi e limitazioni)
- Sicurezza: le informazioni sensibili viaggiano nell'aria e sono leggibili ed intercettabili da chiunque, c'è quindi bisogno della cifratura e dell'autenticazione.
- Problemi di spostamento: quando si viaggia in macchina il pacchetto deve rincorrere un dispositivo che nel tempo e nello spazio sta viaggiando ad una certa velocità
- Servizi Best Effort: la precisione del cablato non è raggiungibile.

Multiplexing – Riguarda la gestione utente-antenna

Il multiplexing è il riuso del canale wireless, l'obiettivo di ogni modello riguardo il riuso di un mezzo condiviso. Ecco l'introduzione alle tipologie di multiplexing:

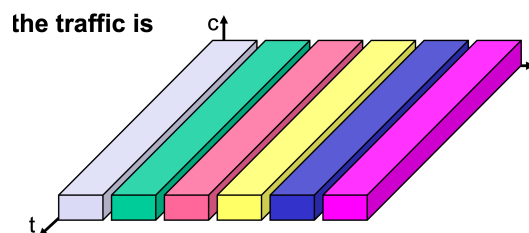
- Spazio (s): la stessa frequenza può essere utilizzata in luoghi diversi (a patto che siano abbastanza distanti) in quanto la soglia di attenuazione delle onde radio è così elevata che un dispositivo che comunica sulla frequenza α a Bologna non intacca un dispositivo che trasmette sulla stessa frequenza a Firenze. In generale, ognuno (in un raggio determinato) utilizza una frequenza diversa.
- Time (t): tutti all'interno dello stesso raggio determinato utilizzano lo stesso canale, ma in tempi e istanti diversi. Questo comporta attese (magari anche lunghe) perché prima di poter trasmettere si ha la necessità di attendere il termine di una trasmissione altrui
- Frequenza (f): ognuno usa il proprio canale.
- Code (c): tutti usano la stessa frequenza, e grazie a un chip-code la comunicazione passa da rumorosa a nitida e ben distinguibile.



Nota! Il canale è una porzione specifica dello spettro (che può avere diverse larghezze). I canali vengono distanziati in modo da evitare interferenze

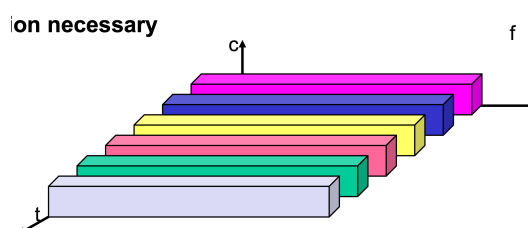
Frequency Multiplexing

Per tutto il tempo della trasmissione si ascolta solo una porzione di spettro, ovvero una specifica frequenza. Il **vantaggio** è che nessuna coordinazione è necessaria e che questo meccanismo funziona molto bene anche per i segnali analogici. Gli **svantaggi** sono uno spreco di frequenza (**spazio di guardia** tra frequenze diverse, necessario per non andare in sovrapposizione), la non flessibilità e specialmente la perdita di spettro se l'utilizzo delle frequenze non è ben distribuito. Inoltre, un attaccante può tranquillamente posizionarsi su una frequenza ed ascoltare o disturbare con molta semplicità la comunicazione



Time Multiplexing

Un canale ottiene l'intero spettro di frequenze utilizzate per un certo lasso di tempo. In sostanza tutti trasmettono dove tutti parlano, ma ognuno può parlare in istanti diversi e per un periodo specifico di tempo. Il **vantaggio** è che è necessario solo un **carrier** e il throughput è equo tra tutti gli utenti. Lo **svantaggio** è che viene richiesta una sincronizzazione maniacale, che è molto difficile da raggiungere. Basta avere una sincronizzazione debole che la comunicazione è compromessa

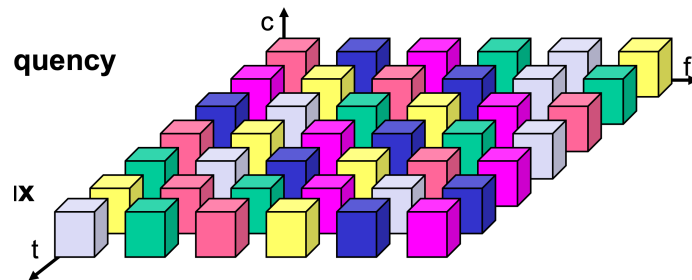


Time e Frequency Multiplexing

Questo è letteralmente il frequency hopping su finestre di frequenze a banda larga. Tecnicamente la trasmissione è continua, ma in istanti temporali e frequenze diverse.

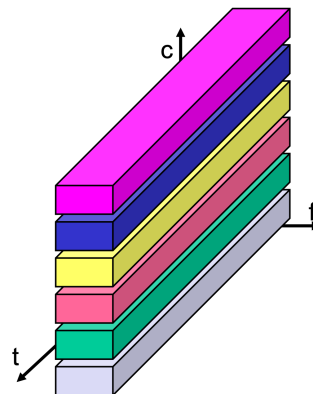
Precisamente, un canale ottiene una certa banda di frequenza per un certo lasso di tempo. Vengo fatti quindi tutti molti salti, che possono sembrare casuali, ma che seguono un algoritmo. Questo metodo ha con sé molti **vantaggi**, il primo è la sicurezza; infatti, un attaccante non riuscirà mai ad intercettare la comunicazione completa, ma solo frammenti random insignificativi.

Inoltre, se una particolare frequenza è molto rumoroso o presenta interferenze, ognuno avrà la stessa interferenza, ma per un periodo di tempo limitato, ciò significa che non sarà solo una persona a subire una comunicazione disturbata. Lo **svantaggio** è un'estrema sincronizzazione richiesta.



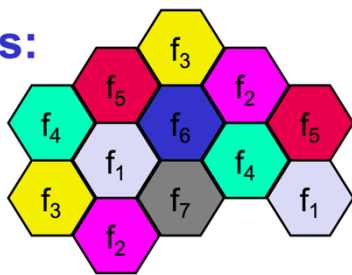
Code Multiplexing

Tutti i canali usano lo stesso spettro allo stesso istante, ma ogni canale ha un codice univoco che permette di identificare, distinguere e capire la trasmissione. In sostanza ogni canale usa la propria chipping sequence e si concentra sulla propria comunicazione. I **vantaggi** sono una soluzione con della larghezza di banda efficiente (solo parte di un grosso spettro) viene utilizzato, inoltre non c'è bisogno di coordinazione e sincronizzazione, inoltre è molto sicuro per gli attacchi (una persona non può far rumore, perché non usa la mia chipping sequence, e dunque non può capirmi). Lo **svantaggio** è un grande costo della rigenerazione del segnale.



Frequency Planning – Riguarda la gestione Antenna-Antenna

es:

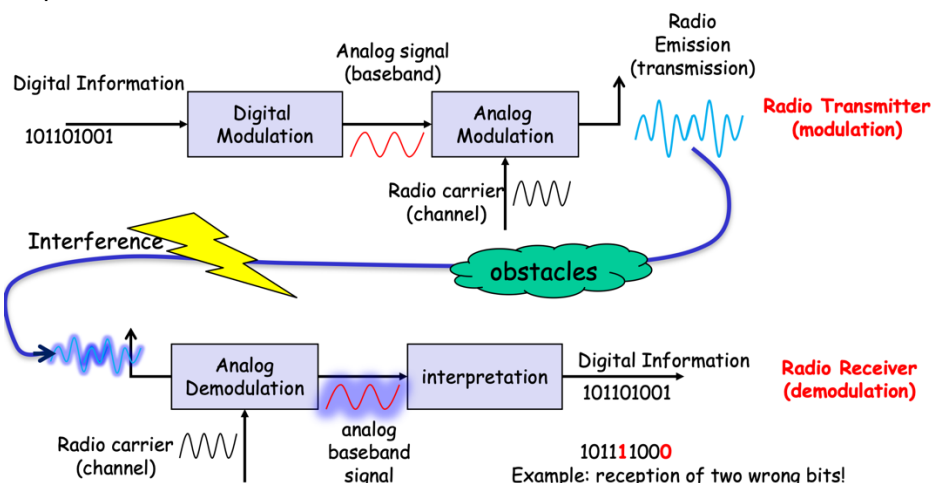


Le frequenze sono una risorsa limitata, non si può assegnare una frequenza unica a ogni dispositivo al mondo. Dunque, il territorio viene suddiviso in esagoni convessi, dove ogni esagono rappresenta l'area di copertura di una singola antenna (chiamata base station). Ogni frequenza, indicata con un colore diverso può essere riutilizzata non in modo adiacente, ma può essere riutilizzata solo se esiste una certa distanza fisica tra le stazioni base.

L'assegnamento delle frequenze può essere **statico**, quindi ogni esagono ha quella frequenza immutabile, oppure **dinamico**. Nel caso statico la rete non è uniforme in caso di sovraccarichi di connessioni, nel caso **dinamico** invece le frequenze non sono bloccate. La stazione base sceglie quali frequenze usare momento per momento, basandosi su cosa stanno utilizzando le celle vicine in quel preciso istante. Questo ha un grande vantaggio in quanto permette di dare maggiore capacità (più frequenze) alle celle dove c'è un picco di traffico.

Modulazione

Dobbiamo adesso comprendere come modulazione, quindi come convertire i bit in frequenze radio. Dall'immagine sottostante capiamo come funziona la modularizzazione, poi spiegheremo le tecniche diverse esistenti per la modulazione.



Abbiamo l'informazione digitale (i bit), che passa attraverso la **digital modulation**. Questo è il primo passo che bisogna compiere, ovvero viene creata una sinusoide in banda base (baseband) leggendo le informazioni digitali. Questa sinusoide è grezza, è una copia dei bit, per ora incomprensibile dal destinatario in quanto usa frequenze troppo basse.

Successivamente la sinusoide grezza passa attraverso la modulazione analogica. In questo istante, alla sinusoide iniziale gli viene aggiunto il carrier. Il carrier non è altro che l'onda fisica vera e propria che oscilla nell'aria alla frequenza su cui è centrato il canale trasmissivo. Il carrier ha quindi il compito di trasportare l'informazione digitale; infatti, una volta terminata la modularizzazione analogica l'onda radio viene trasmessa da un'antenna. L'onda radio viaggia nello spazio, trova ostacoli e subisce interferenze.

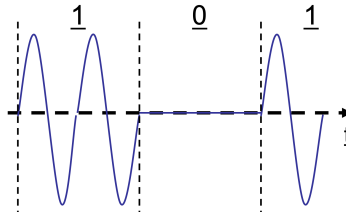
Successivamente l'onda arriva al destinatario finale (più bassa in quanto ha perso energia). Per mezzo del filtro passabanda (gli occhiali speciali che permettono di leggere quella frequenza) e grazie alla topologia di multiplexing usato, il destinatario fa il percorso a ritroso, quindi separa il carrier dal dato, effettua l'interpretazione dell'onda risultante e infine recupera l'informazione digitale originaria, nonché la sequenza di bit iniziale.

Digital Modulation – Tecniche

Esaminiamo quindi le tecniche di modellazione digitale, ovvero il primo scatolotto che si incontra nell'immagine, ovvero quel mezzo che permette il passaggio da bit grezzo a **base band**.

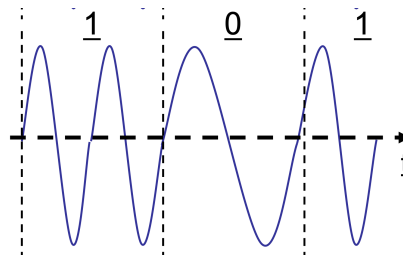
Ci sono diverse tecniche di modulazione digitale, ne esamineremo alcune, tra cui *ASK*, *FSK* e *PSK* e alcuni sottoinsiemi.

ASK (Amplitude Shift Keying) è sicuramente il modello più semplice. Si effettua un semplice on/off (basta un transistor). Questo è un modello semplicissimo ed economico (una frequenza utilizzata), ma è molto sensibile alle interferenze.



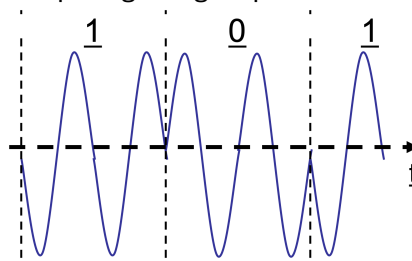
Supponiamo che la frequenza sia 2 Hz, vediamo che la durata di rappresentazione di un bit dura un certo lasso di tempo, questo lasso di tempo viene chiamato: **simbolo**. Il simbolo è quindi quanto dura un bit nel tempo. Dall'immagine vediamo che il bit "1" ha una sinusoide, mentre il bit "0" è spento, è vuoto (OFF). Questo modello, seppur semplice e con una sequenza è debole, in quanto ogni disturbo ed interferenza nel simbolo dello "0" non sarà correttamente interpretabile, potrebbe essere uno "0", ma anche un "1".

Con **FSK** (Frequency Shift Keying) si usa più spettro; infatti, come si vede dall'immagine si usano frequenze alte e frequenze basse.

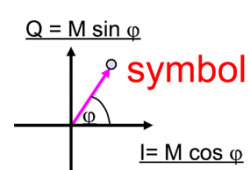


A parità di simbolo, possiamo vedere che la frequenza del bit "1" è 2 Hz e la frequenza dello "0" è 1 Hz. L'uso di frequenze diverse usa più spettro, ma è più robusto contro le interferenze.

Con **PSK** (Phase Shift Keying) si utilizza la stessa frequenza, ma si cambia solo la fase. Questo sfasamento è molto più difficile da implementare, ma è molto più robusto contro le interferenze, inoltre si possono fare più livelli di fase per ogni segnale possibile.



Il metodo ad oggi più usato è proprio quest'ultimo infatti è il più robusto.



Il metodo accennato adesso, e raffigurato nell'immagine è molto semplice, quasi primordiale, in quanto ad oggi si usano tecniche molto più efficaci e complesse. Prima di andare nel dettaglio, abbiamo visto che il segnale si può rappresentare modificando l'ampiezza, la frequenza, la fase, oppure, la fase e l'ampiezza.

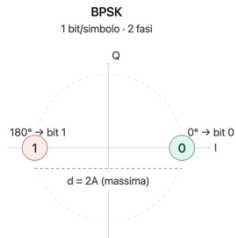
Per quanto tratteremo più avanti è utile considerare il diagramma riportato qui a lato.

Il diagramma riportato è un diagramma in coordinate polari, dove:

$$M = \text{ampiezza e } \varphi = \text{fase}$$

Se a questo diagramma associamo dei punti, e a dei punti associamo dei valori possiamo ottenere diversi modelli di **Phase Shift Keying**.

Il primo modello è il **BPSK**, che non è altro che un sottoinsieme, ovvero fa parte della famiglia del **PSK** accennato prima.

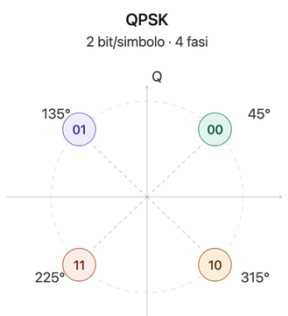


Tralasciando come è disposto il singolo bit (la coordinata in corrispondenza del singolo bit, che in alcune fonti può essere diversa), vediamo che il bit **0** in quest'immagine è trasmesso con $\sin(t)$ phase 0° e che il bit **1** è trasmesso, in modo analogo con $\sin(t)$ phase 180° . Questo è il modello PSK più semplice e robusto, ed è utilizzato nelle comunicazioni satellitari. Tuttavia, questo modello è piuttosto lento, ha un bitrate basso, in quanto si ha un solo bit per simbolo, dove il simbolo non è altro che la coordinata polare.

Dunque, in questo modello si hanno due codifiche per due simboli diversi.

Se ora volessimo utilizzare lo stesso modello, ma volessimo andare più veloci, quindi avere un bitrate più elevato possiamo introdurre più punti sulle coordinate in coordinate polari, dunque definire più simboli e assegnare ad ogni simbolo due bit.

Questo modello si chiama **QPSK**, ovvero, quadrature shift keying.



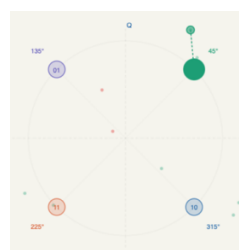
Come è possibile immaginare, questo modello è più complesso, e dunque più vulnerabile. In quanto un simbolo che arriva all'origine degli assi può avere qualsiasi rappresentazione. In ogni modo, se lasciamo da parte un attimo questo problema (risolto con codice di Hamming), chi riceve il messaggio deve interpretare il simbolo, che potrebbe non giungere nel punto esatto previsto dalla codifica, ma potrebbe avere una coordinata polare leggermente distante da quella reale. Anche se ciò può accadere (e nella realtà accade spesso) il ricevitore è in grado di ricostruire il messaggio, in quanto, sulla base del quadrante su cui si posiziona il

simbolo ricevuto, è possibile rideterminare il simbolo esatto che è stato trasmesso, dunque è possibile interpretare correttamente la comunicazione.

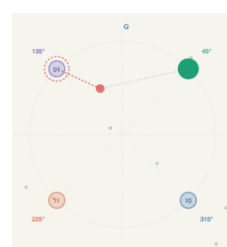
Andando nel dettaglio, nel canale radio è presente un rumore gaussiano nelle direzioni 'I' e 'Q'. Questo rumore impatta sulla corretta ricezione del messaggio, non posizionando il simbolo corretto esattamente sulla coordinata polare prevista dalla mappatura standard. Il ricevente deve quindi applicare la regola del "nearest-neighbour", ovvero del vicino più vicino, che è trovato grazie alla **distanza euclidea minima** tra il simbolo ricevuto e il simbolo codificato dallo standard.

Più ovviamente un canale è rumoroso, più ci sono errori.

Solitamente, e come vedremo più avanti, si usano diversi standard a seconda delle situazioni.



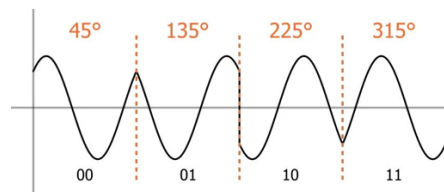
Rx Corretto



Rx Errato

La disposizione dei simboli in coordinate polari non è affatto casuale, ma segue la **codifica di Gray**, dove simboli adiacenti nella costellazione differiscono esattamente di **1 bit**, cioè la **distanza di Hamming** è esattamente pari a 1. Questo è cruciale in quanto il rumore radio tende a spingere un simbolo verso il vicino più prossimo. Se il ricevitore sbaglia quindi un simbolo probabilmente decodifica il vicino, e grazie alla distanza di Hamming pari a 1, quell'esatto simbolo vicino differisce solo di un bit. Un errore produce quindi un solo bit errato e non due.

Quindi, non tutte le associazioni di simboli alle coordinate polari vanno bene, solitamente se ne vedono due, una è quella riportata, l'altra è quella che vede le etichette delle coordinate polari sulla diagonale scambiate.



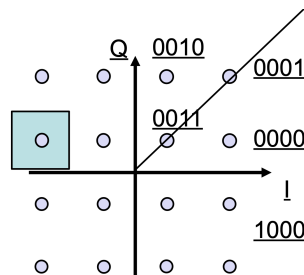
Vediamo quindi come la stessa sinusoide, sfasata è associata a simboli diversi

L'importanza della distanza di Hamming pari a **1** è rafforzata dall'utilizzo del bit di parità, in quanto si può utilizzare il bit di parità per trovare l'errore e la parità incrociata per autocorreggere l'errore quando c'è solo un bit errato nella comunicazione. Questa combinazione ha la massima efficienza e livello di bitrate. Il suddetto meccanismo dei bit di parità viene implementato nel livello MAC/LLC e permette di accorgersi e modificare l'errore ancora prima del livello trasporto, incrementando l'efficienza e riducendo la necessità di ritrasmissione dello stesso segmento.

QAM

Se il canale ha una qualità ancora migliore, si può spingere la codifica ancora di più. Questo metodo, chiamato QAM combina la modulazione sia dell'ampiezza che della fase del segnale per ogni simbolo trasmesso. Quindi, questo metodo gioca sia con la fase che con l'ampiezza.

Nel caso dell'esempio sottostante si abbiamo un caso di **16-QAM**, con 16 simboli, dove un simbolo rappresenta 4 bit. Questo metodo è ottimo, però l'area del simbolo target si riduce quando il numero i bit aumenta. Se prima avevamo un'area di target molto grande, adesso come si può vedere nell'immagine l'area target è nettamente meno vasta.

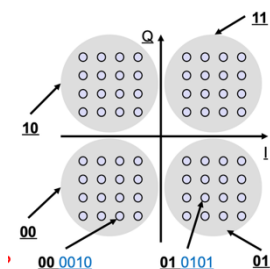


Dove l'area target è raffigurata dal quadrato azzurro. Come si vede, la semiretta tracciata rappresenta la fase; quindi, sulla stessa fase ci sono due simboli diversi, che differiscono solo di un bit, ma hanno ampiezza diversa.

Questo discorso lo si può portare quanto si vuole all'estremo, infatti è possibile introdurre un **64-QAM**, dove ci sono $\#simboli = 64/4_{aree}$, quindi 16 simboli. E ogni **simbolo** ha $\log_2(64) = 6$ bit.

64-QAM è molto prestazionale, ma appena il canale è rumoroso la ricezione presenta troppi, che non si riescono a correggere, in quanto l'area target è troppo piccola per riuscire a determinare un "nearest-neighbour" corretto.

La soluzione è quindi mantenere l'idea della velocità (64 simboli sono molti – tanto bitrate), ma mantenere i quattro quadranti principali per affidabilità. Schematizzando il tutto:



si usano i 64 simboli per trasmissioni di contenuti e informazioni che possono contenere anche degli errori nella ricezione (immagini e video) e si usano quattro simboli (uno per ogni quadrante) per la trasmissione di informazioni che non possono contenere degli errori.

È anche possibile utilizzare dei **multicarrier OFDM**, ovvero dei sotto-canali del canale, riuscendo a costruire bitrate molto elevati.

$$\text{Bitrate} = \#subCarrier * \#simboli/s * \#bit\ codificati$$

La situazione perfetta per immaginare l'utilizzo combinato è quello di una video chiamata, dove la voce di chi comunica è più importante del video di chi parla; dunque, si preferisce avere meno bit per simbolo sulla voce (aumentando la precisione) e avere più bit per simbolo per il video (diminuendo l'accuratezza, ma favorendo la velocità).

Immaginiamo la seguente sequenza:

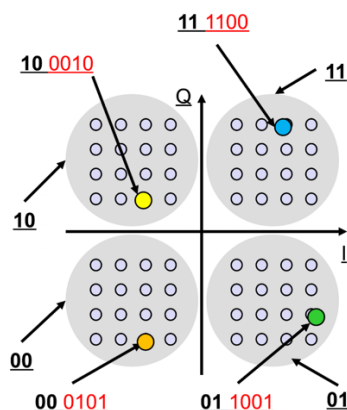
Audio: 10 01 11 00

Video: 0010 1001 1100 0101

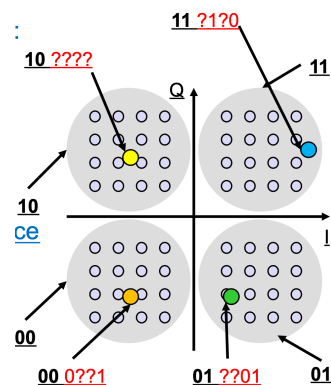
E concateniamo la sequenza audio e video:

100010 011001 111100 000101

In modo tale da poterla trasmettere con 64-QAM, ma suddividendo il "rischio" ottenendo:



Situazione ideale, dove il ricevente riesce ad interpretare correttamente sia la voce che il video, con poco discostamento, rilevabile ad auto correggibile



Situazione pessima, ma comunque accettabile, dove pur essendoci tanta interferenza il ricevente riesce a comprendere bene il dato sensibile (audio), sacrificando il video.

Anteponendo quindi il dato sensibile, anche in caso di trasmissione disturbata, il dato riesce sempre a ricadere nella più ampia zona correttamente interpretabile, ovvero quella dei quattro quadranti.

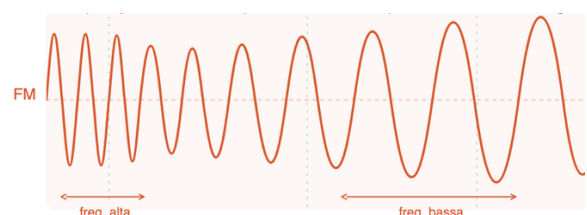
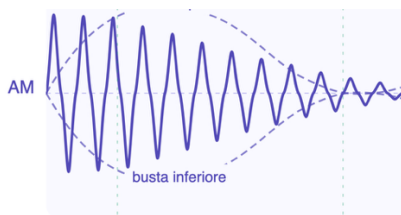
Modulazione Analogica

Nella modulazione analogica, si prende il segnale continuo in banda base (baseband) e lo si usa per variare uno dei tre parametri fisici (ampiezza, frequenza o fase) di un segnale ad alta frequenza, detto **portante** o **carrier**.

L'equazione del portante è la seguente: $\sin(t) = A * \cos(2\pi * f * t + \varphi)$

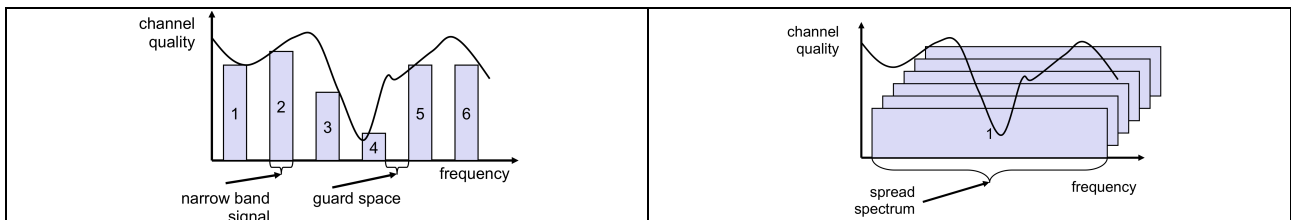
Nel caso della **Amplitude Modulation** (AM), l'ampiezza A del carrier varia proporzionalmente al segnale modulante, mentre la fase e la frequenza restano costanti.

Nel caso di **Frequency Modulation** (FM), la frequenza istantanea f varia proporzionalmente al segnale modulante, mentre ampiezza e fase rimangono costanti. Un suono più forte o più acuto sposta la frequenza del carrier verso l'alto o verso il basso.



Spread Spectrum Technology

Precedentemente abbiamo parlato del code multiplexing e del time + frequency multiplexing. Adesso è ora di entrare nel dettaglio e di capire la vera differenza tra i canali narrowband e i canali spread spectrum.



Come si vede nella prima immagine ritroviamo la situazione tanto discussa dei canali narrowband, dove una sola frequenza subisce delle interferenze, dovuta ad una qualità bassa del canale. Con l'altra tecnologia, invece si ha una situazione opposta, infatti, tutti usano contemporaneamente lo stesso canale. Questo viene raggiunto grazie ad una quarta dimensione puramente matematica.

Il problema della trasmissione in spread spectrum è quando una forte frequenza incombe su un canale radio, spazzando via completamente il segnale per tutta la durata dell'interferenza. Dettagliatamente, il problema si manifesta quando il picco di interferenza, supera lo spread signal, quando questo succede il SNR è basso (c'è più rumore che segnale), alterando e compromettendo la trasmissione. L'obiettivo dei metodi che dopo vedremo nel dettaglio è quindi quello di trasformare il picco di interferenza in picco di segnale, e modellare l'interferenza come un lieve rumore di sottofondo, completamente trascurabile.



Nella prima immagine si vede il problema, e nella seconda, la soluzione desiderata. Intuitivamente possiamo vedere questa soluzione tecnologica nel modo seguente:

Da trasmettere: **Hello World**

Narrow Band: **Hello World**

Frequency Hopping: **Hello World**

Spread Spectrum: **Hello World**

Dove il colore bianco rappresenta l'unione dei colori di tutti i canali che fanno parte della trasmissione

Se a queste tre metodi di comunicazione applichiamo la stessa interferenza XXXXXXXXXX

Otteniamo una situazione di questo tipo:

Narrow Band: **Hello World** +

XXXXXXXXXX = **He ld**, il risultato è una comunicazione compromessa

Frequency Hopping: **Hello World** +

XXXXXXXXXX = **He...lo W...rld**, il risultato è un messaggio parzialmente alterato

Spread Spectrum: **Hello World** +

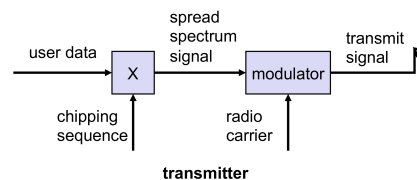
XXXXXXXXXX = **Hello World**, messaggio completamente corretto

Dunque, con il metodo DSSS (Direct Sequence Spread Spectrum) è possibile ridurre le frequenze con un fading selettivo, ovvero quelle frequenze svantaggiate per cause di forza maggiore, inoltre è possibile il riuso di frequenze già utilizzate, comunicando correttamente su tali frequenze.

Tuttavia, ogni tecnologia bella ha un tasto dolente, infatti il meccanismo di sincronizzazione e di riuso del canale è complesso, richiedendo sincronizzazione, processori e moduli dedicati.

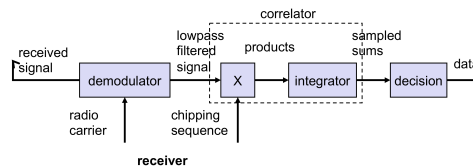
Il cuore pulsante di questo meccanismo è la **chipping sequence**, un codice che permette a tutti gli effetti di criptare e decriptare la trasmissione del messaggio per farlo sopravvivere alle interferenze recate dalle altre comunicazioni.

Prima di fare qualche esempio, vediamo come viene utilizzata la chipping sequence e quali sono i passaggi necessari per inviare il messaggio tramite questa soluzione spread spectrum.



Per il DSSS, questo è il circuito del mittente, che ai dati, aggiunge la sua chipping sequence in fase di modulazione digitale e poi al segnale dello spread spectrum aggiunge il carrier (modulazione analogica). L'unica differenza, rispetto al modello precedentemente visto, per il mittente, è stabilire la chipping sequence e utilizzare tale sequenza nella modulazione digitale.

Per il ricevente, invece le cose si fanno leggermente più complicate in quanto la demodulazione analogica avviene in modo analogo, ma la l'interpretazione avviene in modo diverso, in quanto si ha la necessità di estrarre le informazioni cercando somiglianze (effettuando delle operazioni di calcolo) con la chipping sequence impostata nell'apertura della comunicazione, integrando poi tali informazioni con un circuito integratore per capire il vero e proprio valore del bit trasmesso.

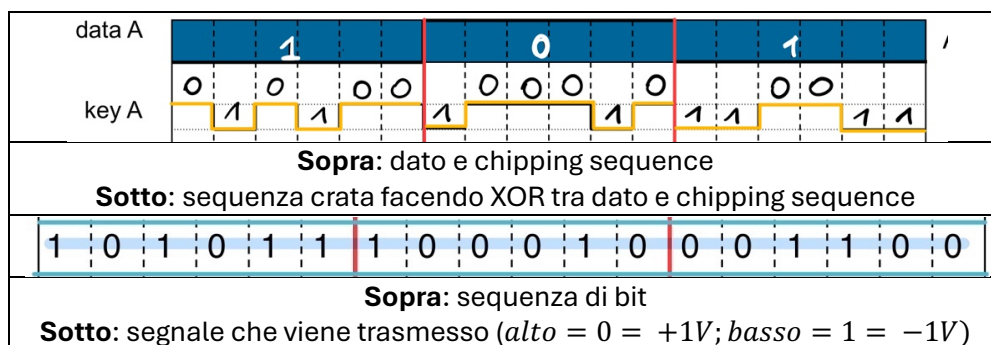


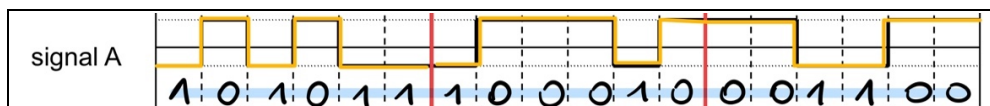
Facciamo un esempio per capire il funzionamento e dare più dettagli su come funziona tale tecnologia. Questo esempio segue la convenzione $0 = +1V$ e $1 = -1V$.

Dato un dato da trasmettere dall'utente A, dove il dato è rappresentato dalla sequenza di bit **101**, e data una chipping sequence (stabilita con il destinatario all'avvio del canale utilizzando le due apposite frequenze lasciate libere dallo standard studiato), il mittente compie il primo passo, porzionando la chipping sequence in blocchi da n bit per bit da trasmettere. Dunque, se i bit da trasmettere sono 3, e la chipping sequence è di 18 bit, si può utilizzare l'intera chipping sequence porzionata in 3 blocchi da 6 bit ognuno.

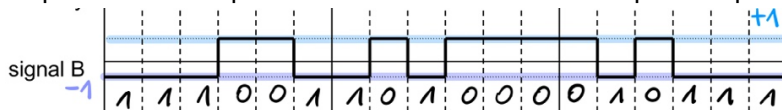
A questo punto si effettua l'operazione XOR tra dato e chipping sequence (rappresentata come linea alta per bit 0 e linea bassa per bit 1). Il risulta dell'operazione matematica è il dato (sotto forma di segnale) che verrà trasmesso dall'antenna.

Come convenzione matematica fondamentale, il segnale ricavato ha il bit 0 come stato **alto** e il bit 1 come stato **basso**.



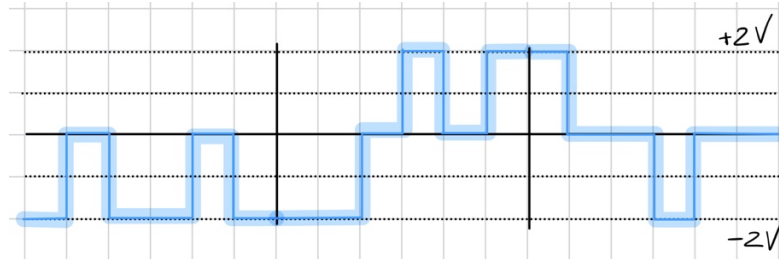


Supponiamo di fare la stessa cosa, ma adesso per un altro segnale B, applicando una chipping sequence diversa possiamo ottenere una cosa di questo tipo:



Di conseguenza, nello spazio, la somma **vettoriale** dei due segnali sarà come segue:

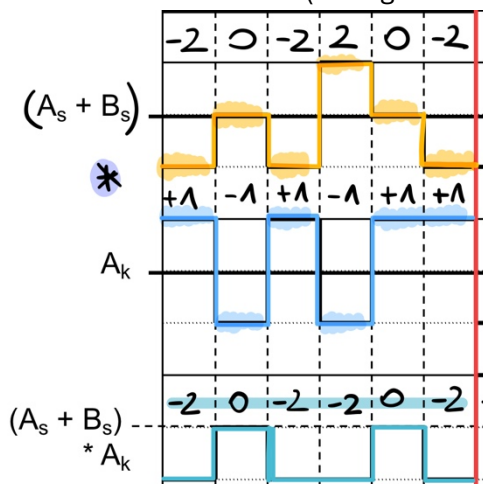
$A_s =$	-1	1	-1	1	-1	-1	-1	1	1	1	-1	1	1	1	-1	-1	1
$B_s =$	-1	-1	-1	1	1	-1	-1	1	-1	1	1	1	1	-1	-1	-1	-1
$A_s + B_s =$	-2	0	-2	2	0	-2	-2	2	0	2	0	2	2	0	0	-2	0



Nello spazio, quindi viene propagato il segnale azzurro, che viaggia nello spazio e nel tempo fino a che non giunge al destinatario di A.

Il destinatario, da questa cosa non capisce assolutamente niente fino a che non prende il segnale ricevuto, e ci moltiplica bit a bit il segnale della chipping sequence, considerando che il valore alto è una moltiplicazione $+1 *$ e il valore basso è una moltiplicazione $-1 *$

Dunque: provvediamo a fare il calcolo del primo blocco da sei bit con la chipping sequence di A e vediamo cosa salta fuori (dettaglio nell'immagine):



vediamo quindi che il risultato è stato ottenuto facendo:

$$\begin{aligned}
 &(-2) * 1 + \\
 &0 * (-1) + \\
 &(-2) * 1 + \\
 &2 * (-1) + \\
 &0 * 1 + \\
 &(-2) * 1 = -2 + (-2) + (-2) = -6
 \end{aligned}$$

Visto che il valore è negativo, il bit del dato è 1

Analogamente, se il valore è positivo, il bit del dato è 0

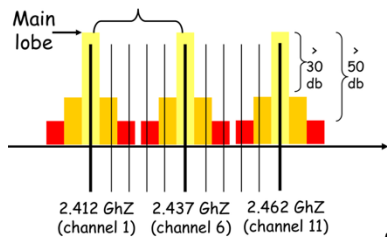
Nota! Con lo standard inverso, il risultato è il medesimo, ma per le comunicazioni è preferito lo standard usato nell'esempio.

Questo meccanismo trasforma un'interferenza in qualcosa di leggibile e reinterpretabile proprio grazie alla chipping sequence, di conseguenza più bit di chipping sequence si utilizzano, più precisione si avrà; dunque, sarà possibile un aumento della distanza di trasmissione migliorando il grado generale della comunicazione.

Il vero e proprio utilizzo di questo meccanismo (DSSS) si vede in IEEE 802.11b, ovvero l'assegnamento del canale wireless del router Wi-Fi.

Il Wi-Fi 2.4 Ghz ha uno spettro di canali che va da 2.412 Ghz fino a 2.484 Ghz, dove ogni canale ha uno spacing, uno spazio di 25 Mhz. In una situazione ideale, dove poche persone hanno un router Wi-Fi

Si può vedere una situazione di questo tipo (dove **ogni** linea nera è un canale):

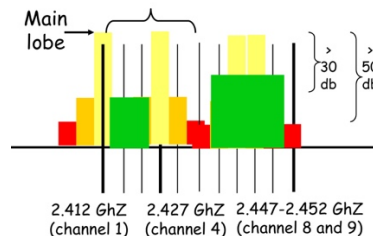


dove ci sono solo tre canali utilizzati, rispettivamente il canale 1, 6, e 11. Ognuno di questi canali è a sé stante, non si tocca, non si conosce e specialmente può trasmettere senza interferenze. Per la precisione, il lobo giallo è quello più potente, quello arancione è dove viene iniettata energia più bassa (si parla di -30dB in meno rispetto al lobo principale) e il lobo rosso è quello dove viene iniettata ancora meno energia (1/100 dell'energia gialla).

Ma visto che non siamo in un film, e visto che c'è una quantità smisurata di router in un condominio per forza di cose si ha una contesa; quindi, canali vicini non sono in contesa nei lobi dove viene trasmessa più energia, ma lo sono dove viene trasmessa meno energia (lobi arancioni e rossi).

Tuttavia, queste contese sono proprio contrastate dalla tecnologia Spread Spectrum, in quanto la chipping sequence personale del router Wi-Fi riesce ad isolarsi dal rumore, trasmettendo correttamente i bit. È però ovvia una cosa, il bitrate è più basso in quanto il circuito integratore fa più fatica; infatti, dove c'è molta contesa del canale si ha necessità di adottare un chip-code più ampio, questo per riuscire a trasmettere in modo affidabile e corretto l'informazione.

Non è raro incontrare overlapping di canali sullo stesso canale, ovvero router diversi usano lo stesso canale, non mandando in contesa solo i lobi minori, ma anche il lobo maggiore, peggiorano la qualità del servizio.



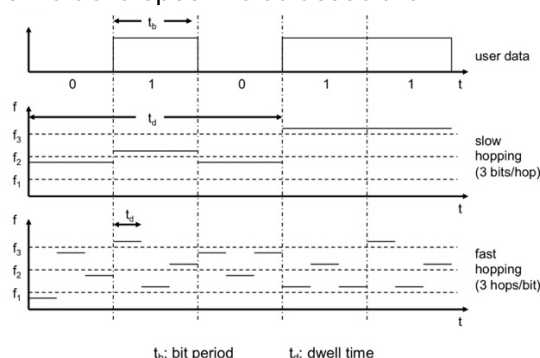
Frequency Hopping Spread Spectrum

Questa tecnica l'avevamo già accennata precedentemente, adesso possiamo dire che ci sono due versioni:

- Fast Hopping: molte frequenze per bit
- Slow Hopping: molti bit per salto

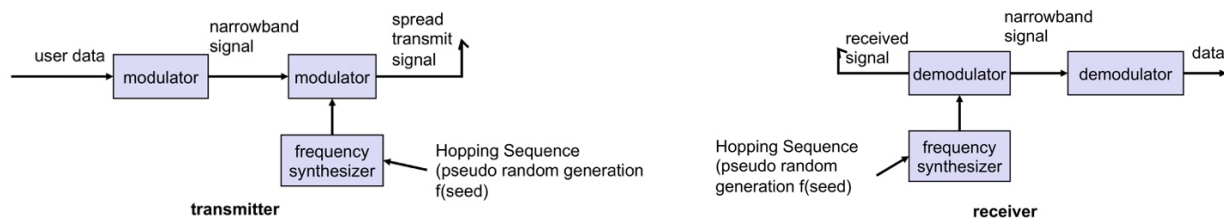
I vantaggi sono quelli già spiegati: interferenze e fading limitati a un breve periodo, implementazione semplice e utilizzo di uno spettro ridotto, mentre lo svantaggio è una robustezza limitata.

I salti, come già detto in precedenza sono determinati da un seed, determinato tramite una funzione che prende come parametro l'id del dispositivo da associare.



Nel grafico si vede che lo slow hopping permane (**dwell time**) sullo stesso canale per la durata di 3 bit, mentre nel fast hopping il canale salta per ben tre volte per la durata di un singolo bit. Come si vede, il salto non è preciso sulla frequenza, ma è un po' più sotto o un po' più sopra. Se è sotto significa bit 0, se è sopra significa bit 1.

Questa sincronizzazione è raggiunta da un circuito che include anche un sintetizzatore di frequenze, dedicato a gestire la hopping sequence.



OFDM – Orthogonal Frequency Division Multiplexing

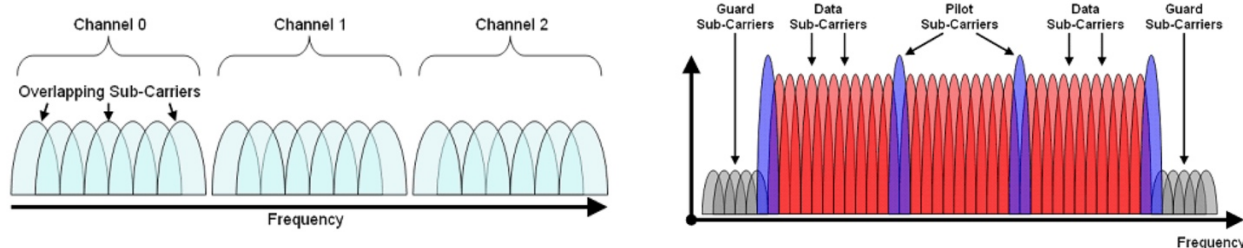
Come dice l'acronimo, questa tecnologia utilizza il multiplexing a divisione di frequenza ortogonale, sostanzialmente è la tecnologia rivisitata e potenziata del frequency multiplexing studiato prima. Questa è una delle tecnologia più importanti e diffuse al giorno d'oggi.

Sostanzialmente, questa tecnologia lavora con più **subcarrier**; invece, di avere un unico carrier che trasporta tante informazioni, si distribuiscono le informazioni, facendole portare a più carrier. L'approccio tradizionale single carrier mette tutte le informazioni su un unico e velocissimo carrier, se questo carrier incontra un'interferenza, o un rumore l'intero carico di informazioni subisce disturbi, ritardi, o peggio, sparisce.

L'approccio multicarrier invece è più furbo, usa più carrier, più lenti, ognuno caricato con meno informazioni. Anche se un carrier incontra un'interferenza, o un rumore, l'informazione che trasporta **solo** quel carrier subisce dei ritardi, mentre gli altri carrier sono intonsi, dunque portano correttamente il payload al destinatario.

Solitamente i canali **OFDM** hanno una dimensione di 20 MHz, ognuno del quale è diviso in 52-subcarrier, dunque, ogni carrier occupa 300 KhZ. Di questi 52 subcarrier, 4 sono utilizzati per il management, gli altri 48 sono utilizzati per il dato da trasportare (è possibile assegnare ad un simbolo più o meno bit, a seconda della qualità della rete).

Questo metodo, dunque, è più efficiente e più scalabile, innanzitutto perché si trasmettono in parallelo più simboli; inoltre, è più scalabile in quanto si mantengono gli stessi simboli e gli stessi bit-per-simbolo, ma qualora si desiderasse più bitrate basterebbe aumentare il numero di subcarrier. Questa tecnologia è molto utilizzata anche perché necessita uno spazio di guardia inferiore. Le trasmissioni radio classiche con frequenze troppo vicine creano interferenza tra di loro, ecco il motivo per cui si necessita di uno spazio di guardia ampio. Tuttavia, OFDM crea frequenze spaziate in modo matematicamente preciso (sono ortogonali), dove ogni frequenza raggiunge il suo picco massimo quando le onde di tutte le altre frequenze adiacenti sono esattamente a zero (la loro integrazione è a zero). Questo multiplexing solitamente viene combinato con il Time and Frequency multiplexing, creando di fatto un ibrido.



Nota! Con OFDM si possono usare tutte le tecniche di modulazione digitale che si vuole, da BPSK, QPSK, 16-QAM, 64-QAM...

Questo metodo si basa fortemente sui subcarrier ortogonali, infatti questo approccio permette una trasmissione simultanea su carrier diversi senza interferenze. Per subcarrier ortogonale si intende che $\int_0^T \sin(t) * \sin(kt) dt = 0$, dove il parametro k viene scelto in modo matematicamente accurato.

Visualmente, non si usa la prospettiva della somma vettoriale dei carrier, ma si utilizza la prospettiva della frequenza.

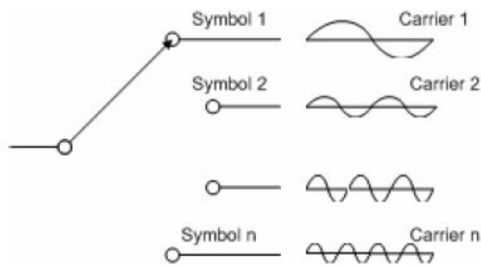
Se vogliamo trasmettere la sequenza **1 1 -1 -1 1 1 -1 -1** e se dividiamo la sequenza in base ai colori in più subcarrier, dove ad ogni subcarrier assegniamo una frequenza precisa si ha una situazione di questo tipo:

C1 (1 Hz): 1 1 1 -1 1 -1 1 -1...

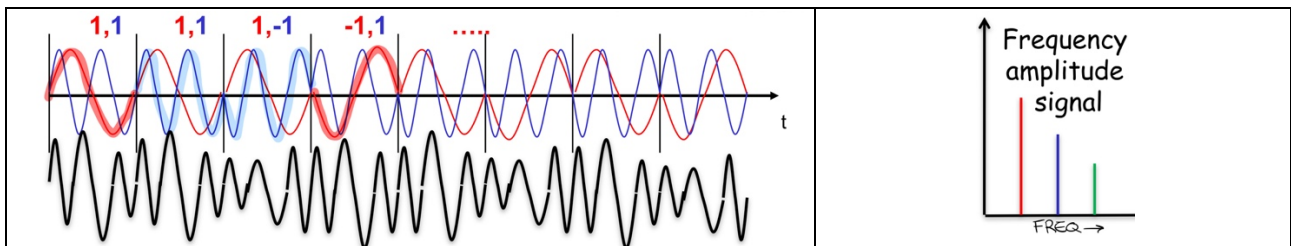
C2 (2 Hz): 1 1 -1 1 -1 -1 1 1....

C3 (3 Hz): -1 1 1 1 1 -1 -1 1....

C4 (4 Hz): -1 -1 1 -1 -1 1 1 -1



Se adesso sommiamo vettorialmente tutte le sinusoide prodotte da ogni tipo di subcarrier nello spazio verrà trasmessa una cosa raffigurata dalla sinusoide nera dell'immagine sottostante.



La visualizzazione di ciò, ovviamente è incomprensibile, non si riesce ad estrarre l'informazione trasmessa. Ma visto che i subcarrier sono ortogonali, tramite un circuito che implementa la trasformata di Fourier si riesce a visualizzare ed interpretare una cosa simile a ciò raffigurato nell'immagine del riquadro di destra della tabella soprastante.

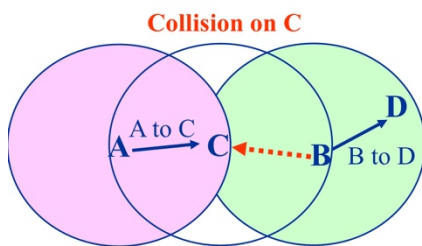
Supponendo un OFM con 250k modulazioni per secondo e 48 subcarrier possiamo ottenere i seguenti bitrate (combinando diverse tecnologie e mezzi)

Data Rate (Mbps) <i>1 Mbps = 1 Mbit/s</i>	modulation	Bits coded per phase transition <i>BIT PER SIMBOLO</i>	R = fraction of carriers used for convolution <i>immaginiamo BIT PARITÀ</i>	Length of 1 symbol at the given data rate (#subcarriers * bits coded per symbol)	Data bits encoded in 1 symbol
$48 \cdot 1 \cdot \frac{1}{2} \cdot 250k$	DBPSK	1	1/2	48 <i>(48 · 1)</i>	24 $\frac{48}{2}$
	DBPSK	1	3/4	48	36 $\frac{48 \cdot 3}{4}$
	DQPSK	2	1/2	96 <i>(48 · 2)</i>	48 $\frac{96}{2}$
	DQPSK	2	3/4	96	72 $\frac{96 \cdot 3}{4}$
	16-QAM	4	1/2	192 <i>(48 · 4)</i>	96 $\frac{192}{2}$
	16-QAM	4	3/4	192	144 $\frac{192 \cdot 3}{4}$
	64-QAM	6	2/3	288 <i>(48 · 6)</i>	192 $\frac{288 \cdot 2}{3}$
	64-QAM	6	3/4	288	216 $\frac{288 \cdot 3}{4}$

Terminali wireless Esposti e Nascosti – RTS/CTS

Fino ad adesso abbiamo parlato di come suddividere i canali, ovvero di come creare una coesistenza tra i canali, agendo di fatto al primo livello dello stack wireless. In questo breve accenno parliamo del secondo livello, ovvero del livello MAC (Medium Access Control). A questo livello si regola la coesistenza dei client all'interno di uno specifico canale.

Supponiamo di aver definito un canale, dove all'interno del canale tutti utilizzano la stessa chipping sequence. Ora, all'interno dello stesso canale è necessario avere dei protocolli interni, altrimenti si ha una guerra fratricida, dove si viene a distruggere tutto il “bello” e difficile creato e costruito al livello uno.

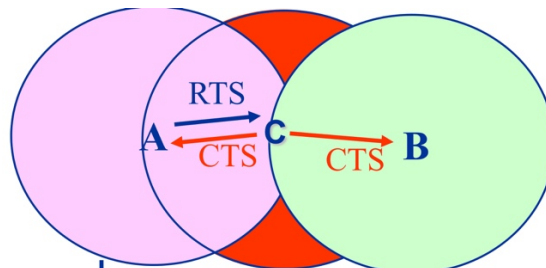


Supponiamo che B voglia trasmettere un messaggio a D, e che A voglia fare lo stesso verso C. All'inizio della dispensa avevamo parlato del protocollo Ethernet, che stabilisce che se il mittente vuole comunicare verso qualcun altro, allora chi vuole iniziare la comunicazione deve aspettare qualche millisecondo, e se in quella frazione di tempo non si sente già qualcun altro parlare, allora si può iniziare a trasmettere. Se durante la comunicazione si ha una collisione, allora il mittente deve aspettare un timeout, e

solo dopo quel timeout il mittente è riabilitato alla trasmissione. La stessa cosa succede anche nelle reti wireless, con un problema in più. Se A sta comunicando a C e B vuole comunicare a D, e B non si trova nella sfera di ricezione di A, ma C si trova esattamente all'interno delle sfere di ricezione di A e B si avrà una collisione, questo perché:

- A sta comunicando a C
- B vuole comunicare, ma non sente traffico, dunque inizia la trasmissione verso D
- C in quell'esatto istante sente sia A che B, dunque si ha una collisione.

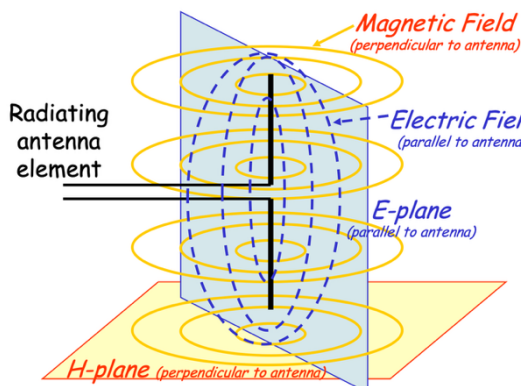
La soluzione è quindi un accordo **CDMA/CA**, dove chi vuole trasmettere richiede al destinatario un *request to send* (RTS). Se il destinatario è libero invia un CTS (*clear to send*), requisendo il canale di comunicazione a tutti gli effetti e dicendo a chiunque si trova nella sfera di influenza del destinatario di stare in silenzio. In questo scenario B non inizierà la comunicazione verso D, rimarrà in silenzio.



Approfondimenti

Capitolo 6

Polarizzazione



H-plane: campo magnetico. In questo caso è perpendicolare all'antenna.

E-plane: campo elettrico. In questo caso è parallelo all'antenna.

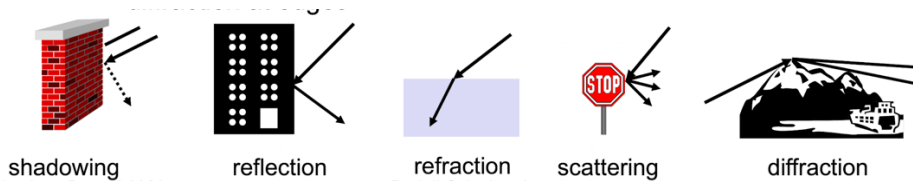
Antenne diverse, posizionate in modo diverso, permettono di leggere meglio o peggio le radiofrequenze che sono state trasmesse su un asse in particolare.

Caso Costruttivo e Caso Distruttivo

Quando viene trasmesso un segnale radio, questo segnale subisce dei rimbalzi, e disturbi e può arrivare in modo diverso all'antenna. Se la somma dei segnali radio è sottrattiva, l'ampiezza della sinusoide è minore, dunque il segnale letto sarà meno forte (ha meno energia).

Soluzione: si spostano le antenne di una certa lunghezza d'onda.

Tipologie di Disturbi



RSSI

Received Signal Strength Indicator. È una misura non standard, cambia da produttore a produttore e non è indicativa. Nella fase di acquisto dei dispositivi, oltre a RSSI (ultima caratteristica da guardare) bisogna considerare innanzitutto il Signal Detection Limit, il bitrate, e i dBm in cui si raggiunge il massimo bitrate.